408 详解 · 2019 年选择题 · 升级版

本册为升级版：每题含完整题干与选项（"题干与选项"节），可脱离原卷独立使用；讲解重心放在方法、推理、易混点上。

只含选择题第 1–40 题，不含综合应用题。

每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。

说明：凡题干问"错误的是"的题（第 12、14、16、20、23、28、40 题），第 4 节说明正确选项（即那个错述）为什么成立，第 5 节说明其余选项为什么是对的（配例子），这样 6 节结构全卷统一。

默认读者是零基础跨考学生：不默认任何 408 前置知识，每题自成体系，不依赖其他题的解析；推理和计算的每一步都写出，不跳步；错误选项的例子选最直观的小数字、小例子、生活类比。

本版语言以大白话为主：每个术语第一次出现都配一句大白话解释；顺序按"从具体到抽象"——先讲生活例子、小数字、具体场景，让人先懂意思，再贴术语名（考试卷子用的就是术语名，认得它也是必须的）。

本版讲解比 2022 版更深：考查知识点与错误选项两节把相关定义、适用条件、边界情况、易混对子讲到位；计组、寻址、流水线等弱项处多讲一步。

图题说明：第 5、38 题题干依赖图片，图片未能直接查看，这 2 题标注"部分核对"，答案按答案表写，图中细节以原卷为准；第 31 题是地址结构表题，表格结构已给出。

升级版说明：每题在原有 6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）；原有题干、选项、答案与 6 节内容未作改动。

第 1 题 · 循环时间复杂度（正确答案：B）

题干与选项：

设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是（ ）。

```
x = 0;
while (n >= (x + 1) * (x + 1))
    x = x + 1;
```

- A. O(log n)
- B. O(n^(1/2))
- C. O(n)
- D. O(n^2)

题意概括：一段 while 循环，循环变量从 0 开始每次加 1，循环条件里有个平方关系，问这段程序的时间复杂度是哪个量级。

入手点： 循环跑多少次，关键看循环变量长到多大才停；条件里是平方，那就反过来想：变量长到平方超过 n 就停。

题干专有名词从零解释：

- 先想"问题规模"：题目里的 n 就是问题有多大，比如数据有多少个；时间复杂度就是看程序跑的时间怎么跟着 n 变大。
- 再想"时间复杂度"：用 O(括号里) 表示时间长大的快慢，只看量级，不管具体系数。比如 O(n) 是"规模翻倍、时间也翻倍"，O(n^(1/2)) 是"规模变 100 倍、时间变 10 倍"。
- 再想"while 循环"：先看条件，条件成立就进循环体跑一遍，跑完再看条件，直到条件不成立才出来。

考查知识点：

- 先想循环变量怎么长：x 从 0 开始，每轮加 1；第 k 轮跑完，x 就是 k。
- 再想什么时候停：条件是 (x+1) 的平方不超过 n 才继续；一旦 (x+1) 的平方超过 n，循环就停了。也就是说，x 长到大约根号 n 就停。
- 再想次数：循环次数大约就是根号 n，所以时间复杂度是 O(n^(1/2))。拿小数字验算：n=100 时，x 长到 10 就停（因为 11 的平方 121 已经超过 100），跑 10 次；n=10000 时，x 长到 100 才停，跑 100 次。规模变 100 倍，次数变 10 倍，正好是平方根关系。
- 边界情况：n=0 时，(0+1) 的平方是 1，已经超过 0，循环一次都不跑，次数是 0——不影响量级。

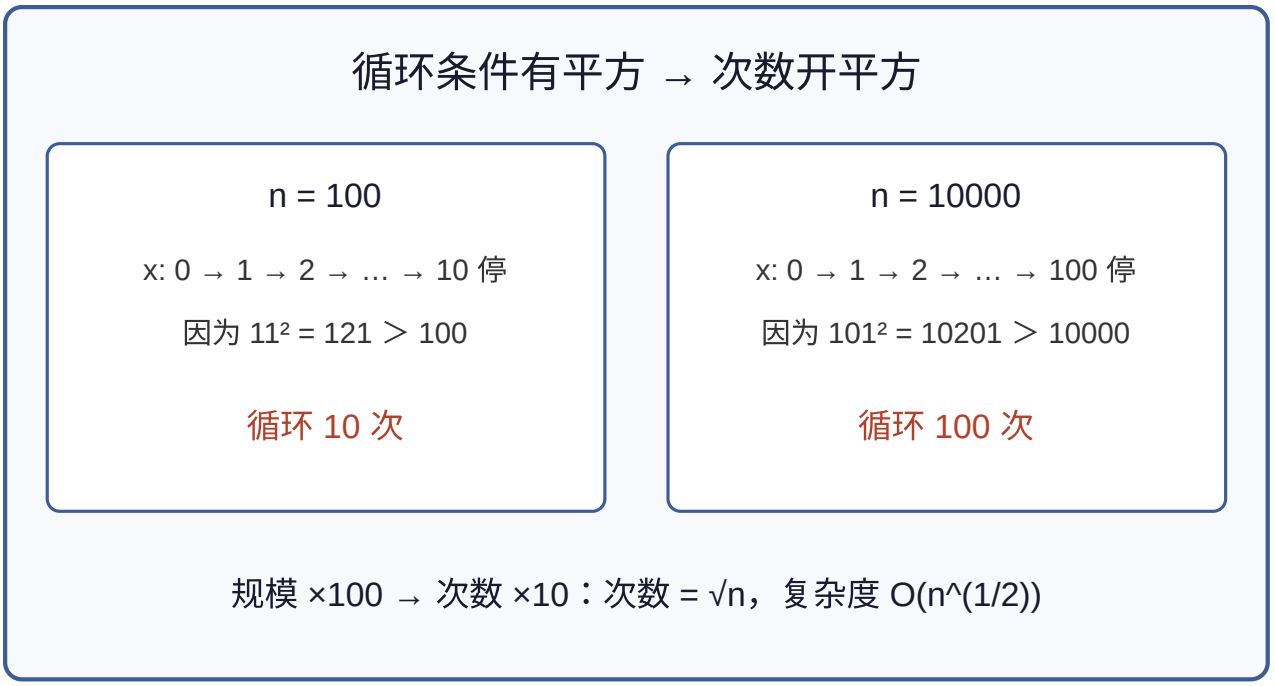
正确选项为什么对： 循环次数大约是根号 n，时间复杂度就是 O(n^(1/2))，选 B。

错误选项为什么错＋举例：

- A（O(logn)）：对数是每次翻倍或者减半才有的量级，比如二分查找每次砍掉一半。本题是每次加 1，不是翻倍也不是减半，凑不上对数。
- C（O(n)）：那是循环变量一路长到 n 才停。本题平方一到 n 就停，早得多，次数只是根号 n。
- D（O(n^2)）：平方量级是两层循环、每层都到 n。本题只有一层循环，套不上。

笔记本一句话： 第 1 题＋循环条件里有平方，反推变量长到根号 n 就停，时间复杂度 O(n^(1/2))。

【图像】



循环条件含平方：规模变100倍、循环次数只变10倍

【定位】 数据结构 · 第1章绪论 · 算法时间复杂度

【易混对比】

- "O(log n)"和"O(n^(1/2))"是两回事：对数是变量每次翻倍或减半才有的量级，比如二分查找每次砍掉一半；本题变量每次只加 1，和翻倍减半不沾边，凑不上对数。
- "O(n)"和"O(n^(1/2))"是两回事：O(n) 是循环变量一路长到 n 才停；本题条件里有平方，变量长到根号 n 就提前停了，次数少得多。
- "O(n^(1/2))"和"O(1)"是两回事：根号 n 会跟着 n 变大，n 越大循环越多；O(1) 是 n 再大次数也不变。
- "一层循环"和"两层循环"是两回事：两层循环每层都到 n 才是 O(n^2)；本题只有一层循环，套不上平方量级。

循环条件里有平方，就反推"变量长到多大停"：次数是根号 n，时间复杂度 O(n^(1/2))。

【坑位】

别一看到循环就写 O(n)，先看条件里有没有平方、开方这类"提前刹车"。

最容易踩的坑是：看到 while 里变量每次加 1，就条件反射写 O(n)，把条件里的平方当摆设。为什么错：循环次数不是"变量从 0 长到 n"，而是"长到条件不成立为止"；条件是平方不超过 n，变量长到根号 n 就停了，后面那么多数根本轮不到。正确做法：先把停止条件反解出来——(x+1) 的平方超过 n 就停，所以 x 大约长到根号 n，次数就是根号 n；再拿小数字验算一遍（n=100 跑 10 次，n=10000 跑 100 次），规模变 100 倍、次数变 10 倍，对上才落笔。

【套路】

- 第一步，看变量怎么长：x 从 0 开始，每轮加 1，第 k 轮跑完 x 就是 k。
- 第二步，看条件何时不成立：条件是 (x+1) 的平方不超过 n 才继续，反解出来 x 长到大约根号 n 就停。
- 第三步，次数等于终值：变量从 0 长到根号 n，循环次数就是根号 n，时间复杂度 O(n^(1/2))。

- 第四步，小数字验算：n=100 跑 10 次，n=10000 跑 100 次，规模变 100 倍、次数变 10 倍，确认是平方根关系。

先看增长、再看停止条件、反推次数、最后验算，平方条件必反推。

【类比】

像水桶接水一样：水龙头每秒滴一滴（变量每次加 1），桶的容量是 n 升，但桶有个怪规矩——水位计显示的是"当前滴数"的平方，平方一超过 n 就自动关阀。桶其实只接到根号 n 滴水就停了，根本装不满 n 滴。时间复杂度问的就是"阀门开多久"，也就是"接到多少滴"，答案是根号 n 滴。

【口诀】 条件有个平方项，循环次数开平方。

【考频】 「时间复杂度」在 2009–2022 年解析中出现 12 次 · 高频

第 2 题 · 树转二叉树、遍历序列与后根遍历对应（正确答案：B）

题干与选项：

若将一棵树 T 转化为对应的二叉树 BT，则下列对 BT 的遍历中，其遍历序列与 T 的后根遍历序列相同的是（）。

- A. 先序遍历
- B. 中序遍历
- C. 后序遍历
- D. 按层遍历

题意概括：一棵普通的树按孩子兄弟的方法变成二叉树，问二叉树的哪种遍历顺序，得到的序列和原来树的后根遍历序列一样。

入手点： 先把"树怎么变二叉树"想清楚，再对遍历；记住一对对应关系：树的先根对应二叉树先序，树的后根对应二叉树中序。

题干专有名词从零解释：

- 先想"树"：分层的结点，根在最上面，根下面有孩子，孩子下面还有孩子，像家谱。
- 再想"二叉树"：每个结点最多两个孩子，左边那个叫左孩子，右边那个叫右孩子。
- 再想"孩子兄弟表示法"：把树变成二叉树的办法——左指针指第一个孩子，右指针指下一个兄弟。这样一转，原来的树就变成了一棵二叉树。
- 再想"遍历"：把所有结点按某种顺序排成一列。树的先根是"先根、再孩子从左到右"；树的后根是"先孩子从左到右、最后根"。二叉树的先序是"根、左、右"，中序是"左、根、右"，后序是"左、右、根"，按层遍历是一层一层从上到下、从左到右数。

考查知识点：

- 先想孩子兄弟法怎么对应：左孩子就是原树的第一个孩子，右孩子就是原树的下一个兄弟。
- 再想为什么后根对中序：二叉树的中序是"左子树、根、右子树"；左子树里装着原树的孩子们，右子树里装着原树的兄弟们。排出来的顺序正好是"孩子们先、根最后"，和树的后根一模一样。
- 拿小例子验算：根 A，孩子 B、C，B 的孩子 D。树后根：D、B、C、A。二叉树是 A 的左孩子 B，B 的右孩子 C，B 的左孩子 D；中序走下来：D、B、C、A，对上了。
- 对照先根：树的先根是 A、B、D、C，对应二叉树先序 A、B、D、C，也是对的——先根对先序、后根对中序，记一对就够。

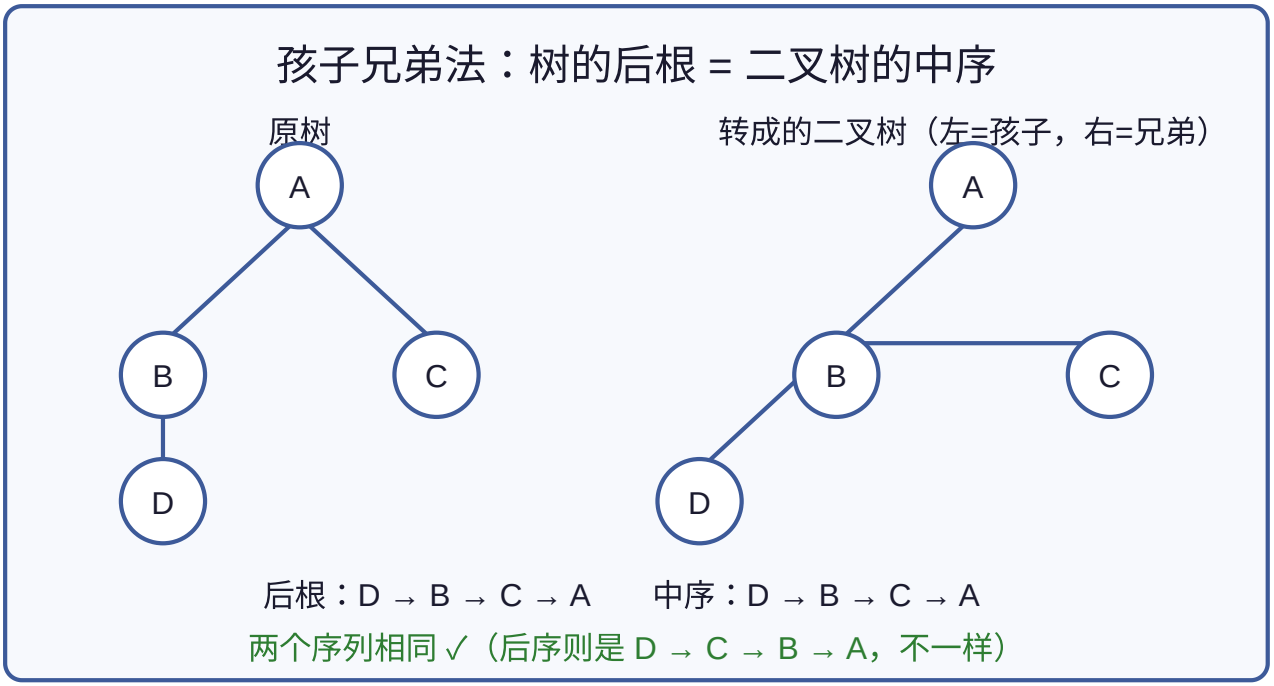
正确选项为什么对： 二叉树的中序遍历序列和树的后根遍历序列相同，选 B。

错误选项为什么错＋举例：

- A（先序遍历）：先序对应的是树的先根（A、B、D、C），不是后根。
- C（后序遍历）：后序是左、右、根，小例子里走出来是 D、C、B、A，和后根 D、B、C、A 不一样（B、C 位置换了）。
- D（按层遍历）：按层是一层一层数，小例子里是 A、B、C、D，和深度优先的后根完全不是一回事。

笔记本一句话： 第 2 题＋树转二叉树：先根对先序，后根对中序，记一对就够。

【图像】



孩子兄弟法转二叉树：树的后根遍历序列等于二叉树的中序遍历序列

【定位】 数据结构 · 第5章树与二叉树 · 树的存储结构与遍历

【易混对比】

- "树的后根遍历"和"二叉树的后序遍历"是两回事：名字里都有个"后"，但树的后根对应的是二叉树的中序（左、根、右），不是后序；小例子里后根走出来 D、B、C、A，后序走出来 D、C、B、A，B 和 C 位置不一样。
- "树的先根遍历"和"二叉树的先序遍历"是一对：这对名字也像，顺序也真一样（A、B、D、C），先根对先序、后根对中序，记一对就够。
- "孩子兄弟表示法"和"双亲表示法"是两回事：孩子兄弟法是左指针指第一个孩子、右指针指下一个兄弟，专门用来把树变成二叉树；双亲表示法是数组里存每个结点的父亲下标，不做这种转换。
- "按层遍历"和"深度优先遍历"是两回事：按层是一层一层横着数（A、B、C、D）；后根、中序这些都是一条道走到黑的深度优先，横竖两种数法结果完全不同。

孩子兄弟法转成二叉树后：树的先根等于二叉树先序，树的后根等于二叉树中序。

【坑位】

"后根"和"后序"只差一个字，但对应关系完全不同，别凭字面猜。

最容易踩的坑是：看到"后根"就顺手选"后序"，觉得名字像就是一回事。为什么错：孩子兄弟法转完之后，左子树里装的是原树的孩子们，右子树里装的是原树的兄弟们；二叉树的中序是"左、根、右"，排出来正好是"孩子们先、根最后"，这才和树的后根"先孩子、最后根"对上。后序是"左、右、根"，兄弟们排到了根前面，顺序就乱了。正确做法：别记名字，记结构——左是孩子、右是兄弟，中序"左根右"就是"孩子、根、兄弟"，和后根的"孩子们先、根最后"一拍即合；拿三四个结点的小例子走一遍，对上才选。

【套路】

- 第一步，画出转换：左指针连第一个孩子，右指针连下一个兄弟，把树变成二叉树。

- 第二步，中序走一遍：按"左、根、右"把二叉树走一遍，得到序列。
- 第三步，后根走一遍：原树按"先孩子从左到右、最后根"走一遍，两个序列对上就确认。

左是孩子、右是兄弟，中序"左根右"即后根"孩子先、根最后"。

【类比】

像把家谱翻译成只有"长子"和"弟弟"两种关系的版本：规定每个人只记两个指针——左手牵第一个孩子，右手牵下一个弟弟。翻译完再按"先看左手边所有人、再看自己、最后看右手边所有人"的顺序点名，点出来的名单，和原来家谱按"孩子们先报、家长最后报"的点名名单一模一样——这就是后根等于中序。

【口诀】 先根对先序，后根对中序；左孩右兄弟，中序即后根。

【考频】 「后根遍历」在 2009–2022 年解析中出现 14 次 · 高频

第 3 题 · 哈夫曼树结点数求符号数（正确答案：C）

题干与选项：

对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有 115 个结点，则 n 的值是（ ）。

- A. 56
- B. 57
- C. 58
- D. 60

题意概括：用互不相同的符号做哈夫曼编码，生成的哈夫曼树一共有 115 个结点，问原来有几个符号。

入手点： 哈夫曼树的结点数和叶子数有个固定关系：总结点等于 2 倍叶子数减 1；叶子就是符号，倒推回去。

题干专有名词从零解释：

- 先想"哈夫曼编码"：出现越多的符号编越短的码，这样总长度最省，省空间。
- 再想"哈夫曼树"：一棵二叉树，做法是每次把最小的两个结点合并成一个新结点，一直合到只剩一个根。
- 再想"叶子结点"：没有孩子的结点；这里一个叶子就是一个符号。
- 再想"结点数"：树里所有结点的个数，叶子加合并出来的内部结点全算上。

考查知识点：

- 先想为什么是 2n−1：哈夫曼树是"满二叉树"，每个非叶子结点恰好有两个孩子。设叶子 n 个，合并出来的内部结点就是 n−1 个（每合并一次少一个结点，从 n 个叶子合到 1 个根，要合 n−1 次），总共 $n + (n-1) = 2n-1$ 个。
- 再想倒推：$115 = 2n-1$，115+1=116，116÷2=58。
- 拿小例子验算：3 个符号，树有 5 个结点（3 个叶子加 2 个合并出来的）；5 个符号，树有 9 个结点。规律就是 2n−1。
- 边界情况：符号互不相同，保证每个符号占一个叶子，不影响公式。

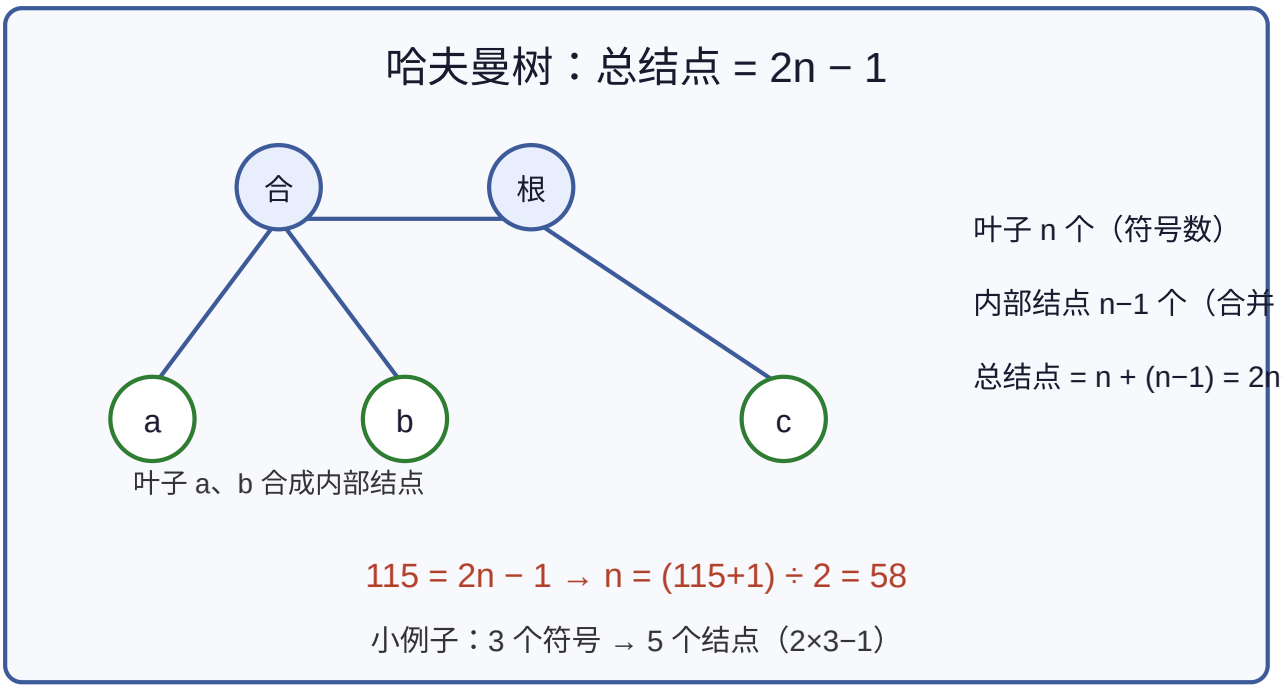
正确选项为什么对： 2n−1=115，解出来 n=58，选 C。

错误选项为什么错＋举例：

- A（56）：115 除以 2 得 57 余 1，没把"减 1"那部分处理好，或者公式记成了 2n+1 再倒推。
- B（57）：(115−1)÷2=57，这是把 115 当成了 2n+1 的结果，公式记反了——应该是 115+1 再除以 2。
- D（60）：凑的整十数，和 115 对不上（2×60−1=119，不是 115）。

笔记本一句话： 第 3 题＋哈夫曼树总结点 2n−1，115 就倒推 n=58。

【图像】



哈夫曼树：叶子 n 个、内部结点 n−1 个，总结点 2n−1

【定位】 数据结构 · 第5章树与二叉树 · 哈夫曼树

【易混对比】

- "2n−1"和"2n+1"是两回事：n 是叶子数（符号数），内部结点是 n−1 个，加起来是 2n−1；记成 2n+1 的，倒推时会算出 57，直接掉坑。
- "叶子数"和"总结点数"是两回事：题目给的是总结点 115，要的是叶子数（符号数），必须倒推，不能直接拿 115 当答案。
- "哈夫曼树"和"一般的二叉树"是两回事：2n−1 这个公式只对哈夫曼树这种"满二叉树"（每个非叶子结点恰好有两个孩子）成立，一般的二叉树结点数和叶子数没有这个固定关系。
- "内部结点数 n−1"和"合并次数 n−1"是同一件事的两面：每合并一次少一个结点、产生一个内部结点，从 n 个叶子合到 1 个根，合 n−1 次，内部结点就是 n−1 个。

哈夫曼树总结点等于 2n−1（n 是叶子数即符号数），115 倒推：(115+1)除以2等于58。

【坑位】

倒推时先加 1 再除以 2，直接除以 2 会掉进 57 的坑。

最容易踩的坑是：115 除以 2 约等于 57，顺手选 57。为什么错：公式是总结点等于 2n−1，解 n 要先把"减 1"挪到左边变成"115+1=2n"，再除以 2 得 58；直接除以 2 等于默认公式是 2n+1，符号记反了。正确做法：默写一遍公式 2n−1，移项得 n=(总结点+1)除以2，代入 (115+1)除以2=58；再拿小例子验算——3 个符号对应 5 个结点（2×3−1），公式方向对了才放心。

【套路】

- 第一步，确认是哈夫曼树：满二叉树，每个非叶子结点恰有两个孩子，才能用 2n−1。
- 第二步，列公式移项：总结点=2n−1，推出 n=(总结点+1)除以2。
- 第三步，代入计算：(115+1)除以2=116除以2=58。

- 第四步，小例子验算：3 个符号对应 5 个结点，公式方向无误。

先加 1 再除以 2，2n−1 倒推别搞反符号。

【类比】

像两两组队打比赛：n 支队伍（叶子），每轮两两合并成一队（产生一个内部结点），比到只剩冠军要打 n−1 轮。所有"队伍"加"合并产生的新队"一共 n+(n−1)=2n−1 支。题目告诉你一共 115 支队伍（含新队），倒回去：(115+1)除以2=58 支原始队伍。

【口诀】 哈夫曼树数结点，二 n 减一；倒推先加一，再除以二。

【考频】 「哈夫曼树」在 2009–2022 年解析中出现 6 次 · 中频

第 4 题 · AVL 树删点再插回、树是否相同（正确答案：A）

题干与选项：

在任意一棵非空平衡二叉树（AVL 树）T1 中，删除某结点 v 之后形成平衡二叉树 T2，再将 v 插入 T2 形成平衡二叉树 T3。下列关于 T1 与 T3 的叙述中，正确的是（）。

Ⅰ. 若 v 是 T1 的叶结点，则 T1 与 T3 可能不相同

Ⅱ. 若 v 不是 T1 的叶结点，则 T1 与 T3 一定不相同

Ⅲ. 若 v 不是 T1 的叶结点，则 T1 与 T3 一定相同

- A. 仅 Ⅰ
- B. 仅 Ⅱ
- C. 仅 Ⅰ、Ⅱ
- D. 仅 Ⅰ、Ⅲ

题意概括：一棵平衡二叉树，删掉一个结点再插回去，得到的新树和原来的树比，有三句话，问哪几句对。

入手点： 先分两种情况：删的是叶子，还是不是叶子；再想"删了再插"会不会触发旋转——旋转一动，树的形状就可能回不去了。

题干专有名词从零解释：

- 先想"平衡二叉树（AVL 树）"：一棵二叉排序树，并且任意结点的左右子树高度差不超过 1；太斜了就自动转一下恢复平衡。
- 再想"叶结点"：没有孩子的结点，树最底下那一层。
- 再想"删除、插入"：删就是把结点拿掉，拿掉后可能要调整；插就是按大小找到位置放进去，放完可能也要调整。
- 再想"旋转"：树太斜了就转一下，分左旋、右旋；转完高度恢复平衡，但结点之间的父子关系变了。

考查知识点：

- 先想删叶子：删完可能失衡，一失衡就要旋转；转完形状变了，再把原来那个结点插回去，插的位置是按旋转后的形状找的，不一定回到原样。所以"删叶子再插回可能不相同"是对的——"可能"二字是关键：也可能相同，但"可能不相同"这个说法成立。
- 再想删非叶子：删除时要用前驱或者后继顶替被删结点的位置，树结构已经动了；再插回原来那个结点，能不能回到原样？不一定——可能恰好回去，也可能回不去。
- 再想"一定"这种词：带"一定"的说法，举出一个反例就倒了。删非叶再插回是"可能相同、可能不同"，所以"一定不相同"不对，"一定相同"也不对。
- 小例子：删一个叶子引起一次旋转，插回后旋转不会自动 undo，形状就和原来不一样了——这就是"可能不相同"的一个实例。

正确选项为什么对： 只有第 Ⅰ 句对（删叶子再插回可能不相同）；第 Ⅱ 句"一定不相同"太绝对，第Ⅲ句"一定相同"也太绝对，都不对。所以只选 Ⅰ，选 A。

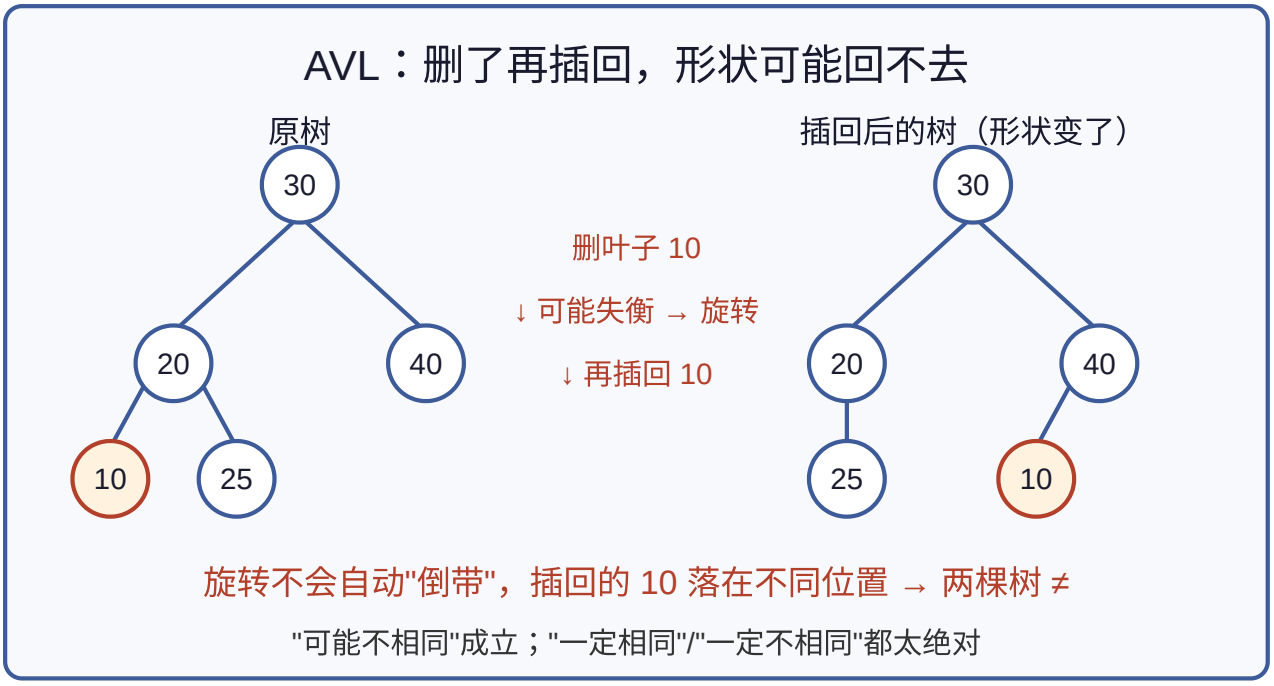
错误选项为什么错＋举例：

- B（仅 Ⅱ）：Ⅱ 的"一定不相同"不对，删非叶再插回也可能恰好回到原样（比如删除顶替后又转回去了），不能说一定。

- C（仅 I 、 II）： II 错，见上； I 对、 II 错，合起来就不对。
- D（仅 I 、 III）： III的"一定相同"不对，中间旋转过就回不去了。

笔记本一句话： 第 4 题＋AVL 删了再插：带"一定"的先怀疑，删叶可能回不去，非叶也是可能同可能不同。

【图像】



AVL 树删叶子再插回：中间一次旋转，形状就可能回不去了

【定位】 数据结构 · 第7章查找 · 平衡二叉树

【易混对比】

- "可能不相同"和"一定不相同"是两回事："可能"是存在一种删完插回不一样的情况，这句话就成立；"一定"是说所有情况都不一样，举出一个能回去的例子就倒了。
- "删叶子"和"删非叶子"是两回事：删叶子是直接拿掉；删非叶子要用前驱或后继顶替位置，树结构动得更多，两种情况都要单独想。
- "AVL 树"和"普通二叉排序树"是两回事：AVL 树失衡会自动旋转调整，旋转一动父子关系就变了；普通二叉排序树没有这一步，删了再插更容易回去。
- "旋转"和"插入删除"是两回事：插入删除是按大小找位置，旋转是失衡时的救场动作；救场动了结构，但不会"自动撤销"，所以插回去不一定复原。

删了再插回"可能相同、也可能不同"：带"一定"的说法先怀疑，本题只有第 I 句对。

【坑位】

看到"删了再插回去"就默认"肯定复原"，漏了中间可能发生旋转。

最容易踩的坑是：觉得删掉再插回同一个值，树当然和原来一样。为什么错：删完那一刻树可能失衡，AVL 会自动旋转恢复平衡，旋转改变了父子关系；再把结点插回去时，是按旋转后的形状找位置，旋转不会"倒带"，形状就回不去了。正确做法：分两种情况想——删叶子：删完可能失衡旋转，插回可能不一样（"可能"二字让第 I 句

成立）；删非叶子：顶替加旋转，回去还是不回去都有可能，所以"一定不相同"和"一定相同"都太绝对；凡是带"一定"的选项，先找反例。

【套路】

- 第一步，圈定语：把"一定""可能"圈出来，"一定"类说法优先怀疑。
- 第二步，分情况讨论：删叶子一种情况，删非叶子（前驱或后继顶替）一种情况。
- 第三步，看旋转动没动：中间只要发生过一次旋转，形状就可能回不去，"一定相同"不成立；也可能恰好回去，"一定不相同"也不成立。

圈定语、分情况、看旋转，"一定"二字先找反例。

【类比】

像拆积木塔再搭回去：塔有点斜（失衡），你抽掉一块（删叶子），塔晃了一下、有人顺手扶正了一下（旋转），扶正时几块积木的上下关系变了。你再把那块积木按原来的记忆塞回去，塔的形状已经和原来不一样了——扶正的动作不会自动倒带。

【口诀】 删了再插回，形状未必回；旋转一出手，复原别指望；"一定"先怀疑，反例找一找。

【考频】 「AVL树」在 2009–2022 年解析中出现 10 次 · 高频

第 5 题 · AOE 网活动 d 最早／最迟开始时间（正确答案：C）

题干与选项：

下图所示的 AOE 网表示一项包含 8 个活动的工程。活动 d 的最早开始时间和最迟开始时间分别是（ ）。

- A. 3 和 7
- B. 12 和 12
- C. 12 和 14
- D. 15 和 15

> 注：本题题干依赖图片（AOE 网图），图片未能直接查看，本题标注"部分核对"。答案按答案表写；b、c 两条边的具体起止顶点未看清，图中细节以原卷为准。

题意概括：一张 AOE 网表示的工程，一共 8 个活动，问其中活动 d 的最早开始时间和最迟开始时间。

入手点： 活动的开始时间看它起点那个事件：最早开始就是起点事件的最早发生时间；最迟开始就是终点事件的最迟发生时间减去活动持续时间。

题干专有名词从零解释：

- 先想"AOE 网"：边表示活动、点表示事件的网；事件就是"前面活动都做完"那个时刻，活动就是边上那段工作，边上标着持续时间。
- 再想"最早发生时间"：从起点到这个事件的最长路径——因为一个事件要等前面所有活动都完，才能发生，所以取最长的那条路。
- 再想"最迟发生时间"：在不拖总工期的前提下，这个事件最晚能到；从终点往前倒推，总工期减去后面最长路径。
- 再想"关键路径"：最长的那条路，它的长度就是总工期；关键路径上的活动没有松弛时间，拖一天总工期就拖一天。
- 再想"最早开始时间"和"最迟开始时间"：活动能开始的最早时刻，和再晚就拖总工期的时刻；两者之差就是这个活动的松弛时间。

考查知识点：

- 先想怎么算最早：从起点往后推，每个事件的最早发生时间等于前面所有路径中最长的那条；活动的最早开始就是它起点事件的最早发生时间。
- 再想怎么算最迟：从终点往前倒推，每个事件的最迟发生时间等于总工期减去它到终点的最长路径；活动的最迟开始就是它终点事件的最迟发生时间减去活动自己的持续时间。
- 再想小例子：三个点一条链，活动 3 天、5 天，总工期 8 天；第二个活动的起点事件最早是 3，最迟也是 3，松弛是 0——这就是关键活动。
- 再想本题：对照原图算，活动 d 从事件②到事件④，持续 7 天；②的最早发生时间是 12，④的最迟发生时间是 21。所以 d 最早开始 12，最迟开始 21−7=14，有 2 天松弛，d 不在关键路径上。

正确选项为什么对： 最早开始 12，最迟开始 14，选 C。

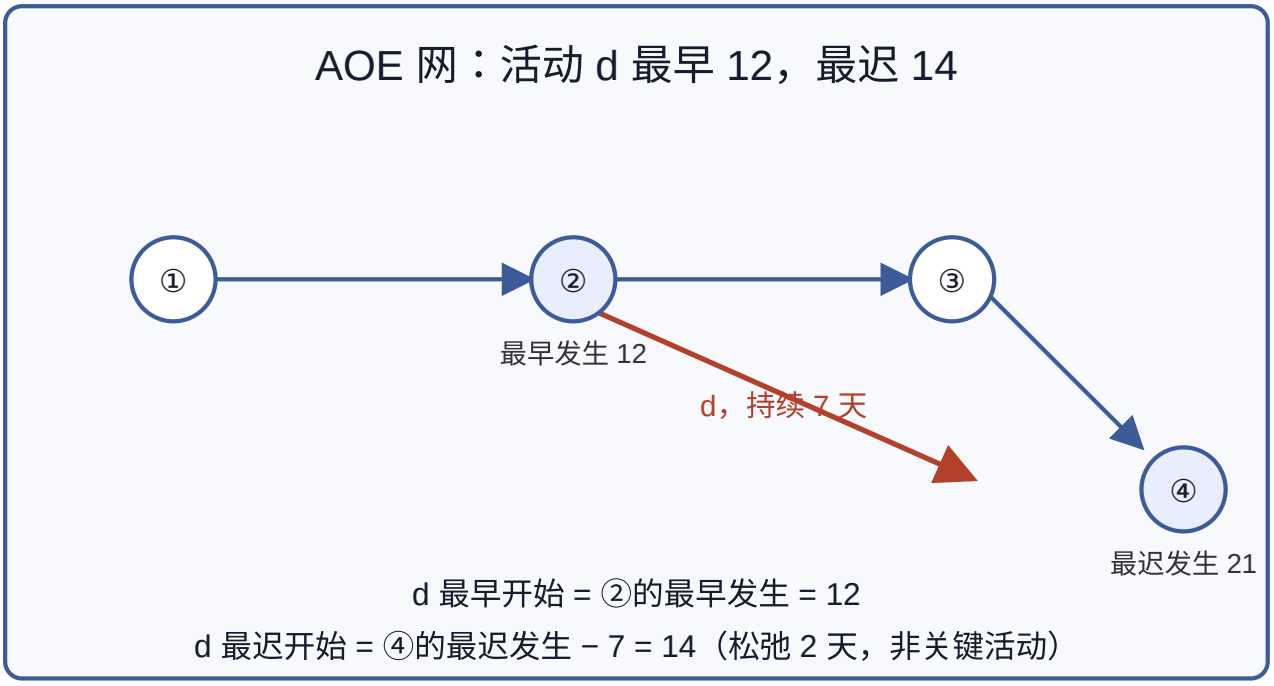
错误选项为什么错＋举例：

- A（3 和 7）：7 正好是 d 自己的持续时间，把"持续 7 天"当成了"最迟开始 7"，概念混了；3 更小，不可能是从起点到②的最长路径（②的最早是 12）。

- B（12 和 12）：最早开始 12 是对的，最迟开始如果也写 12，等于说 d 的最迟等于最早、松弛是 0，也就是把 d 当成了关键活动；但 d 的最迟是 14，有 2 天松弛，所以不对。
- D（15 和 15）：两个数相等且都比正确值大，事件时间算错了不说，还把最迟当成了最早（松弛写成了 0）。

笔记本一句话： 第 5 题＋AOE 活动时间：最早开始看起点事件的最早发生时间，最迟开始等于终点事件的最迟发生时间减持续时间；d 是 12 和 14，有 2 天松弛。

【图像】



AOE 网：活动最早开始看起点事件，活动最迟开始等于终点事件最迟发生减持续时间

【定位】 数据结构 · 第6章图 · AOE 网与关键路径

【易混对比】

- "活动的最早开始时间"和"事件的最早发生时间"是两回事：事件是图里的点（"前面都做完"的时刻），活动是边（一段工作）；活动的最早开始等于它起点事件的最早发生，本题 d 的起点是事件②，所以是 12。
- "活动的最迟开始时间"和"事件的最迟发生时间"是两回事：差了一个"持续时间"——活动最迟开始等于终点事件最迟发生减去活动持续时间，本题是 21−7=14，不能直接拿 21 当答案。
- "关键活动"和"非关键活动"是两回事：关键活动最早开始等于最迟开始（松弛时间为 0），拖一天总工期拖一天；d 的最早 12、最迟 14，有 2 天松弛，不是关键活动。
- "AOE 网"和"AOV 网"是两回事：AOE 网是边表示活动、点表示事件；AOV 网是点表示活动，考关键路径的一定是 AOE 网。

活动最早开始等于起点事件的最早发生；活动最迟开始等于终点事件的最迟发生减去持续时间。

【坑位】

最迟开始不是事件的最迟发生，别忘了减去活动自己的持续时间。

最容易踩的坑有两个：一是把 d 自己的持续时间 7 当成最迟开始，二是把终点事件的最迟发生 21 直接当最迟开始。为什么错：事件的最迟发生是"这个时刻点最晚能到"，而活动还要占 7 天时间才能做完，活动最晚必须

在"21−7=14"这个时刻开工，否则做完就超过 21，拖了总工期。正确做法：先正推所有事件的最早发生（取最长路径），再倒推所有事件的最迟发生（总工期减去到终点的最长路径），最后活动最早看起点、活动最迟用终点减持续；算完检查一下：最迟开始大于等于最早开始，差值就是松弛时间。

【套路】

- 第一步，正推事件最早：从起点往后，每个事件的最早发生等于前面所有路径中最长的那条。
- 第二步，倒推事件最迟：从终点往前，每个事件的最迟发生等于总工期减去它到终点的最长路径。
- 第三步，活动最早看起点：活动的最早开始等于起点事件的最早发生（d 的起点是②，得 12）。
- 第四步，活动最迟用终点减：活动的最迟开始等于终点事件的最迟发生减去持续时间（21−7=14）。

正推最早、倒推最迟，活动最早看起点、最迟终点减持续。

【类比】

像接力赛排班：d 这一棒从第②个交接点跑到第④个交接点，要跑 7 分钟。第②个交接点最早 12 点才能交棒（前面几棒最慢的一棒 12 点才到），所以 d 最早 12 点开跑；第④个交接点最晚 21 点必须交出下一棒，d 自己要跑 7 分钟，所以最晚 14 点必须开跑，再晚就赶不上 21 点的交接。

【口诀】 最早看起点，最迟终点减；松弛是差值，关键零松弛。

【考频】 「AOE网」在 2009–2022 年解析中出现 5 次 · 中频

第 6 题 · 有向无环图表示表达式最少顶点数（正确答案：A）

题干与选项：

用有向无环图描述表达式 (x+y)((x+y)/x)，需要的顶点个数至少是（ ）。

- A. 5
- B. 6
- C. 8
- D. 9

题意概括：用有向无环图表示一个表达式，相同的子表达式只画一次，问最少需要几个顶点。

入手点： 先把表达式拆开：两个 (x+y) 是一样的，只画一次；再数不同的"零件"有几种。

题干专有名词从零解释：

- 先想"有向无环图（DAG）"：边有方向、没有圈的图；画表达式时，边表示"这个结果是那个运算的输入"。
- 再想"表达式"：x、y 是变量，+、/、× 是运算；(x+y) 乘上 ((x+y) 除以 x)，整个就是一个表达式。
- 再想"顶点"：图里的点；这里一个顶点就是一个变量或者一次运算。
- 再想"共享"：相同的子表达式只保留一份，重复出现的地方都指向同一份。

考查知识点：

- 先想拆开数：变量 x、y（2 个）；运算 +（1 个）、÷（1 个）、×（1 个）。两个 (x+y) 完全相同，在 DAG 里合成一个 + 顶点，所以总共 2+3=5 个。
- 再想不共享是多少：按表达式树数是 9 个——根 ×，左孩子 +，右孩子 ÷；左 + 的孩子 x、y；÷ 的孩子 + 和 x；第二个 + 的孩子 x、y。数一遍：× 1 个、+ 2 个、÷ 1 个、x 3 个、y 2 个，一共 9 个。共享后：×、+、÷、x、y，一共 5 个。省了 4 个：第二个 (x+y) 那 3 个结点（+、x、y）省了，再加上 ÷ 的右孩子 x 和左边的 x 是同一个变量、DAG 里变量也只画一次，又省 1 个，9−4=5。
- 再想小例子：(a+b)×(a+b) 不共享是 7 个结点（×、两个 +、两个 a、两个 b），共享后是 4 个（×、+、a、b）。
- 教训：DAG 的"最少"就是找重复子表达式合并，变量重复了也要合并。

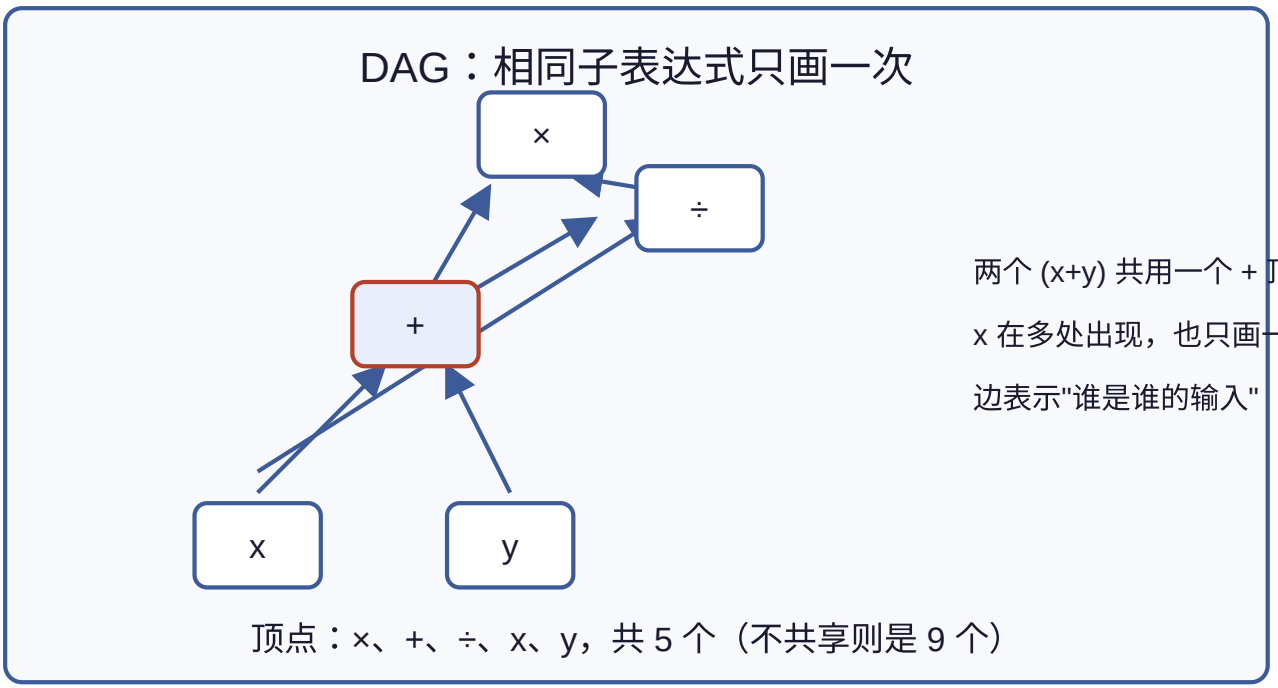
正确选项为什么对： 共享后一共 5 个顶点（x、y、+、÷、×），选 A。

错误选项为什么错＋举例：

- B（6）：多算了一个——可能是第二个 (x+y) 的 x 或者 y 又算了一次，没共享干净。
- C（8）：接近不共享的数，没把共享做彻底（比如只合并了 +，没合并重复的变量）。
- D（9）：就是完全不共享的表达式树结点数，DAG 的"最少"要求共享，所以不对。

笔记本一句话： 第 6 题＋DAG 画表达式：相同的子表达式只画一次，(x+y) 合并后一共 5 个顶点。

【图像】



DAG 表示表达式：相同的子表达式只画一次，变量重复也只画一次

【定位】 数据结构 · 第6章图 · 有向无环图及其应用

【易混对比】

- "DAG"和"表达式树"是两回事：表达式树里相同的子表达式各画各的（本题是 9 个结点）；DAG 里相同的只画一次（本题是 5 个），"最少顶点"考的就是这个共享。
- "顶点最少"和"顶点最多"是两回事：最少是能合并的全合并；最多的画法就是表达式树，一个都不合并。
- "子表达式共享"和"变量共享"是两回事，但都要做：(x+y) 出现了两次要合并，变量 x 在好几个地方出现也只画一个顶点，漏掉任何一处都会多算。
- "顶点"和"边"是两回事：题目数的是顶点（变量或一次运算），边只是表示"谁是谁的输入"，别把边也数进去。

DAG 画表达式：相同的子表达式只画一次，重复的变量也只画一次，本题共享后是 5 个顶点。

【坑位】

合并了 (x+y) 就以为完事，重复出现的变量 x 也要合并，漏了就多算。

最容易踩的坑是：只合并了两个 (x+y)，数出来 6 个，漏了变量 x 的合并。为什么错：DAG 的"最少"要求所有相同的东西都只保留一份——第二个 (x+y) 里的 x、y 和除法右边的 x，和左边的 x、y 是同一个变量，在 DAG 里只能画一次。正确做法：先把表达式拆成零件清单（变量 x、y，运算加、除以、乘），再逐个问"这个零件出现过几次"，出现多次的全部合并成一个顶点；数一遍：乘、加、除以、x、y，一共 5 个。

【套路】

- 第一步，拆零件：把表达式拆成变量和运算的清单。
- 第二步，合并子表达式：完全相同的子表达式只保留一份（两个 (x+y) 合成一个加法顶点）。
- 第三步，合并重复变量：同一个变量在多处出现也只画一个顶点，再数总数。

先拆零件，再合子表达式，最后合变量，一个不漏才最少。

【类比】

像拼乐高：说明书里 (x+y) 这个小组件出现了两次，除法还要单独用一个 x。你不会把相同的组件拼两份，而是拼一份、两处共用；散装的 x 零件也不会多拿，用一颗、几处都插它。最后桌上只剩 5 种零件：乘、加、除以、x、y。

【口诀】 相同只画一遍，变量也合并；先拆再合，数清顶点。

【考频】 「有向无环图」在 2009–2022 年解析中出现 8 次 · 中频

第 7 题 · 选排序算法除时空效率还考虑的因素（正确答案：D）

题干与选项：

选择一个排序算法时，除算法的时空效率外，下列因素中，还需要考虑的是（）。

Ⅰ. 数据的规模 Ⅱ. 数据的存储方式 Ⅲ. 算法的稳定性 Ⅳ. 数据的初始状态

- A. 仅Ⅲ
- B. 仅Ⅰ、Ⅱ
- C. 仅Ⅱ、Ⅲ、Ⅳ
- D. Ⅰ、Ⅱ、Ⅲ、Ⅳ

题意概括：挑排序算法时，除了时间效率和空间效率，还要考虑哪些因素。

入手点： 一个一个因素想：数据多少、存在哪、稳不稳、一开始什么样——每个都想一个"如果不管它会出什么问题"的例子。

题干专有名词从零解释：

- 先想"时间效率"：跑多快，时间复杂度。
- 再想"空间效率"：占多少额外内存，空间复杂度。
- 再想"数据的规模"：数据量大小，十个还是一百万个。
- 再想"存储方式"：顺序存（数组，一格挨一格）还是链式存（链表，一个指下一个）。
- 再想"稳定性"：相等的元素排完相对次序变不变；稳定的算法排完后，原来在前的还是在前。
- 再想"初始状态"：数据一开始是有序、逆序还是乱序。

考查知识点：

- 先想规模：数据量小，简单算法（插入、冒泡）就够了，还省事；数据量大，必须上快速、归并这种快的，不然跑不完。
- 再想存储方式：链式存储没法随机访问，依赖下标跳来跳去的算法（比如快排、堆排）就不好使；链式适合归并这种顺着走的。
- 再想稳定性：比如按成绩排，成绩相同的要保持原来的学号顺序，那就必须用稳定的算法；不稳定的算法会把学号顺序打乱。
- 再想初始状态：数据基本有序时，插入排序、冒泡排序几趟就完事；完全逆序时就慢得多。四个都要考虑。

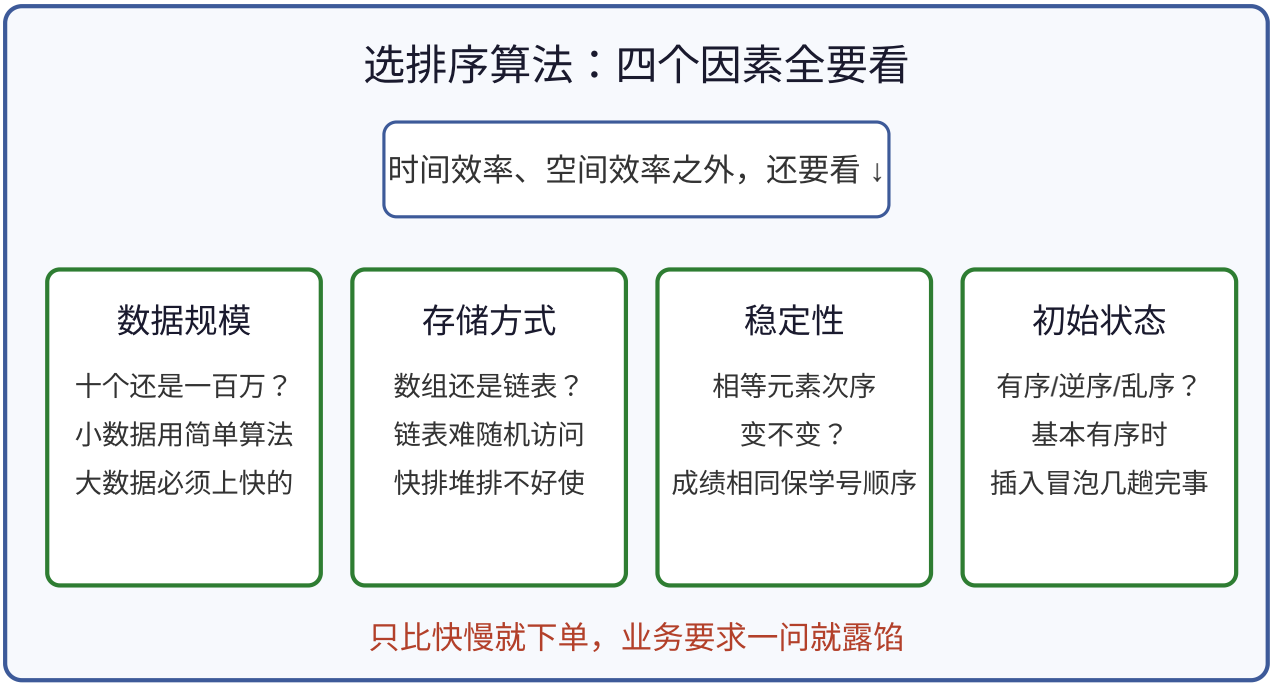
正确选项为什么对： Ⅰ、Ⅱ、Ⅲ、Ⅳ全要考虑，选 D。

错误选项为什么错＋举例：

- A（仅Ⅲ）：只想到稳定性，漏了规模、存储、初始状态——四个里只抓一个。
- B（仅Ⅰ、Ⅱ）：漏了稳定性和初始状态，业务要求（比如成绩相同保学号顺序）和"基本有序"这个常见情况都没顾上。
- C（仅Ⅱ、Ⅲ、Ⅳ）：漏了规模，数据量大小是挑算法时第一个要看的。

笔记本一句话： 第 7 题＋挑排序算法：规模、存储、稳定、初始状态四个全要看，不只看快慢。

【图像】



选排序算法：时间空间效率之外，规模、存储、稳定性、初始状态四项全要看

【定位】 数据结构 · 第8章排序 · 排序算法的选择

【易混对比】

- "稳定性"和"时间效率"是两回事：时间效率是跑多快，稳定性是相等的元素排完相对次序变不变；业务要求（比如成绩相同保持学号顺序）只认稳定性，再快的不稳定算法也白搭。
- "顺序存储"和"链式存储"是两回事：数组能随机跳下标，快排、堆排好使；链表只能顺着走，依赖跳下标的算法就不好使，归并这种顺着走的更合适。
- "内部排序"和"外部排序"是两回事：数据全在内存里是内部排序，看时间空间；数据大到内存装不下是外部排序，还要看读写外存的次数，选法完全不同。
- "平均情况"和"最坏情况"是两回事：快排平均很快、最坏退化成 O(n^2)；数据规模大又怕最坏，就得换堆排或归并，不能只看平均。

挑排序算法：数据规模、存储方式、稳定性、初始状态四个全要考虑，不只看快慢。

【坑位】

只比时间复杂度就下单，等于只看速度买鞋——合不合脚（业务要求）根本没看。

最容易踩的坑是：记住"快排最快"就到处用快排。为什么错：快排不稳定，题目要求"成绩相同按原来学号顺序"时它会打乱顺序；快排要随机访问，数据存在链表上时它跑不起来；数据基本有序时插入排序几趟就完事，快排反而没优势。正确做法：四个因素逐个过一遍——数据多大（规模）、存在哪（顺序还是链式）、相等元素次序能不能变（稳定性）、数据一开始什么样（有序、逆序还是乱序），每个因素想一个"不管它会出什么问题"的例子，全通过才定。

【类比】

像买鞋：不能只看"跑得快"（时间效率）。脚多大（数据规模）决定买大码小码；跑步还是爬山（存储方式）决定买跑鞋还是登山鞋；脚踝受过伤（稳定性要求）决定必须买高帮保护；平时走路多还是跑步多（初始状态）决定

买日常款还是竞速款。四项都合脚，鞋才买得对。

【口诀】 挑排序看四项：规模存储稳初态；只看快慢就下单，业务一问就露馅。

【考频】 「排序算法」在 2009–2022 年解析中出现 8 次 · 中频

第 8 题 · 散列表线性探查、查找失败平均查找长度（正确答案：C）

题干与选项：

现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是（ ）。

- A. 4
- B. 5.25
- C. 6
- D. 6.29

题意概括：一张长 11 的空散列表，散列函数是 key 除以 7 取余，用线性探查解决冲突，按顺序插 8 个数，问查找失败的平均查找长度。

入手点： 先把 8 个数一个一个插进表里，表长出来；再对 7 个散列值（0 到 6）一个一个数"找到空位要探几次"，加起来除以 7。

题干专有名词从零解释：

- 先想"散列表"：按关键字算个位置存，找的时候也按这个位置找，找得快。
- 再想"散列函数"：这里是 key 除以 7 取余，余数就是位置；余数只能是 0 到 6。
- 再想"线性探查"：冲突了（算出来的位置已经有人）就往后一个一个顺延找空位。
- 再想"查找失败"：要找的数不在表里，从散列位置顺着探到第一个空位才敢说"没有"；探过的格子数就是这次查找的查找长度。
- 再想"平均查找长度"：所有情况的查找长度加起来求平均；查找失败的平均，是对所有散列值求平均。

考查知识点：

- 先想一个一个插：87 除以 7 余 3，进 3 号位；40 余 5，进 5 号；30 余 2，进 2 号；6 余 6，进 6 号；11 余 4，进 4 号；22 余 1，进 1 号；98 余 0，进 0 号；20 余 6，6 号已占，顺延到 7 号，进 7 号。表里 0 到 7 号连着占满，8、9、10 号空着。
- 再想查找失败怎么数：从散列值 0 出发：0、1、2、3、4、5、6、7 都占着，到 8 号空才停，一共 9 次；从 1 出发：1 到 8，一共 8 次；从 2 出发：7 次；从 3 出发：6 次；从 4 出发：5 次；从 5 出发：4 次；从 6 出发：3 次。
- 再想平均：加起来 9+8+7+6+5+4+3=42，42 除以 7 等于 6。
- 再想分母为什么是 7 不是 11 也不是 8：查找失败只对散列值算，散列函数是除以 7 取余，只会算出 0 到 6，一共 7 种情况；表长 11 和关键字个数 8 都不是分母。
- 边界情况：如果表里有空位不连着，探查次数就不是这样一串递减的数；本题 0 到 7 连着占满，所以数出来正好是 9 到 3。

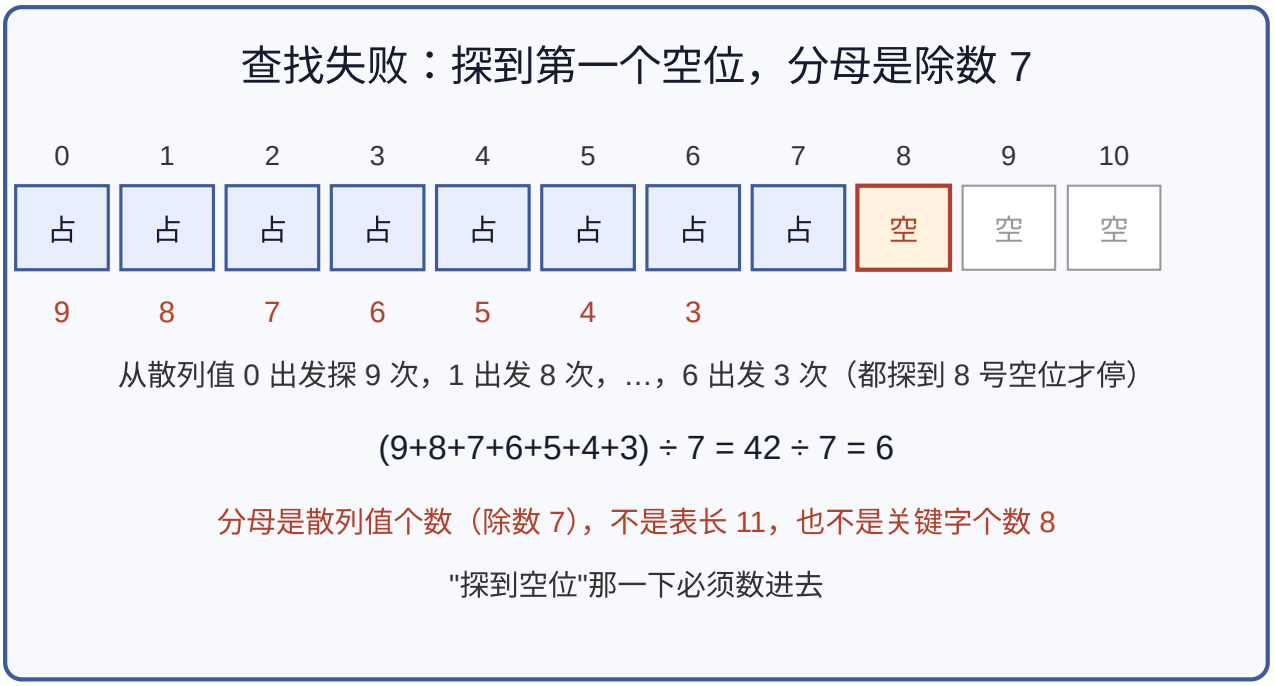
正确选项为什么对： (9+8+7+6+5+4+3)÷7=6，选 C。

错误选项为什么错＋举例：

- A（4）：探查次数数少了，比如从某个散列值出发没数到空位那一下就停了，或者把"探到空位"这一下漏了。
- B（5.25）：42÷8=5.25，错在分母用了 8（关键字个数）；查找失败的分母是散列值的个数 7。
- D（6.29）：相当于总数算成了 44，多了 2 次——比如空位之后又多探了一格，或者某个散列值出发多算了一步。

笔记本一句话： 第 8 题＋散列表查找失败：表里 0 到 7 连着占满，从 0 到 6 每个散列值探到 8 号空位，(9+8+7+6+5+4+3)÷7=6。

【图像】



线性探查查找失败：从每个散列值探到第一个空位，分母是除数 7

【定位】 数据结构 · 第7章查找 · 散列表

【易混对比】

- "查找失败"和"查找成功"是两回事：失败是对每个散列值算"探到第一个空位要几步"，分母是散列值的个数（除数 7）；成功是对每个已插入的关键字算，分母是关键字个数 8；分母搞混是本题第一大坑。
- "线性探查"和"链地址法"是两回事：线性探查是冲突了往后顺延找空位，会连成一片（本题 0 到 7 连着占满）；链地址法是每个位置挂一条链表，查找失败的算法完全不同。
- "散列函数除以 7"和"表长 11"是两回事：余数只能是 0 到 6，所以查找失败只从这 7 个位置出发；表长 11 只决定表有多大，不进分母。
- "探到空位"和"探到目标"是两回事：查找失败没有目标，探到第一个空位就能断定"没有"，这一下必须数进去，漏数结果就偏小。

查找失败：从每个散列值出发，数到第一个空位（含空位这一下），总和除以除数。

【坑位】

分母是除数 7，不是表长 11，也不是关键字个数 8；"探到空位"那一下必须数。

最容易踩的坑有两个：一是分母拿错——看到表里 8 个数就除以 8（得 5.25），看到表长 11 就除以 11；二是漏数"探到空位"那一下。为什么错：查找失败问的是"要找的数不在表里时平均探几次"，出发点只能是散列函数能算出来的值，也就是 0 到 6，一共 7 种情况，和表里有几个数、表有多长没关系；每次探查要到"第一个空位"才敢说没有，空位那一格是实实在在探过的一步。正确做法：先把 8 个数插进表（0 到 7 连着占满，8、9、10 空），再从 0 到 6 逐个出发数步数：9、8、7、6、5、4、3，加起来 42，除以 7 得 6。

【套路】

- 第一步，逐个插入画表：8 个数按"除以 7 取余、冲突往后顺延"插好，0 到 7 连着占满。
- 第二步，找第一个空位：每个散列值出发往后探，第一个空位是 8 号。
- 第三步，逐个散列值数步数：从 0 出发 9 步，从 1 出发 8 步，一直到从 6 出发 3 步，"探到空位"这一下要数进去。
- 第四步，求和除以除数：(9+8+7+6+5+4+3)除以7=42除以7=6。

先画表、再找空位、逐个散列值数步数、除以除数，空位那下别漏。

【类比】

像停车场找车位：停车场 11 个位（表长），入口按车牌尾数除以 7 取余分配入口（散列值 0 到 6），0 到 7 号位都停满了，8、9、10 空着。你开车从 0 号入口进，0 到 7 都有车，只能一路开到 8 号才停下——开了 9 个位。从 1 号入口进就开 8 个位，一直到从 6 号入口进开 3 个位。七个入口各走一遍，平均要开 (9+8+7+6+5+4+3)除以7=6 个位才找到空位。注意：平均是对"7 个入口"算的，不是 11 个位，也不是 8 辆车。

【口诀】 失败找空位，分母是除数；探到空位停，那下也要数。

【考频】 「散列表」在 2009–2022 年解析中出现 5 次 · 中频

第 9 题 · KMP 匹配成功前字符比较次数（正确答案：B）

题干与选项：

设主串 T = "abaabaabcabaabc"，模式串 S = "abaabc"，采用 KMP 算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（）。

- A. 9
- B. 10
- C. 12
- D. 15

题意概括：给一段主串和一段模式串，用 KMP 算法匹配，问从开始到匹配成功，一共做了多少次单个字符的比较。

入手点： 先把模式串的 next 数组算出来，再一格一格对主串走：对上就双双往后，不对就模式串按 next 往回跳、主串不动，每比一次数一次。

题干专有名词从零解释：

- 先想"KMP 算法"：匹配失败时模式串不用从头来，靠 next 数组往回跳，省时间。
- 再想"next 数组"：模式串每个位置"最长相等前后缀"的长度，告诉你失败后跳到哪一位；第 1 位是 0。
- 再想"单个字符比较"：主串一个字符和模式串一个字符比一次，算一次；对上算一次，没对上也算一次。
- 再想"主串"和"模式串"：主串是被找的那段长的，模式串是拿去找的那段短的。

考查知识点：

- 先想 next 数组：模式串是 a、b、a、a、b、c，next 数组算出来是 0、1、1、2、2、3。
- 再想一格一格走：主串是 a、b、a、a、b、a、a、b、c、a、b、a、a、b、c。
- 走法：第 1 到 5 个字符 a、b、a、a、b 全对上（5 次）；第 6 个主串 a 对模式串 c，不对（第 6 次），按 next 跳到第 3 位；主串不动，第 6 个 a 对模式串第 3 位 a，对上（第 7 次）；往后第 7 个 a 对第 4 位 a（第 8 次），第 8 个 b 对第 5 位 b（第 9 次），第 9 个 c 对第 6 位 c（第 10 次），全对上，匹配成功。一共 10 次。
- 关键两点：失败那次也算一次；主串指针不回退，所以总次数不会超过主串长度加几。

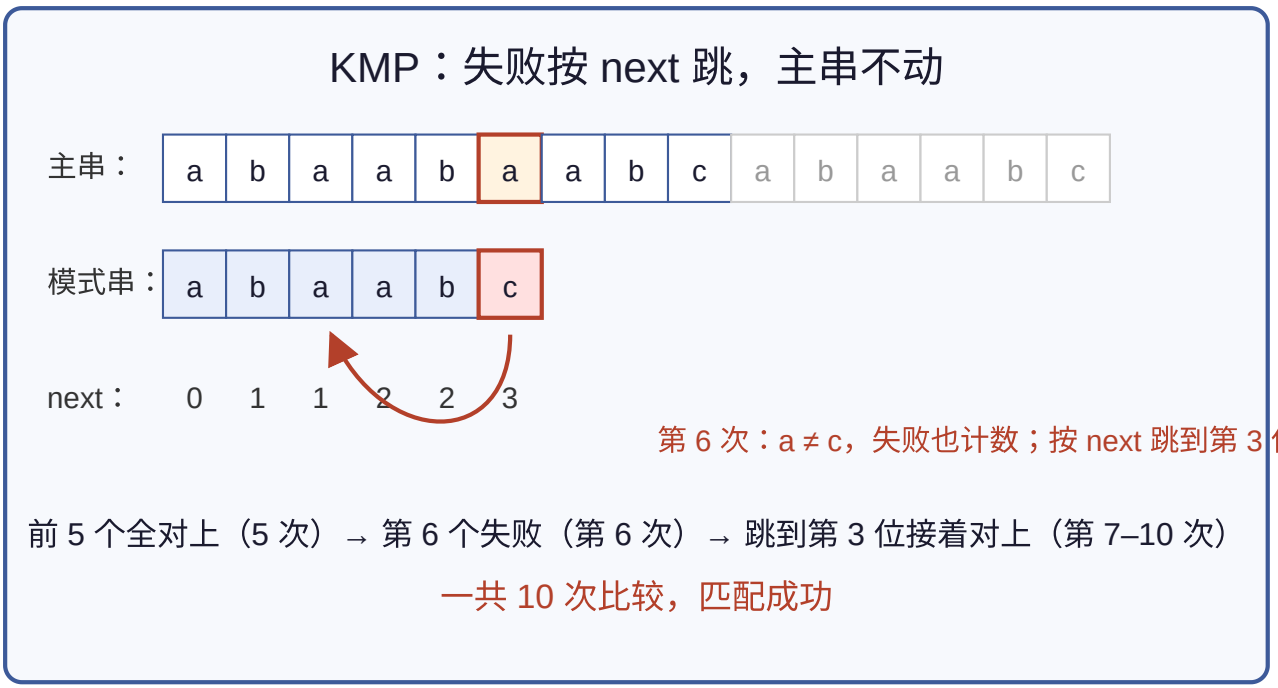
正确选项为什么对： 一共比较 10 次，选 B。

错误选项为什么错＋举例：

- A（9）：漏数了一次——最可能是失败那次没算，或者最后成功那次没算。
- C（12）：多算了 2 次——可能是失败后把主串也回退了（那是朴素算法的走法，不是 KMP）。
- D（15）：把主串长度 15 当成了比较次数，或者按朴素算法一格一格硬比算出来的。

笔记本一句话： 第 9 题＋KMP 数比较次数：对上往后走，不对按 next 跳、主串不动，失败那次也算，本题一共 10 次。

【图像】



KMP 数比较次数：对上双双往后，失败按 next 跳、主串不动，失败那次也算

【定位】 数据结构 · 第4章串 · KMP 算法

【易混对比】

- "KMP"和"朴素匹配"是两回事：KMP 失败时主串指针不回退，模式串按 next 数组往回跳；朴素算法是主串回退一位、模式串从头来。本题按朴素走法数会多算。
- "next 数组"和"nextval 数组"是两回事：next 是"最长相等前后缀长度"，失败后跳到该位置；nextval 是优化版，两处字符相同时再往前跳。本题数比较次数用 next 就够。
- "比较次数"和"移动次数"是两回事：比较次数是"主串一个字符和模式串一个字符比一次"，对上算一次、没对上也算一次；失败那次漏数是第二大坑。
- "主串"和"模式串"是两回事：主串是被找的长的那段（15 个字符），模式串是拿去找的短的那段（6 个字符）；next 数组是对模式串算的，别算反了。

KMP 数比较次数：对上就双双往后，不对就模式串按 next 跳、主串不动，每比一次都算上。

【坑位】

失败那次比较也要计数；主串指针不回退，别按朴素算法多算。

最容易踩的坑有两个：一是第 6 次比较（主串第 6 个 a 对模式串第 6 个 c）失败了，觉得"没对上就不算"，漏数一次得 9；二是失败后把主串也往回退（朴素走法），多算出 12。为什么错：比较次数的定义是"比一次算一次"，失败也是比过一次；KMP 的核心就是主串指针不回退，失败时只有模式串按 next 跳到第 3 位，主串第 6 个字符原地不动接着比。正确做法：先算出模式串的 next 数组（0、1、1、2、2、3），再一格一格走：前 5 个全对上（5 次），第 6 个失败（第 6 次），按 next 跳到第 3 位继续对上（第 7 到 10 次），匹配成功，一共 10 次。

【套路】

- 第一步，算 next 数组：对模式串算每个位置的最长相等前后缀长度，得 0、1、1、2、2、3。
- 第二步，双指针往前走：主串和模式串各一个指针，对上就都往后移一位。

- 第三步，失败按 next 跳：不对时模式串指针跳到 next 指向的位置，主串指针不动，原地接着比。
- 第四步，每次比较都计数：对上算一次，失败也算一次，本题一共 10 次。

先算 next，对上双进、失败按 next 跳主串不动，每比一次都计数。

【类比】

像对暗号：你（主串）站着不动，对方（模式串）一句一句对。对上了就下一句；对错一句，对方不从头开始，而是翻小抄（next 数组）跳到"前面哪句和现在这句长得最像"接着对——你不用退回去重说。每一句对没对上都算"对过一句"，包括对错的那句。

【口诀】 对上往后走，失败按 next 跳；主串不回头，比一次算一次。

【考频】 「KMP」在 2009–2022 年解析中出现 2 次 · 低频

第 10 题 · 快速排序第二趟不可能是哪个序列（正确答案：D）

题干与选项：

排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一"趟"。下列序列中，不可能是快速排序第二趟结果的是（）。

- A. 5, 2, 16, 12, 28, 60, 32, 72
- B. 2, 16, 5, 28, 12, 60, 32, 72
- C. 2, 12, 16, 5, 28, 32, 72, 60
- D. 5, 2, 12, 28, 16, 32, 72, 60

题意概括：快排里把还没定位置的元素都处理一遍叫一趟，给出四个序列，问哪个不可能是第二趟结束时的样子。

入手点： 先把序列排好序，看哪些元素已经在最终位置上（归位）；第二趟的规矩是：第一趟定一个枢轴，第二趟把枢轴两边的每个子序列各划分一次，每边再定一个枢轴；哪个序列凑不出这个结构，哪个就不可能。

题干专有名词从零解释：

- 先想"快速排序"：挑一个枢轴，小的放左、大的放右，枢轴归位，再对左右两边递归同样做。
- 再想"枢轴"：一次划分后就站在最终位置上的那个元素；枢轴一旦归位，后面就不再动它。
- 再想"一趟"：把当前所有没定位置的子序列各划分一次；第一趟只有一个序列（整段），第二趟有两个子序列（枢轴左右两边）。
- 再想"归位"：元素站在排好序后它该在的位置上；归位的元素就是前面某趟的枢轴。

考查知识点：

- 先想排好序：2、5、12、16、28、32、60、72。
- 再想四个序列归位的元素：A 里 28（第 5 位）、72（第 8 位）；B 里 2、28、72；C 里 2、28、32；D 里 12（第 3 位）、32（第 6 位）。
- 再想 A 为什么可能：第一趟枢轴 72（最后一位），第二趟只剩左边一个子序列，枢轴 28 归位，凑得上。
- 再想 B 为什么可能：第一趟枢轴 28，左右两个子序列第二趟各定一个：左边定 2，右边定 72，凑得上。
- 再想 C 为什么可能：第一趟枢轴 28，左边定 2，右边定 32，凑得上。
- 再想 D 为什么不可能：归位的是 12 和 32。假如第一趟枢轴是 32（第 6 位），左边子序列是第 1 到 5 位，右边是第 7 到 8 位；第二趟左边能定 12，但右边 {72，60} 第二趟必须再定一个（60 到第 7 位或 72 到第 8 位），D 的右边却是 72、60 两个都没归位，凑不上。假如第一趟枢轴是 12（第 3 位），左边 {5，2} 第二趟必须定一个（2 到第 1 位或 5 到第 2 位），D 的左边是 5、2 也都没归位，同样凑不上。所以 D 不可能。

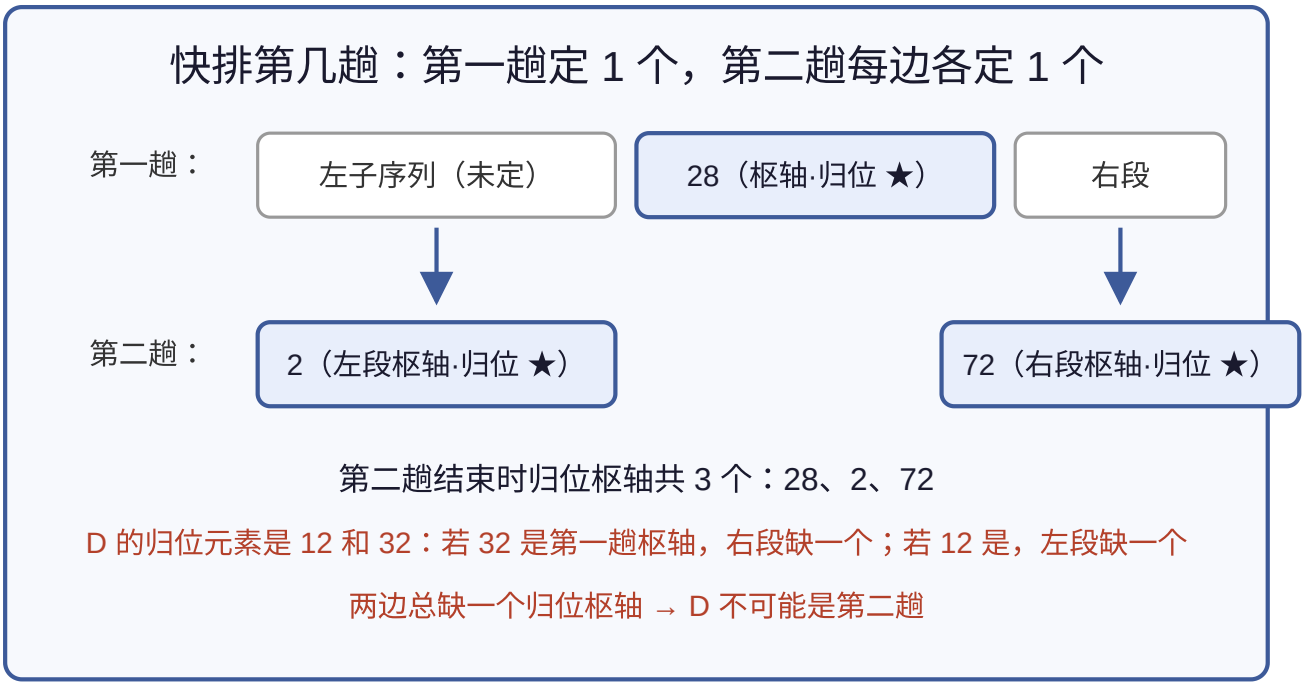
正确选项为什么对： D 两边都缺一个归位元素，凑不出第二趟的结构，选 D。

错误选项为什么错＋举例：

- A：第一趟枢轴 72，第二趟左边定 28，结构完整，是可能的。
- B：第一趟枢轴 28，第二趟左边定 2、右边定 72，是可能的。
- C：第一趟枢轴 28，第二趟左边定 2、右边定 32，是可能的。
- 教训：判断"第几趟"先找归位元素，再按"第一趟定一个、第二趟每边各定一个"套结构，缺一边就排除。

笔记本一句话： 第 10 题＋快排第几趟：先找归位元素，再按"第一趟定一个、第二趟每边各定一个"套结构，D 两边都缺一个所以不可能。

【图像】



快排第几趟：先找归位元素，第一趟定一个、第二趟每边各定一个

【定位】 数据结构 · 第8章排序 · 快速排序

【易混对比】

- "一趟"和"一次划分"是两回事：一趟是把当前所有没定位置的子序列各划分一次；第一趟只有整段、定 1 个枢轴，第二趟是枢轴左右两边各划分一次、再定 2 个。
- "归位"和"恰好站在最终位置"是两回事：归位的是前面某趟的枢轴，之后不再动；有些元素碰巧站在最终位置但不是枢轴，判断趟数时要按枢轴结构套，不能只看位置对没对。
- "枢轴"和"普通元素"是两回事：枢轴一次划分就站在最终位置，后面各趟都不再动它；普通元素在划分中还会被搬来搬去。
- "第二趟"和"第三趟"是两回事：第二趟结束时归位的枢轴一共 3 个（第一趟 1 个加第二趟 2 个）；归位元素个数对不上这个结构，趟数就判错了。

判第几趟：先把序列排好序找归位元素，再按"第一趟定一个、第二趟每边各定一个"套结构，缺一边就排除。

【坑位】

别数"有几个元素位置对了"，要数"归位枢轴凑不凑得出趟数的结构"。

最容易踩的坑是：看到 D 里 12 和 32 都在最终位置，就觉得"两个归位了，第二趟该有两个，D 也可能"。为什么错：第二趟的 2 个新枢轴必须分居第一趟枢轴的两边——左边子序列定一个、右边子序列定一个。D 的归位元素是 12（第 3 位）和 32（第 6 位）：如果第一趟枢轴是 32，右边两个元素第二趟必须再定一个，可右边两个都没归位；如果第一趟枢轴是 12，左边两个元素第二趟必须再定一个，可左边两个也没归位。两边各缺一个，结构凑不上，D 不可能。正确做法：先排序找归位元素，再设第一趟枢轴，看第二趟左右两边是不是"各定一个"，缺一边就排除。

【套路】

- 第一步，排序找归位：把序列排好序，标出哪些元素已经在最终位置（2、5、12、16、28、32、60、72）。
- 第二步，设第一趟枢轴：第一趟只定 1 个枢轴，从归位元素里挑一个当候选。
- 第三步，套第二趟结构：第一趟枢轴把序列分成左右两段，第二趟每段各定 1 个枢轴。
- 第四步，缺一边就排除：哪边该定没定，这个序列就不可能是第二趟（D 两边都缺）。

排序找归位，一趟定一个、二趟每边定一个，缺边排除。

【类比】

像选组长：全班先选一个总组长（第一趟枢轴），总组长把班分成左右两组；第二轮左右两组各选一个小组长（第二趟每边一个）。D 这个班的"组长"名单是 12 和 32——如果 32 是总组长，右组连个小组长都没选出来；如果 12 是总组长，左组又没选出来。两组总有一组缺组长，这轮选举就不成立。

【口诀】 一趟定一个，二趟每边定；归位先找准，缺边就排除。

【考频】 「快速排序」在 2009–2022 年解析中出现 10 次 · 高频

第 11 题 · 12 路归并最佳归并需补虚段数（正确答案：B）

题干与选项：

设外存上有 120 个初始归并段，进行 12 路归并时，为实现最佳归并，需要补充的虚段个数是（ ）。

- A. 1
- B. 2
- C. 3
- D. 4

题意概括：外存上有 120 个初始归并段，做 12 路归并，要达到最佳归并（每次归并都满 12 路），问需要补几个虚段。

入手点： 最佳归并就是每次归并都凑满 k 路；段数不够整除时补虚段（空段，不占实际数据）凑整；先算"段数减 1"除以"k 减 1"的余数。

题干专有名词从零解释：

- 先想"归并段"：已经内部有序的一段数据，归并就是把几段合成一段。
- 再想"k 路归并"：一次把 k 个段合并成 1 个；这里 k=12。
- 再想"虚段"：假想的空段，里面没有数据，只用来凑数，让每次归并都能凑满 k 路。
- 再想"最佳归并"：归并树最省、总读写量最小的归并方式，要求每次归并都尽量满 k 路。

考查知识点：

- 先想每归并一次段数怎么变：k 个段变成 1 个，段数减少 k−1 个；这里每次减少 11 个。
- 再想从 120 个段归并到 1 个，一共要减少 119 个。119 除以 11 得 10 余 9——归并 10 次满 12 路后，还剩 9 个"零头"，不够再凑一次满的。
- 再想补几个：补到能整除，11−9=2。补 2 个虚段后，(120+2−1)=121，121 除以 11 得 11，正好 11 次满归并。
- 公式记法：(段数−1) 除以 (k−1) 的余数是 r，r=0 不用补，否则补 (k−1)−r 个。
- 小例子：5 个段做 3 路归并，(5−1) 除以 2 余 0，不用补；6 个段做 3 路，(6−1) 除以 2 余 1，补 1 个。

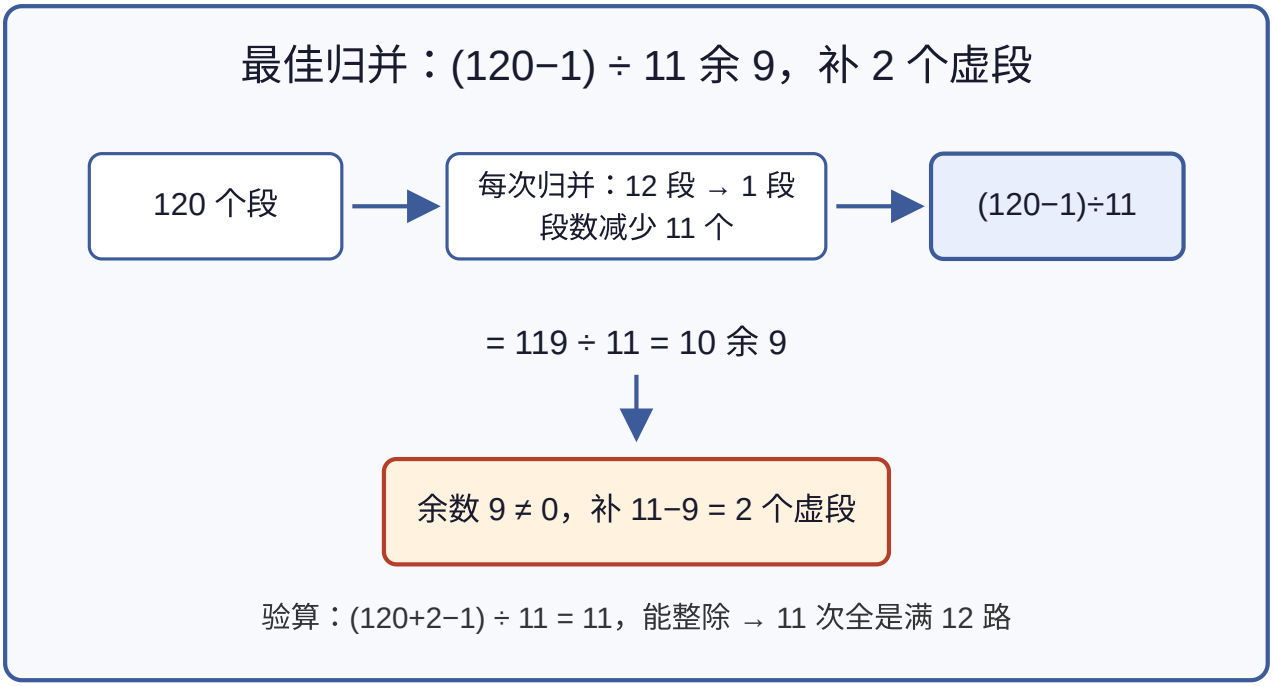
正确选项为什么对： (120−1) 除以 11 余 9，补 11−9=2 个，选 B。

错误选项为什么错＋举例：

- A（1）：120 除以 12 得 10，以为不用补或者只补 1 个——错在公式里是 (段数−1) 除以 (k−1)，不是段数除以 k。
- C（3）：余数算错了（比如把 119 除以 11 的余数算成 8，11−8=3）。
- D（4）：把 k−1 当成 12 去算，或者余数和补数加反了。

笔记本一句话： 第 11 题＋最佳归并补虚段：(120−1) 除以 11 余 9，补 11−9=2 个。

【图像】



最佳归并补虚段：(120−1) 除以 11 余 9，补 11−9=2 个

【定位】 数据结构 · 第8章排序 · 外部排序

【易混对比】

- "(段数−1)除以(k−1)"和"段数除以k"是两回事：归并一次是 k 个段变成 1 个，段数减少 k−1 个，所以公式里两个都是"减 1"；用 120 除以 12 得 10 会以为不用补，直接掉坑。
- "补虚段"和"不补"是两回事：余数是 0 才不用补；余数不是 0，就补到能整除，本题余 9，补 11−9=2 个。
- "虚段"和"实段"是两回事：虚段是假想的空段，里面没有数据，只用来凑数让每次归并都满 k 路；它不占实际数据、不影响结果。
- "最佳归并"和"普通归并"是两回事：最佳归并要求每次归并都尽量满 k 路，总读写量最小；普通归并不凑满，读写量更大。

补虚段公式：(段数−1) 除以 (k−1) 得余数 r，r=0 不补，否则补 (k−1)−r 个。

【坑位】

公式里两个"减 1"都不能丢：120 除以 12 得 10 就收手，是本题最经典的坑。

最容易踩的坑是：120 除以 12 得 10 整除，以为不用补虚段。为什么错：归并一次是 12 个段合成 1 个，段数减少的是 11 个不是 12 个；从 120 个段归并到 1 个，一共要减少 119 个，119 除以 11 得 10 余 9——归并 10 次满 12 路后还剩 9 个零头，不够再凑一次满的。正确做法：牢记公式是 (段数−1)除以(k−1)，(120−1)除以11=119除以11=10 余 9，补 11−9=2 个；验算：(120+2−1)=121，121 除以 11 得 11，正好 11 次满归并。

【套路】

- 第一步，列数：段数 120，k=12，所以 k−1=11。
- 第二步，算余数：(120−1)除以11=119除以11=10 余 9。
- 第三步，补到整除：余数 9 不是 0，补 11−9=2 个虚段。
- 第四步，验算：(120+2−1)除以11=11，能整除，11 次都是满 12 路。

段减一除以 k 减一，余几就补满 k 减一，整除不用补。

【类比】

像凑满一车去郊游：每辆车坐 12 人（k 路），上车后 12 人合成 1 个"团"（段数减 11）。120 个人要合成 1 个团，得"上车"11 次；前 10 次满车走了 110 人，还剩 9 个人不够一车。找 2 个"幽灵乘客"（虚段，不占座、只凑数）凑满 12 人，11 次全是满车出发，这就是最佳归并。

【口诀】 段减一除以 k 减一，余几补满 k 减一；整除不用补，补数别算反。

【考频】 「归并排序」在 2009–2022 年解析中出现 11 次 · 高频

第 12 题 · 冯·诺依曼结构基本思想、错误叙述（正确答案：C）

题干与选项：

下列关于冯·诺依曼结构计算机基本思想的叙述中，错误的是（）。

- A. 程序的功能都通过中央处理器执行指令实现
- B. 指令和数据都用二进制数表示，形式上无差别
- C. 指令按地址访问，数据都在指令中直接给出
- D. 程序执行前，指令和数据需预先存放在存储器中

题意概括：关于冯·诺依曼结构计算机基本思想的四句话，问哪句是错的。

入手点： 本题问"错误的是"，正确选项就是那个错说法；先把四句话一个一个过，错的那个就是答案。

题干专有名词从零解释：

- 先想"冯·诺依曼结构"：现代计算机的基本结构，核心是"存储程序"。
- 再想"中央处理器（CPU）"：执行指令的部件。
- 再想"二进制"：只有 0 和 1 两个数码的表示方法。
- 再想"指令"：告诉计算机干什么的命令；"数据"是被处理的东西。
- 再想"存储器"：放指令和数据的仓库；"地址"是存储器里每个位置的编号。
- 再想"直接给出"：指令里直接写着数据本身，不用再去别处取。

考查知识点：

- 先想 A：程序的功能就是靠 CPU 一条一条执行指令实现的——对的。
- 再想 B：指令和数据都用二进制表示，放在存储器里形式上没有差别——对的；这正是"存储程序"思想容易混淆的地方：同一串 0、1，CPU 按指令取就是指令，按数据取就是数据。
- 再想 C：指令是按地址访问的（CPU 按地址去存储器取指令），但数据不都在指令中直接给出——数据可以放在寄存器里、存储器里，指令里给的是地址或者一部分数据（立即数），不是所有数据都在指令里。
- 再想 D：程序执行前，指令和数据要预先存放在存储器中，这就是"存储程序"——对的。

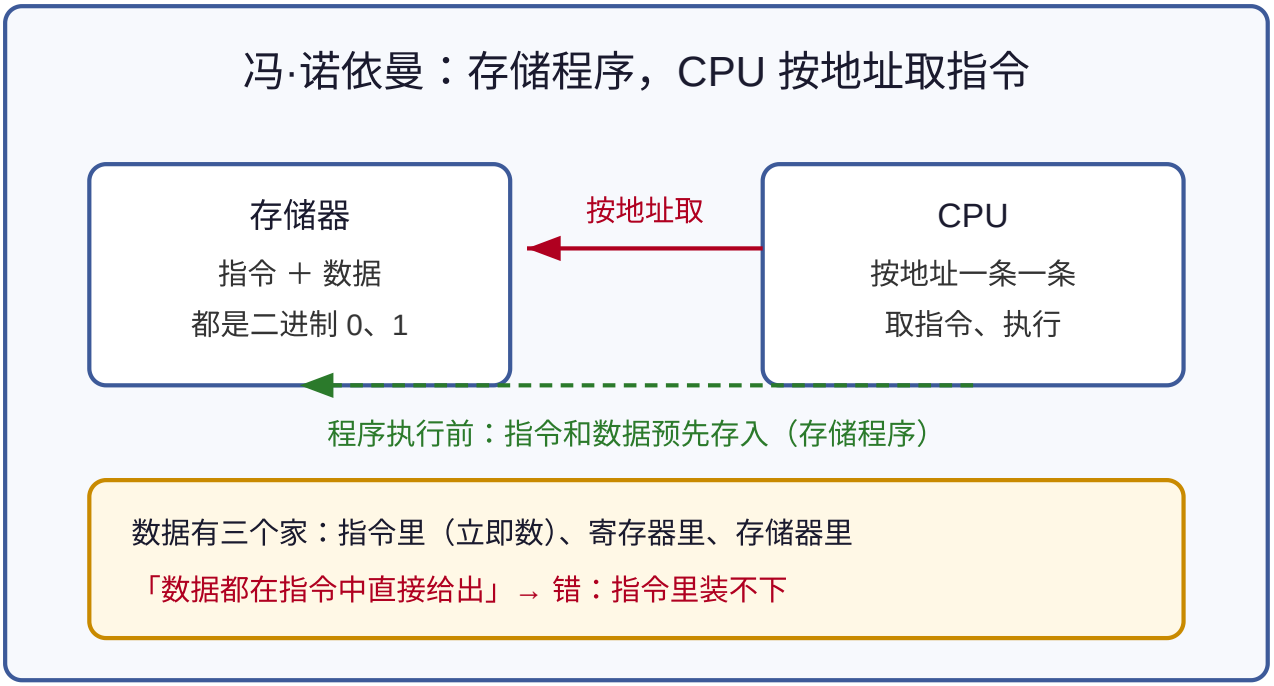
正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。C 说"数据都在指令中直接给出"，不对。数据有三种来路：指令里直接带的（立即数）、寄存器里的、存储器里的；说"都在指令中"把后两种漏了，所以 C 是错说法，选 C。

错误选项为什么错＋举例：

- A：程序就是指令的集合，功能当然靠执行指令实现——对的说法。
- B：指令和数据都是二进制串，形式无差别——对的说法；正因为无差别，取错了才会把数据当指令执行。
- D：存储程序就是先存后执行——对的说法。

笔记本一句话： 第 12 题＋冯·诺依曼：指令按地址取，数据不一定在指令里——寄存器、存储器里都有。

【图像】



冯·诺依曼结构：程序预先存入存储器，CPU 按地址一条条取指令执行

【定位】 计算机组成原理 · 第 1 章计算机系统概述 · 冯·诺依曼结构基本思想

【易混对比】

- "存储程序"和"程序控制"是两回事：存储程序是"先把指令和数据都存进存储器"，程序控制是"CPU 按地址一条一条取出来执行"；一个是存法，一个是执行法。
- "指令"和"数据"是两回事：同一串二进制 0、1，被 CPU 当指令取来执行就是指令，被当成数来算就是数据；存进存储器时两者形式上没差别，身份由"怎么取"决定。
- "按地址取指令"和"数据在指令里直接给出"是两回事：指令一定按地址从存储器取；数据却不一定在指令里——可以住在寄存器里、存储器里，指令里带的只是地址或一部分数（叫立即数）。

指令按地址取，数据却有三个家：指令里、寄存器里、存储器里。

【坑位】

问"错误的是"，先放下"每句都对"的惯性，一句一句找反例。

最容易踩的坑，是看到"数据都在指令中直接给出"觉得顺眼。想个具体场景：加法指令要把两个数相加，这两个数真会写进指令里吗？一条 32 位指令才 4 个字节，两个 32 位数要 8 个字节，根本装不下。所以数据常常住在寄存器或存储器里，指令里只给地址。记住"装不下"这个画面，就不会被这句骗了。

【类比】

像餐馆点菜：菜单（指令）上写着"宫保鸡丁"，但鸡丁（数据）不可能印在菜单上——它在后厨（寄存器/存储器）里。菜单上只写菜名（地址/编号），服务员按菜名去后厨取。说"所有食材都印在菜单上"，一看就知道不对，装不下嘛。

【口诀】 存储程序先存好，指令按址一条条；数据三处可安家，指令里头装不下。

【考频】 「冯·诺依曼」在 2009–2022 年解析中出现 2 次 · 低频

第 13 题 · unsigned short 转 short 后的值（正确答案：A）

题干与选项：

考虑以下 C 语言代码：

```
unsigned short usi = 65535;
short si = usi;
```

执行上述程序段后，si 的值是（）。

- A. -1
- B. -32767
- C. -32768
- D. -65535

题意概括：一段 C 语言代码，把 65535 存进 unsigned short 再转给 short，问 short 的值是多少。

入手点： 关键看位模式不变、解释变了：65535 的 16 位全是 1，换成有符号数解释就是 −1。

题干专有名词从零解释：

- 先想"unsigned short"：无符号短整数，16 位，全表示非负数，范围 0 到 65535。
- 再想"short"：有符号短整数，16 位，最高位是符号位，范围 −32768 到 32767。
- 再想"补码"：有符号数在机器里的表示方法，负数是"取反加 1"。
- 再想"位模式"：16 个 0、1 的排列本身；同一个位模式，换一种类型解释，值就变了。

考查知识点：

- 先想 65535 的位模式：unsigned short 的 65535 是 16 个 1（0xFFFF）。
- 再想转给 short：位模式原样搬过去，解释变了；最高位 1 说明是负数，按补码读：16 个 1 取反是 16 个 0，加 1 是 1，所以真值是 −1。
- 再想小例子：unsigned short 65534（0xFFFE）转 short 是 −2；unsigned short 32768（0x8000）转 short 是 −32768。
- 规律：超过 32767 的数转过去都变负数，值等于原数减 65536（65535−65536=−1）。

正确选项为什么对： 位模式 16 个 1，按有符号补码读就是 −1，选 A。

错误选项为什么错＋举例：

- B（−32767）：把 0xFFFF 当成了 0x8001 之类的数，或者符号位和数值位搞混了。
- C（−32768）：那是 0x8000 的值（最小的 short），不是 0xFFFF 的值。
- D（−65535）：以为"转过去就加个负号"，不对——有符号数的范围只有到 −32768，−65535 根本存不进 short。

笔记本一句话： 第 13 题＋unsigned 转 short：位模式不变、解释变，65535 全 1 转过去就是 −1。

【图像】



65535 的位模式 16 个 1 不变，换成 short 解释就是 -1

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 定点数的补码表示与类型转换

【易混对比】

- "位模式"和"值"是两回事：16 个 0、1 的排列是位模式，固定不变；同一个位模式，用无符号解释是一个值，用有符号解释是另一个值。
- "unsigned short"和"short"是两回事：都是 16 位，但前者 16 位全表示大小（0 到 65535）；后者最高位是符号位（专门表示正负的 1 个比特，1 代表负），范围是 −32768 到 32767。
- "取反加 1"和"加个负号"是两回事：补码求负数的真值是"全部取反再加 1"，不是简单在前面加个负号；65535 转过去是 −1，不是 −65535。

位模式原样搬家，解释一换，值就跟着变。

【坑位】

最高位是 1 就是负数：先看符号位，再决定按补码读还是直接读。

最容易踩的坑是"转过去就加个负号"，以为 65535 转成 short 是 −65535。先看范围：short 最小只能到 −32768，−65535 根本装不下，不可能。正确做法是盯住位模式：65535 是 16 个 1，搬到 short 里最高位是 1，说明是负数；16 个 1 取反是 16 个 0，加 1 得 1，所以真值是 −1。

【套路】

- 第一步，写出位模式：65535 用 16 位表示是 16 个 1（0xFFFF）。
- 第二步，看符号位：最高位是 1，转成 short 后是负数，按补码规则读。
- 第三步，取反加 1 求真值：16 个 1 取反得 16 个 0，加 1 得 1，真值就是 −1。

位模式 → 符号位 → 取反加 1，65535 转 short 得 −1。

【类比】

像同一个手势在不同地方意思不同：竖大拇指（位模式 16 个 1）在这边是"第一名"（无符号解释 65535），在那边是"倒数第一"（有符号解释 −1）。手势本身没变，变的是"当地规矩"（类型）。

【口诀】 位模不变解释变，全 1 转来是负一；取反加一求真值，先看范围装不下。

【考频】 「补码」在 2009–2022 年解析中出现 18 次 · 高频

第 14 题 · 缺页处理叙述、错误项（正确答案：D）

题干与选项：

下列关于缺页处理的叙述中，错误的是（）。

- A. 缺页是在地址转换时 CPU 检测到的一种异常
- B. 缺页处理由操作系统提供的缺页处理程序来完成
- C. 缺页处理程序根据页故障地址从外存读入所缺失的页
- D. 缺页处理完成后回到发生缺页的指令的下一条指令执行

题意概括：关于缺页处理的四句话，问哪句是错的。

入手点： 本题问"错误的是"，先想缺页处理的完整流程：谁发现、谁处理、从哪读、处理完回哪——哪一步说错了就是答案。

题干专有名词从零解释：

- 先想"缺页"：要访问的页不在内存里。
- 再想"地址转换"：把虚拟地址翻译成物理地址，方法是查页表。
- 再想"异常"：CPU 执行中发现的不正常情况，停下转去处理。
- 再想"缺页处理程序"：操作系统里专门处理缺页的一段程序。
- 再想"页故障地址"：引起缺页的那个虚拟地址。
- 再想"外存"：磁盘，存着不在内存里的页。

考查知识点：

- 先想 A：缺页是在地址转换时发现的——查页表发现"存在位"是 0，CPU 就知道这页不在内存，触发异常。对的。
- 再想 B：缺页处理由操作系统提供的缺页处理程序完成，硬件只负责发现和转交。对的。
- 再想 C：处理程序按页故障地址去外存把缺失的那页读进内存。对的。
- 再想 D：处理完不是回到"下一条指令"，而是回到发生缺页的那条指令本身，重新执行——因为那条指令还没执行完（它要的数据还没到），必须重来一遍。

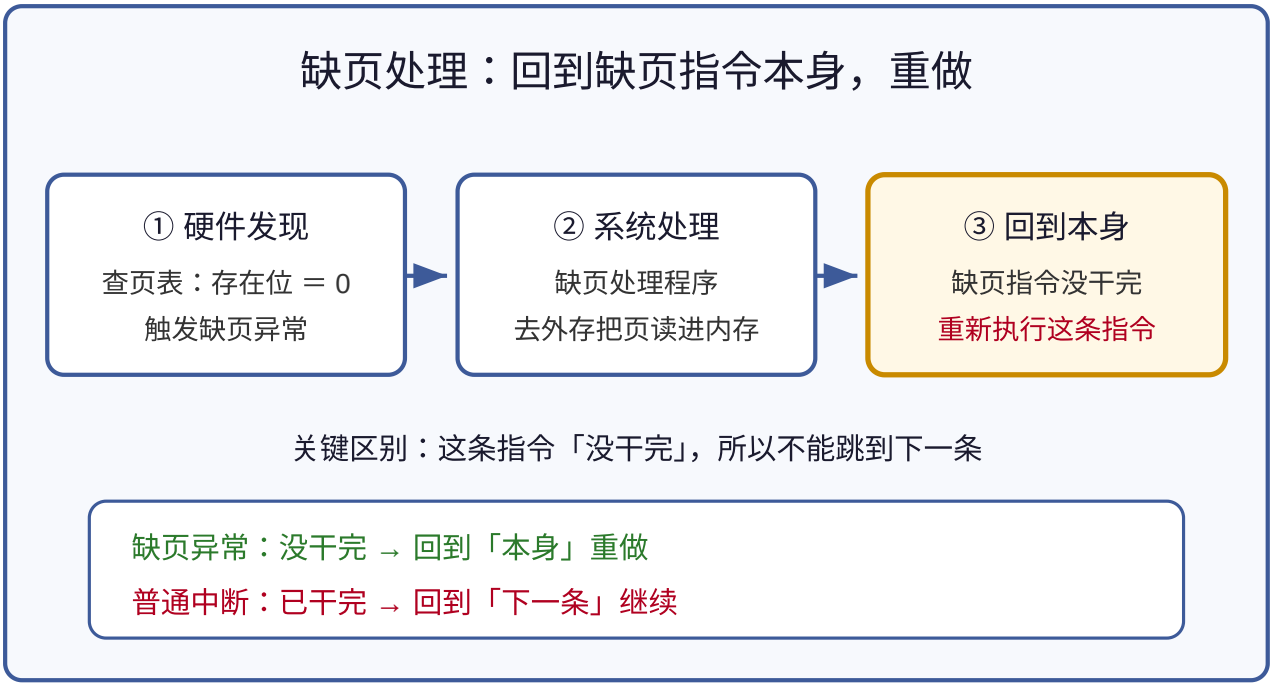
正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。D 说回到下一条指令，不对。缺页的指令是"还没执行完"（卡在取数据那一步），处理完必须回到它本身重新执行；回到下一条就等于跳过了这条指令，结果就错了。选 D。

错误选项为什么错＋举例：

- A：地址转换查页表时发现存在位为 0，就是缺页——对的说法。
- B：硬件发现、软件处理，分工就是这样——对的说法。
- C：按故障地址去外存读页——对的说法；不按故障地址就读错页了。

笔记本一句话： 第 14 题＋缺页处理完回到缺页指令本身重做，不是下一条。

【图像】



缺页处理流程：发现→操作系统处理→外存读页→回到缺页指令本身重做

【定位】 计算机组成原理 · 第 3 章存储系统 · 虚拟存储器（缺页处理）

【易混对比】

- "缺页异常"和"一般中断"是两回事：缺页是执行中发现数据没到、必须重来的异常，处理完要回到"惹事的那条指令本身"；一般中断（比如外设通知）是事情办完才被打断，处理完回到"下一条"继续。
- "硬件"和"操作系统"的分工是两回事：硬件只负责发现缺页（查页表时看到存在位是 0；存在位就是页表里标记"这页在不在内存"的 1 个比特）和转交；真正去外存搬页的是操作系统的缺页处理程序。
- "缺页"和"页面置换"是两回事：缺页是"要的页不在内存里"；页面置换是"内存满了，挑一页腾地方"——置换只在内存不够时才发生，不是缺页的必备步骤。

缺页那条指令还没干完，处理完必须回到它本身重做。

【坑位】

回到"下一条"还是回到"本身"，看这条指令干完没有：没干完就重做。

最容易踩的坑是把缺页当成普通中断，顺手选"回到下一条指令"。想想具体过程：这条指令要读内存里的数据，数据还没到，它卡在半路——如果处理完跳到下一条，这条指令的活就永远没人干，结果就错了。正确做法是记住"缺页＝没干完＝重做"，普通中断＝干完了被打断＝继续下一条。

【套路】

- 第一步，定流程：地址转换查页表 → 存在位 0 发现缺页 → 硬件转交操作系统。
- 第二步，定动作：缺页处理程序按故障地址去外存把页读进内存。
- 第三步，定返回点：问"这条指令干完没有"——没干完，回到缺页指令本身重新执行。

查表发现 → 系统搬页 → 回到本身重做。

【类比】

像做菜做到一半发现盐没了（缺页）：你去楼下买盐（操作系统去外存搬页），回来是继续"放盐这一步"（回到本身重做），而不是跳过直接炒下一道菜（下一条指令）——盐没放，这道菜就没做完。

【口诀】 缺页指令没干完，处理回来重做它；硬件发现系统搬，下一条是陷阱呀。

【考频】 「缺页中断」在 2009–2022 年解析中出现 11 次 · 高频

第 15 题 · 大端、基址寻址、操作数 LSB 地址（正确答案：D）

题干与选项：

某计算机采用大端方式，按字节编址。某指令中操作数的机器数为 1234FF00H，该操作数采用基址寻址方式，形式地址（用补码表示）为 FF12H，基址寄存器的内容为 F0000000H，则该操作数的 LSB（最低有效字节）所在的地址是（ ）。

- A. F000FF12H
- B. F000FF15H
- C. EFFFFF12H
- D. EFFFFF15H

题意概括：一台大端机器、按字节编址，给出操作数、寻址方式、形式地址和基址寄存器的值，问操作数最低字节的地址。

入手点： 分两步：先算有效地址（基址加形式地址，注意形式地址是补码、有正负）；再看大端规则：最低字节在最高地址，所以有效地址加 3。

题干专有名词从零解释：

- 先想"大端方式"：高位字节放低地址，低位字节放高地址——"大"在前。
- 再想"按字节编址"：每个字节地址加 1。
- 再想"基址寻址"：有效地址等于基址寄存器的值加上形式地址。
- 再想"形式地址"：指令里给出的地址部分，这里用补码表示，有正负。
- 再想"补码"：有符号数的表示，最高位 1 是负数。
- 再想"LSB（最低有效字节）"：就是数值最低的那 8 位。
- 再想"操作数"：这里是 4 个字节的一段数据。

考查知识点：

- 先算形式地址的真值：FF12H 是 16 位补码，最高位 1，是负数；取反加 1 得 00EEH，所以真值是 −238（也就是 −0xEE）。
- 再算有效地址：基址 F0000000H 加上 −238。F0000000H 减去 0x100 是 EFFFFF00H，再加回 0x12（因为 0x100 减 0xEE 等于 0x12），得 EFFFFF12H——这就是操作数第一个字节（最高字节，大端放低地址）的地址。
- 再想最低字节：操作数一共 4 个字节，占 EFFFFF12H、EFFFFF13H、EFFFFF14H、EFFFFF15H；最低字节在最高地址，就是 EFFFFF15H。

正确选项为什么对： 有效地址 EFFFFF12H，最低字节再加 3，得 EFFFFF15H，选 D。

错误选项为什么错＋举例：

- A（F000FF12H）：形式地址没按补码算（把 FF12H 当成无符号正数加了），有效地址第一步就错了。
- B（F000FF15H）：有效地址算错了位，但"加 3 找最低字节"那步做对了——错在第一步。
- C（EFFFFF12H）：有效地址算对了，但忘了大端规则，写成了最高字节的地址；最低字节要再加 3。

笔记本一句话： 第 15 题＋大端基址：先补码算有效地址 EFFFFF12H，最低字节在最高地址，再加 3 得 EFFFFF15H。

【图像】



大端：高位字节在低地址；有效地址 EFFFFF12H，LSB 在 +3 处

【定位】 计算机组成原理 · 第 4 章指令系统 · 寻址方式（基址寻址）与大端方式

【易混对比】

- "大端"和"小端"是两回事：大端是"大的在前"——数值最高的那字节放在最低的地址；小端反过来，最低字节放最低地址。先想具体数：0x12345678 在大端机器里，地址最小的那字节存 0x12。
- "形式地址"和"有效地址"是两回事：形式地址是写在指令里的那部分（这里是补码，可能为负）；有效地址是真正去访存的地址，基址寻址下等于"基址寄存器的值＋形式地址"。
- "最高字节地址"和"最低字节（LSB）地址"是两回事：4 字节操作数占 4 个连续地址；大端下最高字节在最低地址，最低字节在最高地址，两者差 3。

先算有效地址（补码形式地址可正可负），大端下 LSB 地址 ＝ 有效地址 ＋ 3。

【坑位】

FF12H 这种形式地址先看符号位：最高位是 1 就是负数，先求真值再加。

最容易踩的坑有两个。第一，看到 FF12H 直接当正数加上，基址 F0000000H 越加越大就错了——它是 16 位补码，最高位 1 说明是负数，取反加 1 得 −0xEE（−238）。第二，有效地址算对了（EFFFFF12H）就停笔，忘了这是最高字节的地址；大端机器里最低字节住在最高地址，还要再加 3 才是 EFFFFF15H。

【套路】

- 第一步，形式地址求真值：FF12H 最高位 1，是负数，取反加 1 得 −238。
- 第二步，算有效地址：F0000000H ＋ (−238) ＝ EFFFFF12H，这是最高字节的地址。
- 第三步，大端找 LSB：4 字节占 4 个地址，最低字节在最高地址，有效地址 ＋ 3 ＝ EFFFFF15H。

补码求真值 → 基址相加 → 大端加 3 找 LSB。

【类比】

像一排 4 个储物柜，柜门编号从小到大（地址递增）：大端规定"最重要的东西放编号最小的柜子"。你要找最不重要的那件（LSB），就去编号最大的柜子——第一个柜子的编号加 3。

【口诀】 大端大在前，低址存高端；形式先补码，有效加三找低端。

【考频】 「大端」在 2009–2022 年解析中出现 8 次 · 中频

第 16 题 · 处理器时钟脉冲信号叙述、错误项（正确答案：D）

题干与选项：

下列有关处理器时钟脉冲信号的叙述中，错误的是（）。

- A. 时钟脉冲信号由机器脉冲源发出的脉冲信号经整形和分频后形成
- B. 时钟脉冲信号的宽度称为时钟周期，时钟周期的倒数为机器主频
- C. 时钟周期以相邻状态单元间组合逻辑电路的最大延迟为基准确定
- D. 处理器总是在每来一个时钟脉冲信号时就开始执行一条新的指令

题意概括：关于处理器时钟脉冲信号的四句话，问哪句是错的。

入手点： 本题问"错误的是"，先想时钟是干什么的：它像节拍器，一个节拍干一小步，不是一拍干一条指令。

题干专有名词从零解释：

- 先想"时钟脉冲信号"：周而复始的方波信号，给处理器打节拍。
- 再想"机器脉冲源"：产生原始脉冲的电路。
- 再想"整形和分频"：把脉冲修整齐、按需要放慢。
- 再想"时钟周期"：两个脉冲之间的时间。
- 再想"机器主频"：1 秒里有多少个周期，倒数关系（主频等于 1 除以时钟周期）。
- 再想"状态单元"：寄存器这种存状态的部件；"组合逻辑电路"是算东西的电路，夹在两级状态单元之间。

考查知识点：

- 先想 A：时钟脉冲就是机器脉冲源发出的脉冲经整形、分频后形成的。对的。
- 再想 B：时钟脉冲一个接一个，两个脉冲之间的时间就是一个时钟周期；1 秒里有多少个周期，就是主频（主频等于 1 除以时钟周期）。对的。
- 再想 C：时钟周期不能太短，至少要够相邻状态单元之间的组合逻辑算完——以最大延迟为基准确定，不然算没完就打下一拍，结果就错了。对的。
- 再想 D：一条指令要取指、译码、执行、访存、写回好几步，每步至少占一个周期；一个时钟脉冲只推进一小步，不是一拍就执行完一条新指令。

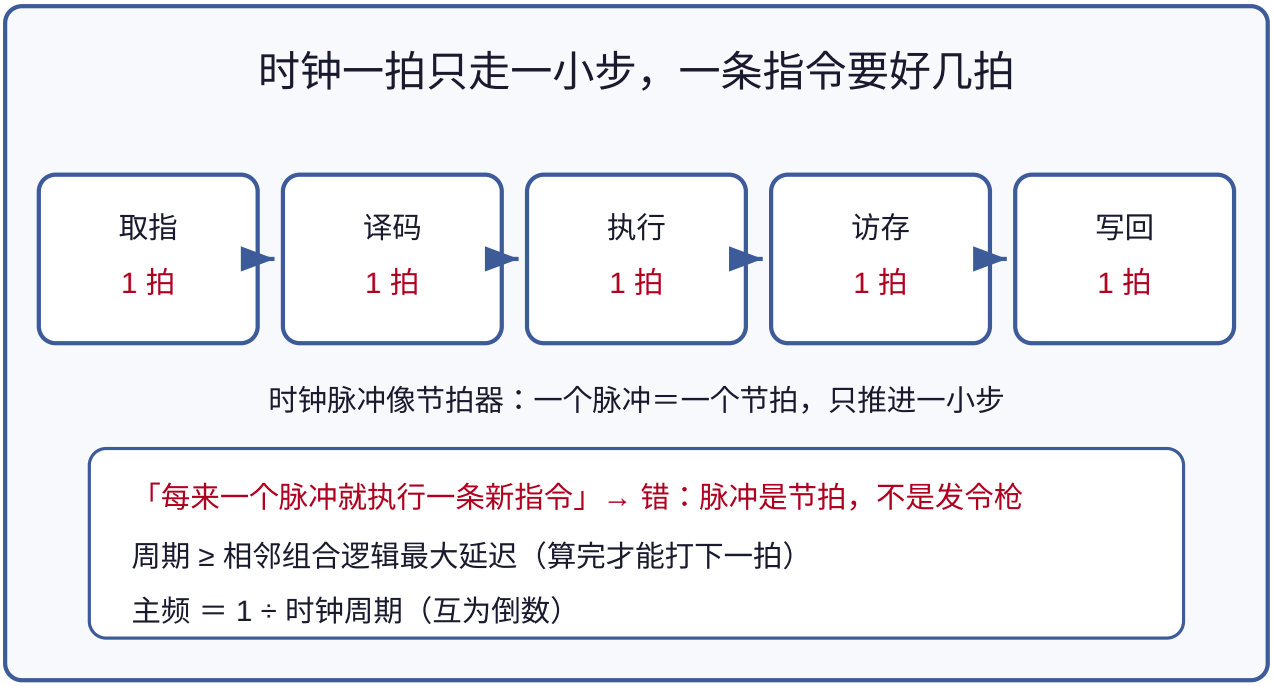
正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。D 说"每来一个时钟脉冲就开始执行一条新的指令"，不对。指令是多周期执行的，一个脉冲只推进一个节拍；"开始执行一条新指令"和"执行完一条指令"也不是一回事。选 D。

错误选项为什么错＋举例：

- A：脉冲源、整形、分频——时钟信号就是这么来的，对的说法。
- B：周期和主频互为倒数——对的说法。
- C：周期以相邻组合逻辑的最大延迟为基准——对的说法，周期定短了，算术还没算完就被下一拍打断了。

笔记本一句话： 第 16 题＋时钟一拍只推进一小步，一条指令要好几拍，不是每拍一条指令。

【图像】



时钟脉冲像节拍器：一条指令要经过取指译码执行访存写回多个节拍

【定位】 计算机组成原理 · 第 5 章中央处理器 · 指令执行过程与时钟周期

【易混对比】

- "时钟脉冲"和"指令"是两回事：脉冲是节拍器的"滴答"声，一个滴答只推进一小步；一条指令要取指、译码、执行、访存、写回好几步，占好几个滴答。
- "时钟周期"和"机器主频"是两回事：时钟周期是两次滴答之间的时间；主频是 1 秒里有多少个滴答，两者互为倒数（主频 ＝ 1 ÷ 时钟周期）。
- "周期定多长"看的是"最慢的那段路"：时钟周期至少要够相邻状态单元（寄存器这种存中间结果的部件）之间的组合逻辑（做加减、译码这类纯计算的电路）算完，以最大延迟为准；定短了，算术还没算完就被下一个滴答打断，结果就错了。

一拍只走一小步，一条指令要好几拍。

【坑位】

"每来一个脉冲就执行一条新指令"——把"推进一小步"偷换成了"干完一条"。

最容易踩的坑是觉得时钟一响就有一条指令从头干到尾。想想具体过程：取指这一步本身就要占一个周期，译码、执行、访存、写回各占周期，一条指令前后要好几个脉冲才走完。所以"每来一个脉冲就开始执行一条新指令"不成立——脉冲只是节拍，不是发令枪。

【类比】

像乐队指挥打拍子：一个拍子乐手只走一步（拉一个音），一首曲子（一条指令）要好几十个拍子才演完。说"每打一个拍子就从头演一首新曲子"，指挥的手得有多忙——不可能。

【口诀】 时钟滴答打节拍，一拍只走一小步；指令多步好几拍，主频周期互为倒。

【考频】 「时钟周期」在 2009–2022 年解析中出现 24 次 · 高频

第 17 题 · 寄存器＋寄存器间接寻址指令需用部件（正确答案：B）

题干与选项：

某指令功能为 R[r2] ← R[r1] + M[R[r0]]，其两个源操作数分别采用寄存器、寄存器间接寻址方式。对于下列给定部件，该指令在取数及执行过程中需要用到的是（）。

Ⅰ. 通用寄存器组（GPRs） Ⅱ. 算术逻辑单元（ALU） Ⅲ. 存储器（Memory） Ⅳ. 指令译码器（ID）

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅰ、Ⅱ、Ⅲ
- C. 仅 Ⅱ、Ⅲ、Ⅳ
- D. 仅 Ⅰ、Ⅲ、Ⅳ

题意概括：一条指令的功能是寄存器等于寄存器加内存里的数，两个源操作数一个是寄存器、一个是寄存器间接寻址，问取数和执行过程中需要用到哪些部件。

入手点： 先分清阶段：题目问的是"取数及执行过程"，不是取指、不是译码；再一个一个部件想：取数要谁、加法要谁、访存要谁。

题干专有名词从零解释：

- 先想"通用寄存器组（GPRs）"：一排寄存器，放操作数和结果。
- 再想"算术逻辑单元（ALU）"：做加减运算的部件。
- 再想"存储器"：内存，放数据。
- 再想"指令译码器（ID）"：把指令翻译成控制信号的部件。
- 再想"寄存器寻址"：操作数就在寄存器里，直接读。
- 再想"寄存器间接寻址"：寄存器里放的是地址，按这个地址去内存取数。
- 再想"取数"和"执行"：取数是把操作数取出来，执行是做运算、写回结果。

考查知识点：

- 先想指令是 R[r2] ← R[r1] + M[R[r0]]。第一个源操作数 R[r1]：寄存器寻址，直接从寄存器组读——要 Ⅰ。
- 再想第二个源操作数 M[R[r0]]：寄存器间接寻址，先读 R[r0] 得到地址，再去存储器取数——要 Ⅰ 和Ⅲ。
- 再想加法运算要 ALU——要 Ⅱ。写回结果到 r2——要 Ⅰ。
- 所以取数及执行要 Ⅰ、Ⅱ、Ⅲ。指令译码器（Ⅳ）是在译码阶段用的，题目问的是取数及执行过程，译码已经做完了，所以Ⅳ不算。

正确选项为什么对： 取数要寄存器组和存储器，运算要 ALU，合起来是 Ⅰ、Ⅱ、Ⅲ，选 B。

错误选项为什么错＋举例：

- A（仅 Ⅰ、Ⅱ）：漏了存储器——第二个操作数在内存里，不访存就拿不到数。
- C（仅 Ⅱ、Ⅲ、Ⅳ）：漏了寄存器组（两个源操作数都要从寄存器组读），还多算了译码器（译码阶段的部件）。
- D（仅 Ⅰ、Ⅲ、Ⅳ）：漏了 ALU（加法谁来做？），还多算了译码器。

笔记本一句话： 第 17 题＋取数执行要谁：寄存器组读数、存储器取数、ALU 做加法；译码器是译码阶段的，不算。

【图像】



取数执行用到的部件：寄存器组读R1与地址R0，ALU做加法，存储器按地址取数

【定位】 计算机组成原理 · 第 4 章指令系统 · 寻址方式

【易混对比】

- "寄存器寻址"和"寄存器间接寻址"是两回事：前者"数就在寄存器里"，直接读；后者"寄存器里装的是地址"，要拿着这个地址再去存储器取一趟数——多一次访存。
- "取数执行阶段"和"译码阶段"是两回事：题目问的是取数和执行要用谁，译码（把指令翻译成控制信号）在更早的阶段已经干完，指令译码器这时候已经下班了。
- "ALU"和"存储器"是两回事：ALU（算术逻辑单元，专门做加减运算的部件）负责算；存储器负责存数；一个管算，一个管存，别互相顶替。

按阶段点名：取数要寄存器组和存储器，执行要 ALU；译码器不算。

【坑位】

先看清题目问的是哪个阶段，再点名；"顺手"多选译码器是重灾区。

最容易踩的坑是看到"指令"就把译码器也勾上。题目问的是"取数及执行过程"：R[r1] 从寄存器组直接读（要Ⅰ），R[r0] 里是地址、拿着它去存储器取数（要Ⅰ和Ⅲ），加法运算（要Ⅱ），结果写回寄存器（要Ⅰ）。指令译码器（Ⅳ）是译码阶段用的，取数执行时它已经没戏了，勾上就错。

【套路】

- 第一步，拆操作数：R[r1] 是寄存器寻址 → 要寄存器组；M[R[r0]] 是寄存器间接寻址 → 要寄存器组读地址＋存储器取数。
- 第二步，拆运算：加法 → 要 ALU；结果写回 → 要寄存器组。
- 第三步，卡阶段：问的是取数执行，译码阶段的指令译码器排除。

拆操作数 → 拆运算 → 卡阶段，Ⅰ Ⅱ Ⅲ 不含译码器。

【类比】

像厨房做菜：寄存器组是案板上的备菜（顺手拿到），存储器是地窖（要按地址跑一趟去取），ALU 是灶台（负责炒），指令译码器是菜谱翻译（开工前就翻译完了，做菜时不用再翻）。问"做菜要用到谁"，翻译员已经下班了。

【口诀】 寄存直读间址访，ALU 管算存管放；译码下班别点名，取数执行三件套。

【考频】 「寄存器间接寻址」在 2009–2022 年解析中出现 9 次 · 中频

第 18 题 · 5 段流水线指令序列、无数据冒险的指令对（正确答案：C）

题干与选项：

在采用"取指、译码/取数、执行、访存、写回"5 段流水线的处理器中，执行如下指令序列，其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

```
I1: add s2, s1, s0    // R[s2] ← R[s1]+R[s0]
I2: load s3, 0(t2)    // R[s3] ← M[R[t2]+0]
I3: add s2, s2, s3    // R[s2] ← R[s2]+R[s3]
I4: store s2, 0(t2)   // M[R[t2]+0] ← R[s2]
```

下列指令对中，不存在数据冒险的是（）。

- A. I1 和 I3
- B. I2 和 I3
- C. I2 和 I4
- D. I3 和 I4

题意概括：5 段流水线里有四条指令，问列出的四对指令里，哪一对不存在数据冒险。

入手点： 数据冒险就是"后面指令要读前面指令还没写好的数"；一对一对查：后面读的寄存器，前面有没有写。

题干专有名词从零解释：

- 先想"5 段流水线"：取指、译码/取数、执行、访存、写回，五步像流水线一样重叠干。
- 再想"数据冒险"：后一条指令要用的数据，前一条还没算好、还没写回。
- 再想"寄存器编号"：s0、s1、s2、s3、t2 都是寄存器名字。
- 再想"load"：从内存取数到寄存器；"store"是把寄存器里的数存到内存；"add"是加法。

考查知识点：

- 先想四条指令：I1 写 s2（读 s1、s0）；I2 写 s3（读 t2，访存）；I3 写 s2（读 s2、s3）；I4 读 s2、t2，写内存（不写寄存器）。
- 一对一对查：I1 和 I3：I3 读 s2，I1 写 s2——有冒险。I2 和 I3：I3 读 s3，I2 写 s3——有冒险（还是 load 后面紧跟用，典型的 load-use）。I3 和 I4：I4 读 s2，I3 写 s2——有冒险。I2 和 I4：I4 读 s2 和 t2；I2 写的是 s3，和 s2、t2 都不沾；t2 是两条指令共读（读不冲突，只有写才冲突）——没有冒险。

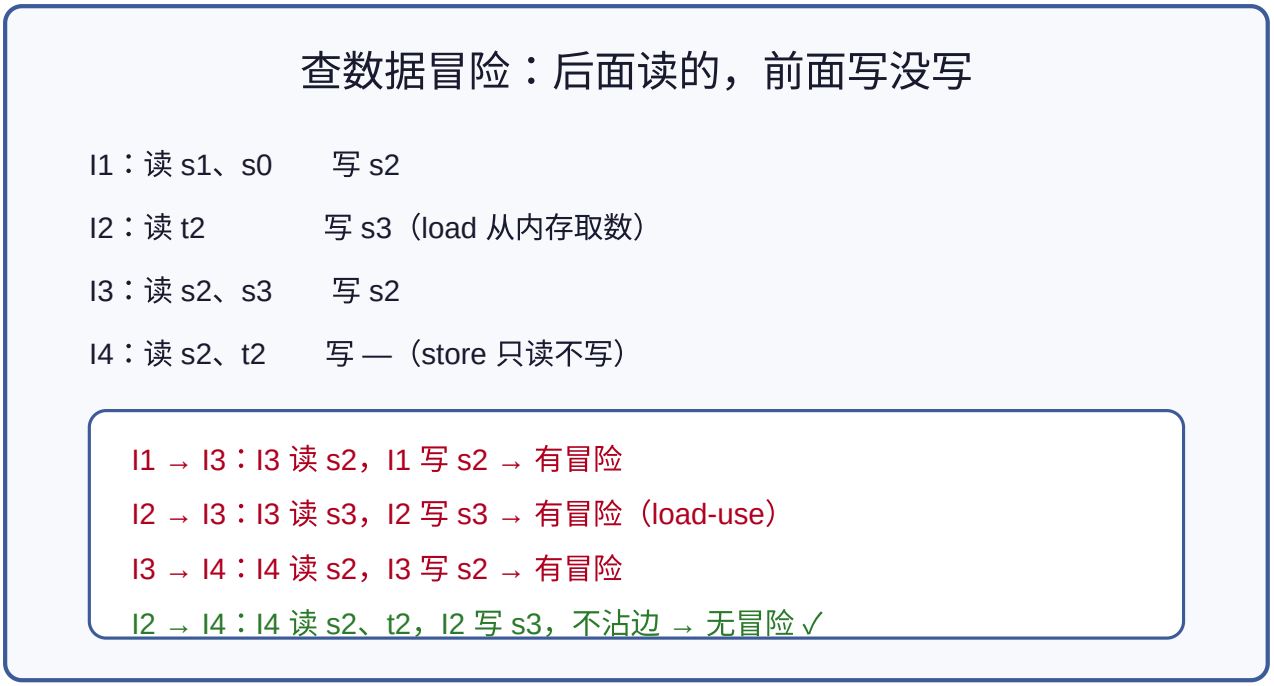
正确选项为什么对： I2 写 s3，I4 读 s2、t2，不沾边，所以没冒险，选 C。

错误选项为什么错＋举例：

- A（I1 和 I3）：I3 要读 s2，I1 刚写 s2，冒险。
- B（I2 和 I3）：I3 要读 s3，I2 刚从内存取数写 s3，冒险。
- D（I3 和 I4）：I4 要读 s2，I3 刚写 s2，冒险。
- 教训：查冒险就看"后面读、前面写"是不是同一个寄存器；两个都读同一个寄存器不算冒险。

笔记本一句话： 第 18 题＋查数据冒险：后面读、前面写，同一个寄存器就是冒险；I2 写 s3，I4 读 s2、t2，不沾边所以没冒险。

【图像】



查数据冒险：后面读、前面写，同一寄存器才算；I2写s3、I4读s2t2不沾边

【定位】 计算机组成原理 · 第 5 章中央处理器 · 指令流水线与数据冒险

【易混对比】

- "数据冒险"和"结构冒险"是两回事：数据冒险是"后面指令要读的数，前面指令还没写好"；结构冒险是"两条指令抢同一个硬件"。本题只查数据冒险。
- "读-写冲突"和"读-读"是两回事：后面读、前面写同一个寄存器才是冒险；两条指令都只读同一个寄存器，谁也不改谁，不算冒险。
- "load-use"是数据冒险里最凶的一种：load（从内存取数到寄存器的指令）刚把数取出来，下一条指令马上要用——数还在路上，最容易出事。

查冒险只看一句话：后面读的，前面写没写。

【坑位】

store 指令不写寄存器：I4 是 store，它读 s2、t2 往内存里存，不写任何寄存器。

最容易踩的坑是看到 I4 里有 s2 就以为它"写"了 s2。先看指令是干什么的：I4 是 store，把寄存器里的数存到内存，它只读不写。四对里：I1 写 s2 被 I3 读（冒险），I2 写 s3 被 I3 读（冒险，还是 load-use），I3 写 s2 被 I4 读（冒险）；只有 I2 写 s3、I4 读 s2 和 t2，两边不沾，才没有冒险。

【套路】

- 第一步，列出每条指令的读集合和写集合：I1 写 s2；I2 写 s3；I3 读 s2、s3，写 s2；I4 读 s2、t2，写内存（不写寄存器）。
- 第二步，一对一对查：后面读的寄存器，有没有出现在前面写的集合里。
- 第三步，读-读不算：t2 两条都读，不冲突。

列读写集 → 后读前写对号 → 读读不算。

【类比】

像接力赛交棒：前面的人（写）还没把棒（数据）递出来，后面的人（读）就伸手要——这就是冒险，两人抓同一根棒才出事。如果两人都只是看看棒（都读），谁也不抢，就没事。

【口诀】 后读前写同寄存，冒险就在这一对；读读不算写才算，store 只读不写人。

【考频】 「数据冒险」在 2009–2022 年解析中出现 15 次 · 高频

第 19 题 · 3 通道 DDR3-1333 存储器总线总带宽（正确答案：B）

题干与选项：

假定一台计算机采用 3 通道存储器总线，配套的内存条型号为 DDR3-1333，即内存条所接插的存储器总线的工作频率为 1333MHz，总线宽度为 64 位，则存储器总线的总带宽大约是（ ）。

- A. 10.66GB/s
- B. 32GB/s
- C. 64GB/s
- D. 96GB/s

题意概括：3 通道存储器总线，内存条是 DDR3-1333，总线宽度 64 位，问总线总带宽大约是多少。

入手点： 先算一条通道的带宽，再乘 3；关键是 1333 这个数已经是每秒传输次数，不要再乘 2。

题干专有名词从零解释：

- 先想"通道"：内存条和 CPU 之间独立的数据路，3 通道就是 3 条路并行。
- 再想"DDR3-1333"：每秒传输 1333M 次的内存条型号，这里的 1333 已经是算上"双倍速率"后的传输次数。
- 再想"总线宽度"：一次传多少位，这里是 64 位。
- 再想"带宽"：1 秒能传多少字节。

考查知识点：

- 先算一条通道：1333M 次/秒 × 64 位 ÷ 8 = 1333M × 8 字节 = 10.664GB/s，大约 10.66GB/s。
- 再算三条通道并行：10.66 × 3 = 32GB/s。
- 注意：1333MT/s 里的"双倍"已经算进去了——DDR 是时钟上下沿都传数据，1333 这个数就是上下沿加起来的传输次数；如果再乘 2，就多算了一倍。

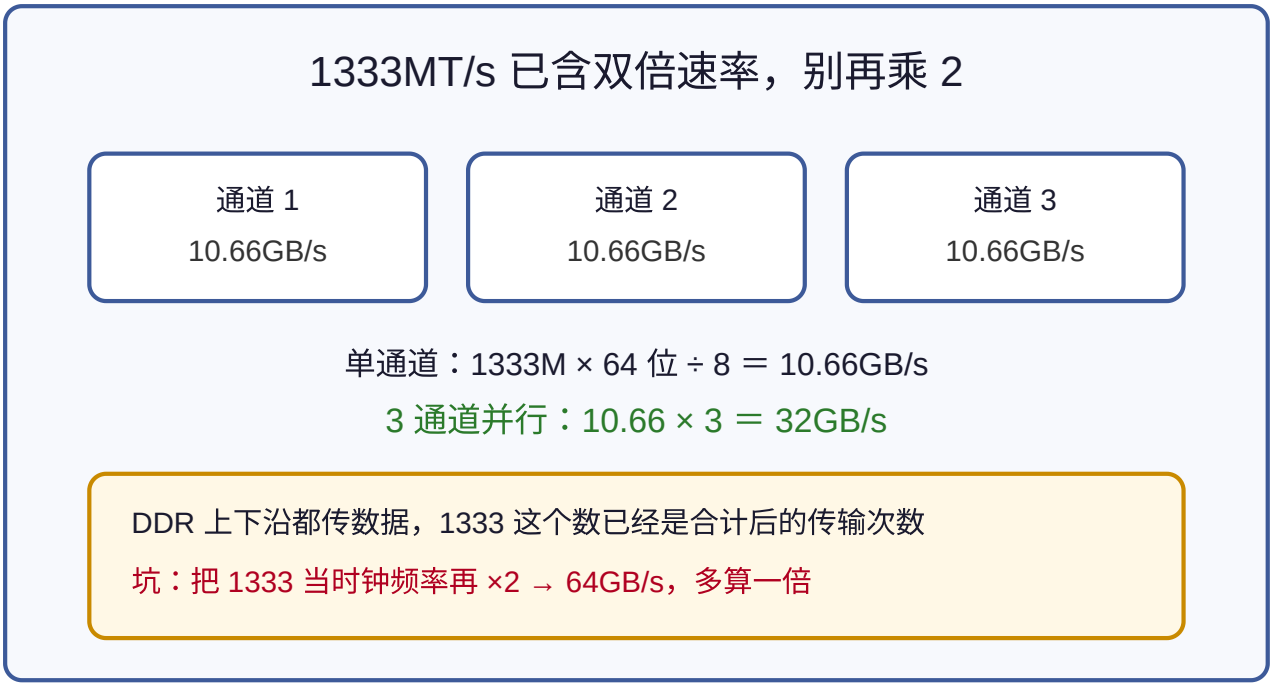
正确选项为什么对： 单通道约 10.66GB/s，3 通道就是 32GB/s，选 B。

错误选项为什么错＋举例：

- A（10.66GB/s）：只算了一条通道的，忘了乘 3。
- C（64GB/s）：多乘了 2——把 1333 当成时钟频率又乘了一次双倍速率，实际上 1333MT/s 里已经含了双倍。
- D（96GB/s）：通道数或者双倍速率算重了（比如在 32 的基础上又乘了 3）。

笔记本一句话： 第 19 题＋DDR 带宽：1333MT/s 已含双倍，单通道 1333×8≈10.66GB/s，3 通道就是 32GB/s。

【图像】



单通道 1333×64/8≈10.66GB/s，3通道×3=32GB/s；1333已含双倍速率

【定位】 计算机组成原理 · 第 6 章总线 · 总线带宽计算

【易混对比】

- "1333MT/s"和"1333MHz 时钟"是两回事：DDR（时钟上沿下沿都传数据的内存）标号里的 1333 是每秒传输 1333M 次，已经把"上下沿各传一次"的双倍算进去了；再乘 2 就是重复计算。
- "位"和"字节"是两回事：总线宽度 64 位是 64 个比特，算带宽要除以 8 换成字节——64 位 ＝ 8 字节。
- "单通道"和"多通道"是两回事：先算一条通道的带宽，再乘通道数；3 通道就是 3 条路并行传。

1333 已含双倍别再乘；单通道 10.66GB/s，3 通道 32GB/s。

【坑位】

看到 DDR 就想乘 2：先看单位——MT/s 里"双倍"已经算过，MHz 才要乘。

最容易踩的坑是把 1333 当成时钟频率再乘一次 2，算出 64GB/s。记住：型号里的 1333 后面跟的是 MT/s（每秒百万次传输），这个"次"已经是上下沿加起来的。正确算法：1333M × 64 位 ÷ 8 ＝ 10.66GB/s（单通道），再 × 3 通道 ＝ 32GB/s。

【套路】

- 第一步，定传输次数：DDR3-1333 ＝ 1333M 次/秒，双倍已含，不再乘 2。
- 第二步，算单通道：1333M × 64 位 ÷ 8 ＝ 10.66GB/s。
- 第三步，乘通道数：10.66 × 3 ＝ 32GB/s。

定次数（不重乘） → 单通道 → 乘通道数。

【类比】

像 3 条并行的传送带：每条每秒过 1333M 个包裹（1333 这个数已经是"正反两面都计数"后的总数），每个包裹 8 字节。一条带每秒 10.66GB，3 条就是 32GB。非要把"正反两面"再数一遍，数量就虚增一倍了。

【口诀】 双倍已含别重乘，位除以八换字节；单道十点六六，三道三十二。

【考频】 「存储器带宽」在 2009–2022 年解析中出现 2 次 · 低频

第 20 题 · 磁盘存储器叙述、错误项（正确答案：C）

题干与选项：

下列关于磁盘存储器的叙述中，错误的是（ ）。

- A. 磁盘的格式化容量比非格式化容量小
- B. 扇区中包含数据、地址和校验等信息
- C. 磁盘存储器的最小读写单位为一字节
- D. 磁盘存储器由磁盘控制器、磁盘驱动器和盘片组成

题意概括：关于磁盘存储器的四句话，问哪句是错的。

入手点： 本题问"错误的是"，先想磁盘读写的最小单位：不是字节，是扇区。

题干专有名词从零解释：

- 先想"磁盘存储器"：硬盘，靠磁头在盘片上读写。
- 再想"格式化容量"：格式化后真正能用的容量；"非格式化容量"是盘片物理上能存的总量。
- 再想"扇区"：磁盘上最小的读写单位，一段连续的磁道区域，带数据、地址、校验信息。
- 再想"磁盘控制器"：管读写的控制电路；"磁盘驱动器"是转盘片、移动磁头的机械部分；"盘片"是存数据的圆盘。

考查知识点：

- 先想 A：格式化后一部分空间被管理信息（地址、校验、间隙）占了，所以格式化容量比非格式化容量小。对的。
- 再想 B：扇区里除了数据，还有地址信息（这是哪一扇区）和校验信息（检查读对了没有）。对的。
- 再想 C：磁盘最小读写单位是扇区，不是一字节——磁头一次至少读/写一个扇区，想改 1 个字节也要把整个扇区读出来、改完再写回去。
- 再想 D：磁盘存储器就是控制器、驱动器、盘片三部分组成的。对的。

正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。C 说最小读写单位是一字节，不对。字节是主存的编址单位，磁盘的读写单位是扇区。选 C。

错误选项为什么错＋举例：

- A：格式化要花掉一部分空间做管理，所以格式化容量小——对的说法。
- B：扇区等于数据加地址加校验——对的说法。
- D：控制器、驱动器、盘片——对的说法。

笔记本一句话： 第 20 题＋磁盘最小读写单位是扇区，不是一字节。

【图像】



磁盘最小读写单位是扇区：改1字节也要整扇区读出改完写回

【定位】 计算机组成原理 · 第 3 章存储系统 · 辅助存储器（磁盘）

【易混对比】

- "扇区"和"字节"是两回事：字节是主存的编址单位；磁盘读写的最小单位是扇区——磁头一次至少读/写一个扇区。
- "格式化容量"和"非格式化容量"是两回事：格式化要划出一部分空间写管理信息（地址、校验、间隙），所以格式化后能用的反而比物理总量小。
- "磁盘控制器"和"磁盘驱动器"是两回事：控制器是管读写的电路（发命令），驱动器是转盘片、移动磁头的机械部分（干活）。

磁盘按扇区读写，想改 1 个字节也要搬整个扇区。

【坑位】

一看到"最小单位"就想字节：先问"这是主存还是磁盘"，磁盘答案永远是扇区。

最容易踩的坑是把主存的习惯带到磁盘上。主存按字节编址没错，但磁盘的磁头一次读写最少一个扇区——想改 1 个字节，得把整个扇区读出来、改完再写回去。所以"最小读写单位是一字节"这句一听就是把主存和磁盘搞混了。

【套路】

- 第一步，定考的是哪种存储器：磁盘 → 读写单位按扇区想。
- 第二步，逐句验：格式化容量变小（管理信息占地方）→ 对；扇区含地址和校验 → 对；控制器/驱动器/盘片三部分 → 对。
- 第三步，剩下的"字节"就是错项：磁盘最小读写单位是扇区。

定存储器 → 逐句验 → 字节项出局。

【类比】

像图书馆借书：你想看其中一页（1 个字节），管理员不会把那页撕下来给你——必须整本借走（一个扇区），看完还整本。磁盘的"一本书"就是一个扇区。

【口诀】 磁盘读写按扇区，字节是主存的规矩；格式化后容量小，管理信息占了地。

【考频】 「磁盘」在 2009–2022 年解析中出现 57 次 · 高频

第 21 题 · 中断方式数据交换、CPU 时间占比（正确答案：A）

题干与选项：

某设备以中断方式与 CPU 进行数据交换，CPU 主频为 1GHz，设备接口中的数据缓冲寄存器为 32 位，设备的数据传输率为 50kB/s。若每次中断开销（包括中断响应和中断处理）为 1000 个时钟周期，则 CPU 用于该设备输入/输出的时间占整个 CPU 时间的百分比最多是（）。

- A. 1.25%
- B. 2.5%
- C. 5%
- D. 12.5%

题意概括：设备用中断方式和 CPU 换数据，给出主频、缓冲寄存器位数、设备速率、每次中断开销，问 CPU 花在输入输出上的时间占整个 CPU 时间的百分比最多是多少。

入手点： 先算 1 秒要中断几次（速率除以每次传的字节数），再算 1 秒中断花多少周期，最后除以主频。

题干专有名词从零解释：

- 先想"中断方式"：设备准备好数据就打断 CPU，CPU 停下手头活去处理。
- 再想"主频"：1 秒多少个时钟周期，这里是 1GHz 就是 10 亿个。
- 再想"数据缓冲寄存器"：设备接口里暂存数据的寄存器，32 位就是 4 个字节。
- 再想"数据传输速率"：设备 1 秒传多少，这里是 50kB/s。
- 再想"中断开销"：一次中断从响应到处理完花的时钟周期，这里是 1000 个。

考查知识点：

- 先算一次中断传多少：32 位除以 8 等于 4 个字节。
- 再算 1 秒中断几次：设备 1 秒传 50kB，等于 50000 个字节，50000 除以 4 等于 12500 次。
- 再算 1 秒中断花多少周期：12500 次乘以 1000 个周期，等于 1250 万个周期。
- 再算占比：主频 1GHz 就是 1 秒 10 亿个周期，1250 万除以 10 亿等于 1.25%。
- 注意"最多"：设备按最高速率传时中断最频繁，占比最大；速率低了占比更小。

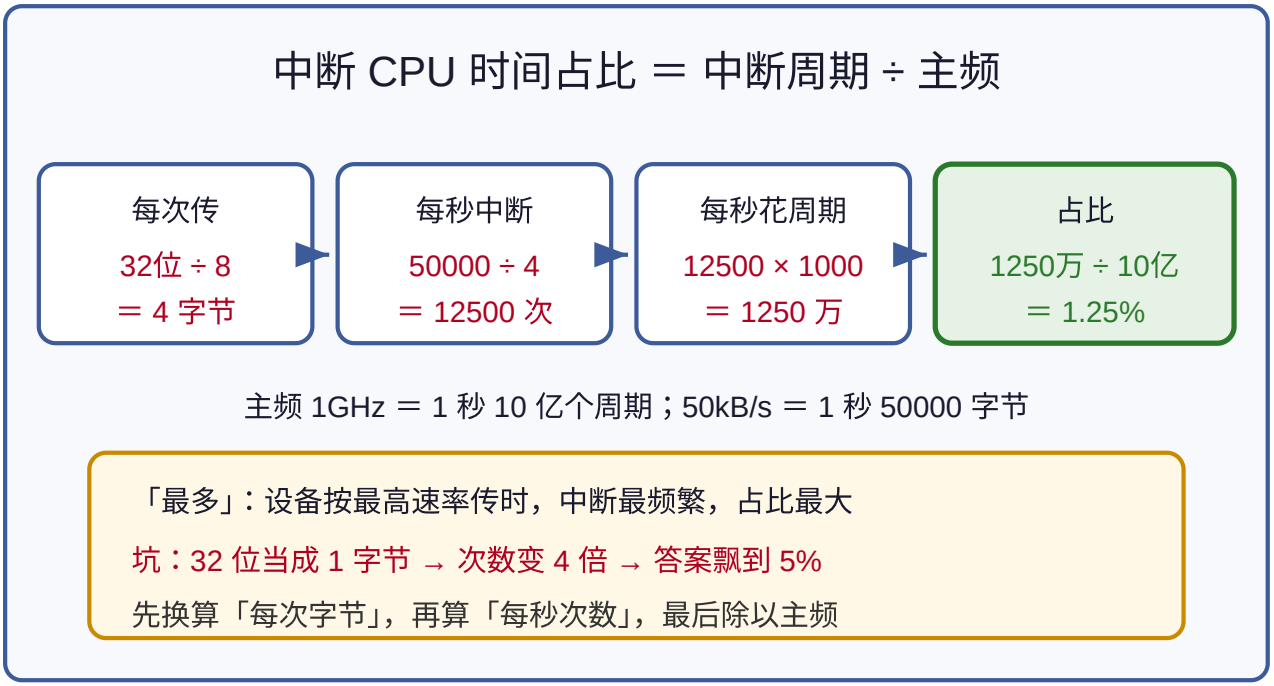
正确选项为什么对： 1 秒中断 12500 次，花 1250 万周期，占 10 亿周期的 1.25%，选 A。

错误选项为什么错＋举例：

- B（2.5%）：相当于一次中断只传了 2 个字节（把 32 位当成 16 位），中断次数翻了一倍。
- C（5%）：相当于一次中断传 1 个字节（忘了 32 位是 4 个字节），次数变成 4 倍。
- D（12.5%）：相当于把 50kB 当成了 500kB，或者周期数多算了一个 0。

笔记本一句话： 第 21 题＋中断占比：1 秒中断 50000÷4=12500 次，花 1250 万周期，占 10 亿周期的 1.25%。

【图像】



1秒中断12500次×1000周期=1250万周期，占10亿周期的1.25%

【定位】 计算机组成原理 · 第 7 章输入输出系统 · 中断方式

【易混对比】

- "中断次数"和"传输字节数"是两回事：中断次数 = 总字节数 ÷ 每次中断传的字节数；32 位缓冲寄存器一次传 4 个字节，不是 1 个。
- "周期数"和"时间占比"是两回事：先算 1 秒中断花掉多少个时钟周期，再除以 1 秒的总周期数（主频），才是占比。
- "最多"和"实际"是两回事：设备按最高速率传时中断最频繁，CPU 花的时间最多；速率降下来，占比只会更小。

次数 = 总字节 ÷ 每次字节；占比 = 中断周期 ÷ 主频。

【坑位】

32 位先除以 8：一次中断传 4 个字节，不是 32 个，也不是 1 个。

最容易踩的坑是单位换算：32 位缓冲寄存器，一次中断传 32 位 = 4 字节。50kB/s 就是 1 秒 50000 字节，50000 ÷ 4 = 12500 次中断；每次 1000 周期，1 秒花 1250 万周期；主频 1GHz 是 1 秒 10 亿周期，1250 万 ÷ 10 亿 = 1.25%。把 32 位当成 1 字节，次数就变成 4 倍，答案就飘到 5% 去了。

【套路】

- 第一步，算每次传多少：32 位 ÷ 8 = 4 字节。
- 第二步，算 1 秒中断几次：50000 ÷ 4 = 12500 次。
- 第三步，算 1 秒花多少周期：12500 × 1000 = 1250 万。
- 第四步，除以主频：1250 万 ÷ 10 亿 = 1.25%。

每次字节 → 中断次数 → 周期总数 → 除以主频。

【类比】

像快递分拣：传送带 1 秒送来 50000 件小包裹（字节），每辆小车一次装 4 件（32 位缓冲），1 秒要发 12500 趟车（中断）；每趟车调度要花 1000 秒（周期）。算调度时间占全天比例，就是"趟数 × 每趟耗时 ÷ 全天时长"。

【口诀】 位除以八算字节，总量除之得次数；次数乘上千周期，除以主频得占比。

【考频】 「中断方式」在 2009–2022 年解析中出现 6 次 · 中频

第 22 题 · DMA 方式叙述、正确项（正确答案：D）

题干与选项：

下列关于 DMA 方式的叙述中，正确的是（ ）。

Ⅰ. DMA 传送前由设备驱动程序设置传送参数

Ⅱ. 数据传送前由 DMA 控制器请求总线使用权

Ⅲ. 数据传送由 DMA 控制器直接控制总线完成

Ⅳ. DMA 传送结束后的处理由中断服务程序完成

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅰ、Ⅲ、Ⅳ
- C. 仅Ⅱ、Ⅲ、Ⅳ
- D. Ⅰ、Ⅱ、Ⅲ、Ⅳ

题意概括：关于 DMA 方式的四句话，问哪几句对。

入手点： 按 DMA 传送的时间线想：传送前谁准备、传送中谁控制、传送完谁收尾——四句话正好对应这三个阶段。

题干专有名词从零解释：

- 先想"DMA（直接存储器访问）"：设备和内存直接传数据，不经过 CPU 一个字节一个字节搬。
- 再想"设备驱动程序"：操作系统里管某个设备的程序。
- 再想"DMA 控制器"：专门管 DMA 传送的硬件。
- 再想"总线使用权"：用总线传数据的资格，CPU 和 DMA 控制器轮流用。
- 再想"中断服务程序"：中断来了去处理的那段程序。

考查知识点：

- 先想Ⅰ：传送前由设备驱动程序设置传送参数（从哪读、往哪写、传多少）——对的。
- 再想Ⅱ：数据传送前由 DMA 控制器请求总线使用权（拿到总线才能传）——对的。
- 再想Ⅲ：数据传送由 DMA 控制器直接控制总线完成（一个字节一个字节搬的是 DMA 控制器，不是 CPU）——对的。
- 再想Ⅳ：传送结束后的处理由中断服务程序完成（DMA 控制器发个中断，告诉 CPU"传完了"，中断服务程序去收尾）——对的。

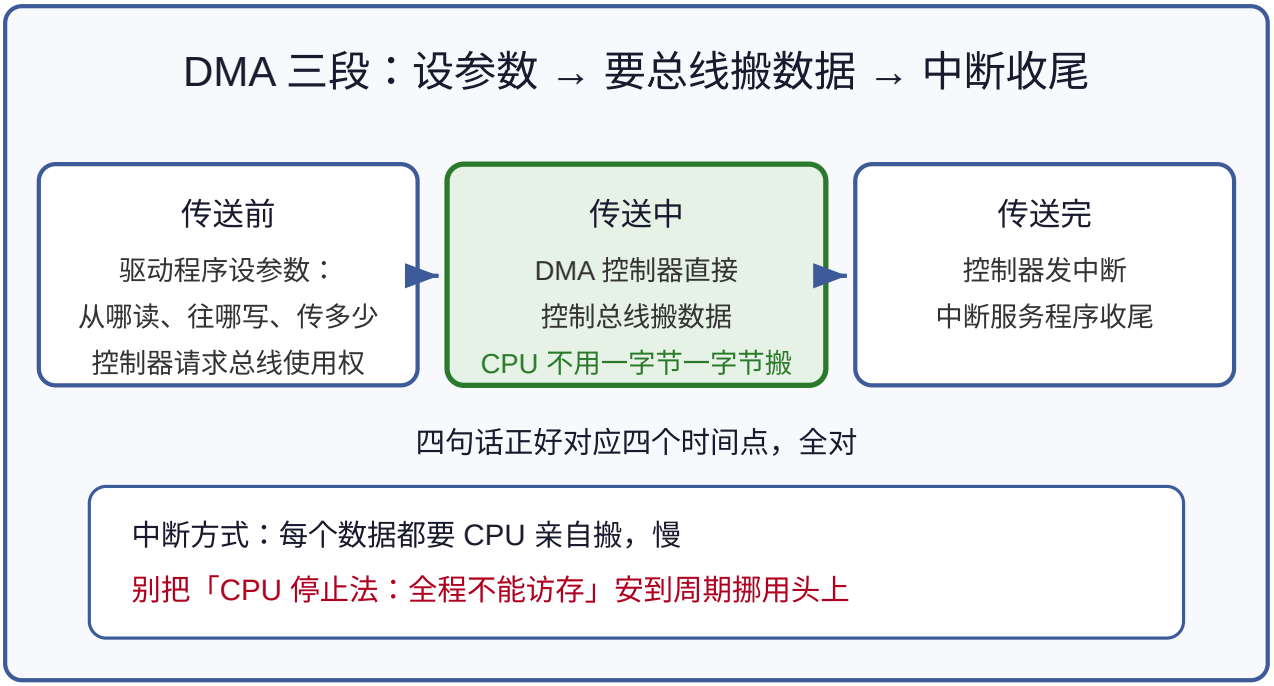
正确选项为什么对： Ⅰ、Ⅱ、Ⅲ、Ⅳ全对，选 D。

错误选项为什么错＋举例：

- A（仅Ⅰ、Ⅱ）：漏了传送中的控制和传送完的收尾——DMA 的核心就是传送中不用 CPU、结束靠中断。
- B（仅Ⅰ、Ⅲ、Ⅳ）：漏了请求总线使用权——不拿总线就传不了。
- C（仅Ⅱ、Ⅲ、Ⅳ）：漏了传送前驱动程序设置参数——不设置参数，DMA 控制器不知道往哪传。

笔记本一句话： 第 22 题＋DMA 三段：传送前驱动设参数，传送前控制器要总线，传送中控制器搬数据，传送完中断程序收尾。

【图像】



DMA三段：传送前驱动设参数+控制器要总线，传送中控制器搬数据，传送完中断收尾

【定位】 计算机组成原理 · 第 7 章输入输出系统 · DMA 方式

【易混对比】

- "DMA"和"中断方式"是两回事：中断方式每个数据都要 CPU 亲自搬；DMA（直接存储器访问，设备和内存直连传数据）传送中不用 CPU 一个字节一个字节搬，只在开始和结束时麻烦 CPU。
- "DMA 控制器"和"CPU"是两回事：传送中真正一个字节一个字节搬数据的是 DMA 控制器这个硬件；CPU 这段时间可以去干别的。
- "周期挪用"和"CPU 停止法"是两回事：周期挪用（DMA 只借 CPU 不用的时钟周期，像见缝插针）下 CPU 还能访存；CPU 停止法下 DMA 占着总线不放，CPU 全程不能访存；别把停止法的特征安到周期挪用头上。

DMA 三段：驱动设参数 → 控制器要总线搬数据 → 中断收尾。

【坑位】

传送"前"和传送"中"分开看：要总线是传送前，搬数据是传送中。

最容易踩的坑是把"DMA 控制器请求总线使用权"当成传送中的事。按时间线捋：传送前，设备驱动程序（操作系统里管设备的程序）先设置好"从哪读、往哪写、传多少"；也是传送前，DMA 控制器先请求总线使用权（拿到总线这个"路权"才能传）；传送中，DMA 控制器直接控制总线搬数据；传送完，发个中断，中断服务程序去收尾。四句话正好对应四个时间点，全对。

【套路】

- 第一步，画时间线：传送前 → 传送中 → 传送完。
- 第二步，对号入座：驱动设参数（前）、控制器要总线（前）、控制器搬数据（中）、中断服务程序收尾（后）。
- 第三步，全对就选全对：四个阶段都没毛病，Ⅰ Ⅱ Ⅲ Ⅳ全选。

画时间线 → 对号入座 → 全对就全选。

【类比】

像搬家公司搬家：你先告诉他们"从 A 搬到 B、搬多少箱"（驱动设参数），他们先去物业拿小区通行证（要总线使用权），然后工人一箱一箱搬（控制器搬数据，你不用亲自动手），搬完给你打电话验收（中断收尾）。

【口诀】 驱动先行设参数，控器要线再开搬；传送全程无 CPU，完事中断来收官。

【考频】 「DMA」在 2009–2022 年解析中出现 16 次 · 高频

第 23 题 · 线程描述、错误项（正确答案：B）

题干与选项：

下列关于线程的描述中，错误的是（）。

- A. 内核级线程的调度由操作系统完成
- B. 操作系统为每个用户级线程建立一个线程控制块
- C. 用户级线程间的切换比内核级线程间的切换效率高
- D. 用户级线程可以在不支持内核级线程的操作系统上实现

题意概括：关于线程的四句话，问哪句是错的。

入手点： 本题问"错误的是"，先分清两种线程：内核级是操作系统认识的，用户级是操作系统不认识的——"操作系统为每个用户级线程建控制块"这句一听就可疑。

题干专有名词从零解释：

- 先想"线程"：进程里的一条执行线，一个进程可以有多个线程。
- 再想"内核级线程"：操作系统内核认识、由操作系统调度的线程。
- 再想"用户级线程"：用户程序自己管的线程，操作系统看不见。
- 再想"线程控制块"：记录线程状态、寄存器、栈等信息的一张表。
- 再想"切换"：从一个线程换到另一个线程执行。

考查知识点：

- 先想 A：内核级线程的调度由操作系统完成——内核认识它们，当然由内核调度。对的。
- 再想 B：操作系统为每个用户级线程建立一个线程控制块——不对。用户级线程对操作系统是透明的，操作系统只认识进程；用户级线程的控制块是由用户级的线程库建立和管理的，操作系统根本不知道这些线程的存在。
- 再想 C：用户级线程间的切换比内核级线程间的切换效率高——用户级切换不用陷入内核，不用切换特权态，开销小。对的。
- 再想 D：用户级线程可以在不支持内核级线程的操作系统上实现——因为它全靠用户程序自己管，不依赖操作系统提供内核级线程。对的。

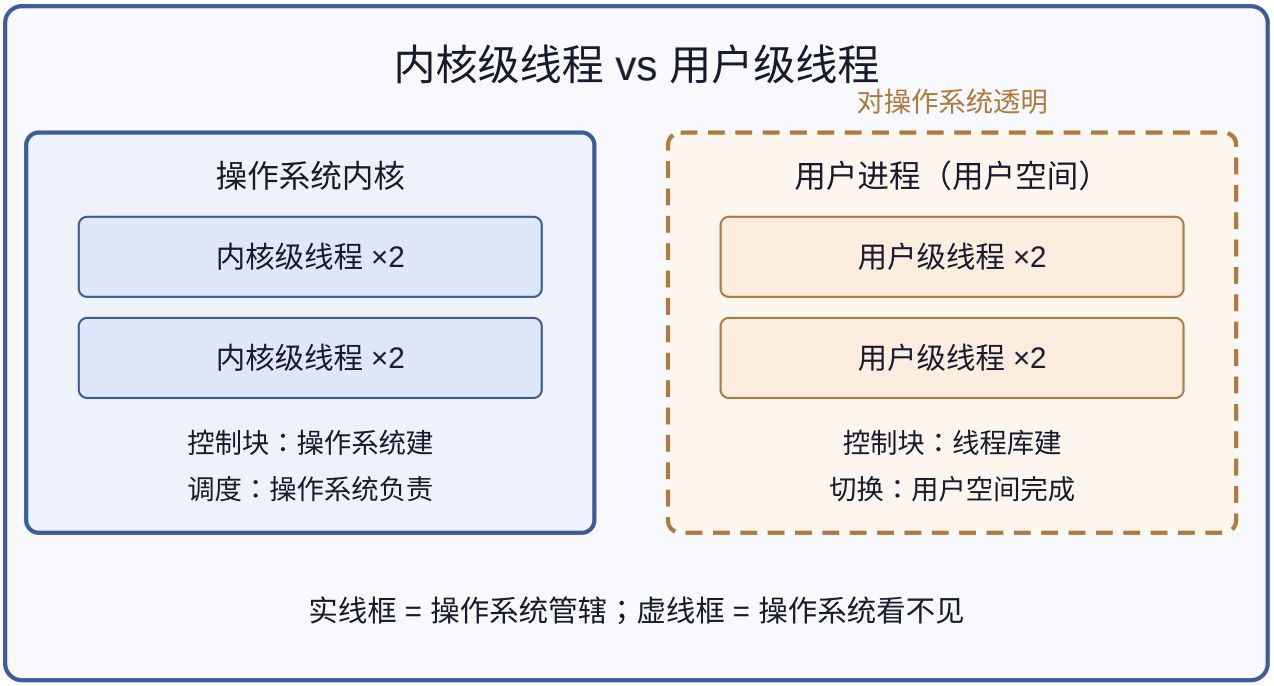
正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。B 说操作系统给每个用户级线程建控制块，不对。用户级线程的控制块在用户空间、由线程库管；操作系统只给内核级线程建控制块。选 B。

错误选项为什么错＋举例：

- A：内核级线程归操作系统调度——对的说法。
- C：用户级切换不用进内核，所以快——对的说法。
- D：用户级线程不依赖内核级线程支持——对的说法。

笔记本一句话： 第 23 题＋用户级线程操作系统看不见，控制块是线程库建的，不是操作系统建的。

【图像】



内核级线程与用户级线程：操作系统只看得见内核级线程

【定位】 操作系统 · 第2章进程管理 · 线程模型（内核级线程与用户级线程）

【易混对比】

- "内核级线程"和"用户级线程"是两回事：前者操作系统看得见、由操作系统调度；后者操作系统看不见，全靠用户程序自己的线程库管理。
- "调度"和"切换"是两回事：调度是"决定下一个跑谁"，这是操作系统的活；切换是"保存现场、换人上场"的动作，用户级线程的切换可以在用户空间完成，不必惊动操作系统。
- "操作系统建的控制块"和"线程库建的控制块"是两回事：操作系统只给自己认识的内核级线程建控制块；用户级线程的控制块建在用户空间、由线程库管，操作系统里没有这份记录。

操作系统看不见用户级线程：只调度内核级线程，只给内核级线程建控制块；用户级线程的控制块是线程库建的。

【坑位】

看见"用户级线程"，先默念"系统看不见"：凡是说"操作系统给用户级线程建表、分配、调度"的说法，统统判错。

最容易踩的坑是把"线程"当成一个整体背结论——背了"线程由操作系统调度"，一看到用户级线程也套上去。用户级线程对操作系统是透明的：操作系统连它存在都不知道，怎么可能给它建控制块、怎么可能调度它？正确做法是先看题在说哪种线程：内核级归操作系统；用户级归线程库，操作系统看不见。

【类比】

像学校和学生小群。学校（操作系统）只认有学籍的学生（内核级线程）：点名、排课、建档案都只管他们。几个同学自己拉的兴趣小群（用户级线程），群主（线程库）管群规、管换届，学校根本不知道这个群存在——学校不可能给小群成员建学籍档案。用户级线程就是那个"学校不知道的小群"。

【口诀】 内核线程系统管，用户线程系统盲；谁的控制块谁来建，库建库管别搞反。

【考频】 「线程」在 2009–2022 年解析中出现 9 次 · 中频

第 24 题 · 可能唤醒进程的事件（正确答案：C）

题干与选项：

下列选项中，可能会将进程唤醒的事件是（）。

Ⅰ. I/O 结束 Ⅱ. 某进程退出临界区 Ⅲ. 当前进程的时间片用完

- A. 仅 Ⅰ
- B. 仅Ⅲ
- C. 仅 Ⅰ、 Ⅱ
- D. Ⅰ、 Ⅱ、Ⅲ

题意概括：三个事件，问哪些可能会把进程唤醒。

入手点： 先分清"唤醒"是什么意思：阻塞变就绪；再看每个事件是让谁从阻塞变就绪，还是只影响运行和就绪之间的调度。

题干专有名词从零解释：

- 先想"进程"：正在运行的程序。
- 再想"唤醒"：阻塞态变就绪态。
- 再想"I/O 结束"：等的那次输入输出做完了。
- 再想"临界区"：一次只允许一个进程进入的那段代码。
- 再想"时间片"：轮转调度里一次分给进程的 CPU 时间，用完就换下一个。

考查知识点：

- 先想Ⅰ I/O 结束：等 I/O 的进程一直在阻塞，I/O 一完就被唤醒变就绪——会唤醒。
- 再想Ⅱ 某进程退出临界区：等在临界区外面的进程一直在阻塞，里面那个一出来，外面那个就被唤醒去抢临界区——会唤醒。
- 再想Ⅲ 当前进程的时间片用完：当前进程是从运行态变就绪态（被剥夺 CPU），不是从阻塞态变就绪态；"唤醒"专指阻塞变就绪，时间片用完不唤醒任何人。

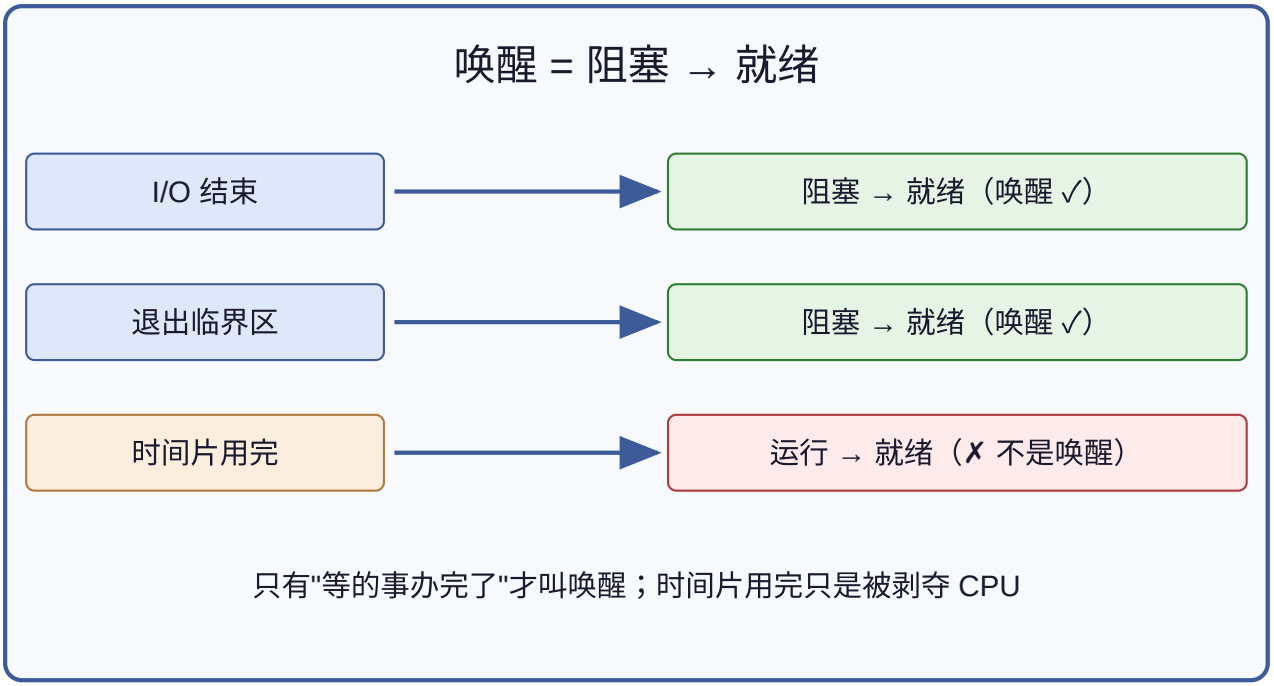
正确选项为什么对： Ⅰ、Ⅱ会唤醒，Ⅲ不会，选 C（仅 Ⅰ、 Ⅱ）。

错误选项为什么错＋举例：

- A（仅 Ⅰ）：漏了 Ⅱ——等临界区的进程也是阻塞，里面一出来就唤醒。
- B（仅Ⅲ）：时间片用完是运行变就绪，不是唤醒，Ⅲ根本不算。
- D（ Ⅰ、 Ⅱ、Ⅲ）：Ⅲ多算了，时间片用完不唤醒。

笔记本一句话： 第 24 题＋唤醒专指阻塞变就绪：I/O 完、退出临界区会唤醒，时间片用完只是运行变就绪。

【图像】



唤醒专指阻塞变就绪：时间片用完只是运行变就绪

【定位】 操作系统 · 第2章进程管理 · 进程状态转换（阻塞、就绪、运行）

【易混对比】

- "唤醒"和"调度/剥夺"是两回事：唤醒专指阻塞态变就绪态；调度是就绪变运行，剥夺（比如时间片用完）是运行变就绪，都不叫唤醒。
- "等 I/O"和"等临界区"是两回事：前者等的是外设把事办完，后者等的是别的进程让出"一次只让一人进"的那段代码；但两种等待都是阻塞，等的事一办完都会被唤醒。
- "阻塞态"和"就绪态"是两回事：阻塞是在等某件事、没法跑；就绪是万事俱备、只等 CPU。唤醒就是把"等事的人"放进"排队等 CPU 的人"里。

唤醒=阻塞→就绪，只有"等的事办完了"才唤醒；时间片用完是运行→就绪，不叫唤醒。

【坑位】

别被"重新获得机会"这类大白话骗了：只要没从阻塞里放出来，都不是唤醒。

最容易踩的坑是把"时间片用完"也算成唤醒——"当前进程让出 CPU，别人有机会上"听起来像唤醒，其实它只是从运行态被剥夺回就绪态，从没进过阻塞态。正确做法：拿到一个事件先问两句——"谁在等？等的那件事是不是这件？"只有等 I/O 的等到 I/O 结束、等临界区的等到别人退出临界区，这才叫唤醒。

【类比】

像医院排队。阻塞是"在检查室门口等报告"——没报告你看不了病；唤醒是"报告出来了，护士叫号让你进诊室排队"。时间片用完是"正在诊室看病的人被医生说先出去、让下一位进"，他只是被请出诊室、回大厅重新排队，根本不是"被叫号"。叫号（唤醒）只发生在等报告的人身上。

【口诀】 阻塞变就绪，才是被唤醒；运行变就绪，唤醒没它的戏。

【考频】 「进程唤醒」在 2009–2022 年解析中出现 24 次 · 高频

第 25 题 · 系统调用叙述、正确项（正确答案：C）

题干与选项：

下列关于系统调用的叙述中，正确的是（）。

Ⅰ. 在执行系统调用服务程序的过程中，CPU 处于内核态

Ⅱ. 操作系统通过提供系统调用避免用户程序直接访问外设

Ⅲ. 不同的操作系统为应用程序提供了统一的系统调用接口

Ⅳ. 系统调用是操作系统内核为应用程序提供服务的接口

- A. 仅Ⅰ、Ⅳ
- B. 仅Ⅱ、Ⅲ
- C. 仅Ⅰ、Ⅱ、Ⅳ
- D. Ⅰ、Ⅲ、Ⅳ

题意概括：关于系统调用的四句话，问哪几句对。

入手点： 先想系统调用是干什么的：应用程序想用操作系统服务（读写外设、申请内存）时走的正规入口；再看每句话是不是符合这个定位。

题干专有名词从零解释：

- 先想"系统调用"：应用程序请求操作系统服务的入口。
- 再想"内核态"：CPU 执行操作系统代码的状态，权限高；"用户态"是 CPU 执行应用程序的状态，权限低。
- 再想"外设"：键盘、磁盘、打印机这些外部设备。
- 再想"接口"：调用服务时用的函数名和参数约定。

考查知识点：

- 先想Ⅰ：在执行系统调用服务程序的过程中，CPU 处于内核态——服务程序是操作系统代码，当然在内核态跑。对的。
- 再想Ⅱ：操作系统通过提供系统调用避免应用程序直接访问外设——应用程序不能直接碰硬件，必须通过系统调用这个正规入口，操作系统在中间把关。对的。
- 再想Ⅲ：不同的操作系统为应用程序提供了统一的系统调用接口——不对。各操作系统的接口不一样（比如 Linux 和 Windows 的系统调用就不同）；POSIX 只是个标准，不是所有操作系统都统一实现。
- 再想Ⅳ：系统调用是操作系统内核为应用程序提供服务的接口——定义就是这样。对的。

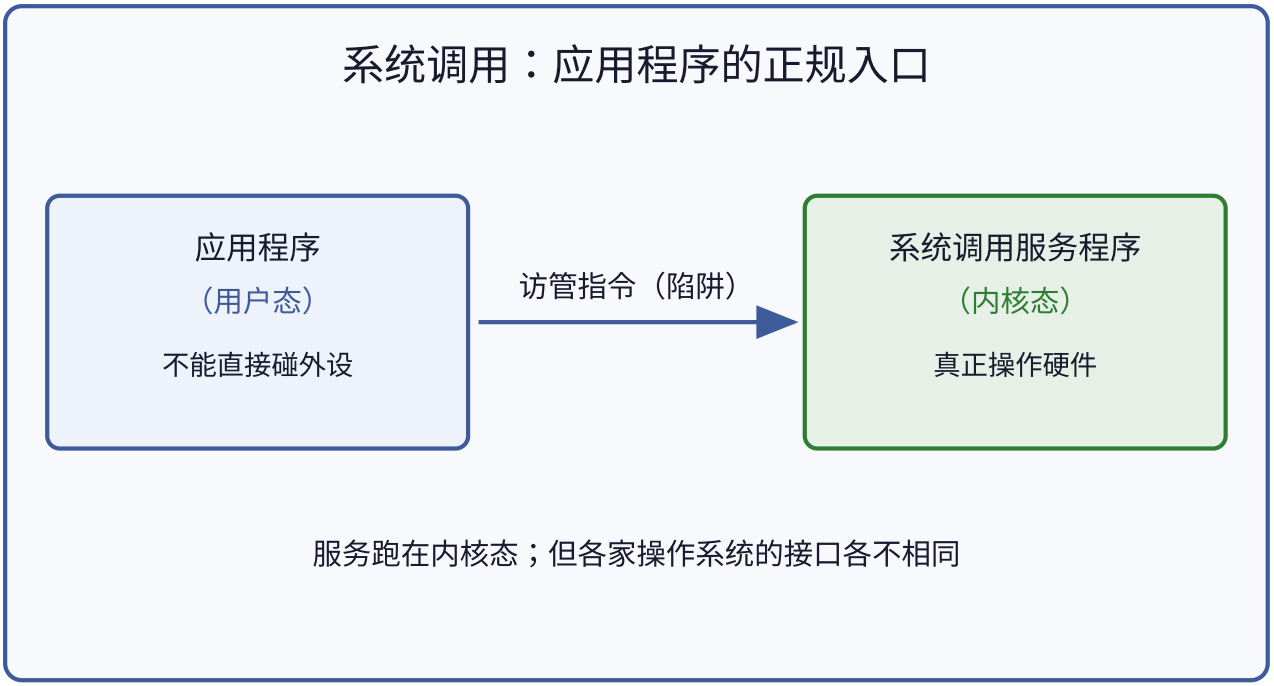
正确选项为什么对： Ⅰ、Ⅱ、Ⅳ对，Ⅲ错，选 C（仅Ⅰ、Ⅱ、Ⅳ）。

错误选项为什么错＋举例：

- A（仅Ⅰ、Ⅳ）：漏了Ⅱ——系统调用的重要作用就是隔离应用程序和外设。
- B（仅Ⅱ、Ⅲ）：Ⅲ错（接口不统一），还漏了Ⅰ和Ⅳ。
- D（仅Ⅰ、Ⅲ、Ⅳ）：Ⅲ错，见上。

笔记本一句话： 第 25 题＋系统调用：内核态跑服务、隔离应用和外设、是内核给应用的接口；但各家接口不统一。

【图像】



系统调用：应用程序经访管指令进入内核态用服务

【定位】 操作系统 · 第2章进程管理 · 系统调用（用户态与内核态）

【易混对比】

- "系统调用"和"访管指令"是两回事：访管指令是触发系统调用的那条特殊指令（推开一扇门），系统调用是门后面整套操作系统服务程序（门后的办事大厅）。
- "接口不统一"和"POSIX 标准"是两回事：POSIX 是各方商量出来的"建议稿"，不是强制统一；各家操作系统的系统调用长相真的不一样，不能混用。
- "库函数"和"系统调用"是两回事：库函数是用户程序里的一层包装纸，有的包着系统调用（比如 printf 底层调 write），有的只是纯用户态计算，根本不进内核。

系统调用是跑在内核态、隔离应用和外设的正规入口；但各家操作系统的接口各不相同。

【坑位】

"统一"二字一出现就要警惕：操作系统的系统调用接口不统一、硬件接口不统一，标准不等于强制统一。

最容易踩的坑是把"有标准"当成"已统一"——POSIX 只是建议，各家实现各行其是。另一个坑是漏掉系统调用的"隔离"作用：应用程序不能直接碰外设（键盘、磁盘、打印机），必须走系统调用这个正规入口，操作系统在中间把关。正确做法：每句话先问"这是不是在说内核态跑服务、隔离应用和外设、接口是内核给应用的"——三条里只要有一条反了（比如说接口统一），整句判错。

【类比】

像银行柜台。客户（应用程序）不能进金库（内核态）自己拿钱（直接碰外设），只能通过柜台窗口（系统调用）填单子办业务，柜员（操作系统）在窗口后面真正操作金库。但每家银行的窗口表格长得不一样（各家接口不统一），你在 A 银行填的单子拿到 B 银行用不了——这就是"接口不统一"的日常版。

【口诀】 应用想用系统货，系统调用开窗口；服务内核态里跑，各家接口不统一。

【考频】 「系统调用」在 2009–2022 年解析中出现 18 次 · 高频

第 26 题 · 文件系统管理空闲磁盘块的数据结构（正确答案：B）

题干与选项：

下列选项中，可用于文件系统管理空闲磁盘块的数据结构是（）。

Ⅰ. 位图 Ⅱ. 索引结点 Ⅲ. 空闲磁盘块链 Ⅳ. 文件分配表（FAT）

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅰ、Ⅲ、Ⅳ
- C. 仅 Ⅰ、Ⅲ
- D. 仅 Ⅱ、Ⅲ、Ⅳ

题意概括：四种数据结构，问哪些可以用来管理文件系统的空闲磁盘块。

入手点： 先想"管理空闲块"要干什么：记下哪些块空着、哪些占着，分配时找空块、回收时还空块；再看每种结构是不是干这个活的。

题干专有名词从零解释：

- 先想"位图"：一个位表示一个块，1 表示占用、0 表示空闲，或者反过来。
- 再想"索引结点"：记录文件信息（大小、位置、权限）的一张表，是文件的"身份证"。
- 再想"空闲磁盘块链"：把空块串成一条链。
- 再想"文件分配表（FAT）"：一张表记录每个块的下一块是谁、空闲与否。

考查知识点：

- 先想 Ⅰ 位图：一个位管一个块，找空块就是找 0，回收就是把位清零——可以。
- 再想 Ⅱ 索引结点：记录的是某个文件的信息，不是空闲块的信息——不可以。
- 再想Ⅲ空闲磁盘块链：把空块串起来，分配时从链头取、回收时挂回链上——可以。
- 再想Ⅳ文件分配表：表里每个表项对应一个块，空闲块的表项有专门标记，顺着表就能找到空块——可以。

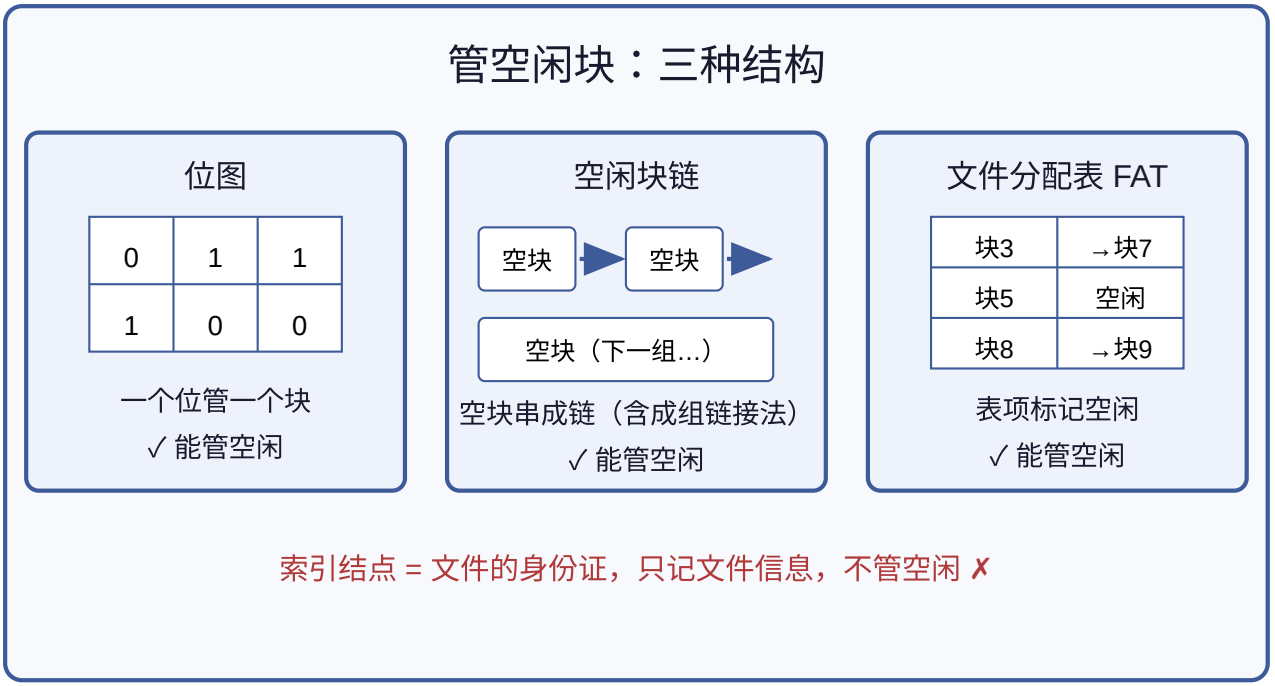
正确选项为什么对： Ⅰ、Ⅲ、Ⅳ可以，Ⅱ不可以，选 B（仅 Ⅰ、Ⅲ、Ⅳ）。

错误选项为什么错＋举例：

- A（仅 Ⅰ、Ⅱ）：Ⅱ是文件的信息表，不管空闲块；还漏了Ⅲ和Ⅳ。
- C（仅 Ⅰ、Ⅲ）：漏了Ⅳ——FAT 表项也能标记空闲。
- D（仅 Ⅱ、Ⅲ、Ⅳ）：Ⅱ不对，见上。

笔记本一句话： 第 26 题＋管空闲块：位图、空闲块链、FAT 都行；索引结点是文件的身份证，不管空闲。

【图像】



管理空闲盘块的三种结构：位图、空闲块链、FAT

【定位】 操作系统 · 第4章文件管理 · 空闲磁盘空间的管理

【易混对比】

- "管空闲块"和"管文件"是两回事：前者记"哪些块没人用"，后者记"某个文件占了哪些块"；索引结点是文件的"身份证"，不管空闲。
- "位图"和"文件分配表（FAT）"是两回事：位图是一张单独的图、一个位管一个块；FAT 是块号对块号的表，用表项里的特殊标记表示"这块空闲"。
- "成组链接法"和"普通空闲块链"是两回事：成组链接法是链的升级版——把空闲块按固定组数先堆成小堆栈、再把堆栈串起来，而不是一块一块简单串。

记空闲的才算数：位图、空闲块链（含成组链接法）、FAT 都能管空闲块；索引结点只给文件办身份证。

【坑位】

看到"表"就以为都管空闲——先问"这张表记的是文件还是记的是空闲"，记文件的信息表再精美也不管空闲。

最容易踩的坑是被"索引结点"四个字带偏：它听起来像管磁盘的"索引"，其实它是某个文件的专属档案（大小、权限、占了哪些块），回答的是"这个文件在哪"，不是"哪些块空着"。正确做法：抓一句判据——"它记的是空闲块的状态吗？"位图记的是、空闲块链记的是、FAT 的空闲标记记的是，索引结点记的不是。

【类比】

像停车场管理。管"哪些车位空着"有三种办法：墙上挂一张大格子图、一个格子管一个车位（位图）；把空车位的钥匙串成一串（空闲块链）；大本子每页记一个车位的状态（FAT）。而"车辆登记证"（索引结点）只记一辆车的信息，不记车位——查登记证永远查不到哪个车位空着。

【口诀】 位图空闲链，FAT 一起算；索引管文件，空闲它不管。

【考频】 「空闲块」在 2009–2022 年解析中出现 6 次 · 中频

第 27 题 · 二级反馈队列调度、P1／P2 平均等待时间（正确答案：C）

题干与选项：

系统采用二级反馈队列调度算法进行进程调度。就绪队列 Q1 采用时间片轮转调度算法，时间片为 10ms；就绪队列 Q2 采用短进程优先调度算法；系统优先调度 Q1 队列中的进程，当 Q1 为空时系统才会调度 Q2 中的进程；新创建的进程首先进入 Q1；Q1 中的进程执行一个时间片后，若未结束，则转入 Q2。若当前 Q1、Q2 为空，系统依次创建进程 P1、P2 后即开始进程调度，P1、P2 需要的 CPU 时间分别为 30ms 和 20ms，则进程 P1、P2 在系统中的平均等待时间为（）。

- A. 25ms
- B. 20ms
- C. 15ms
- D. 10ms

题意概括：二级反馈队列调度，Q1 时间片轮转 10ms，Q2 短进程优先；P1 要 30ms、P2 要 20ms，先后创建，问两个进程的平均等待时间。

入手点： 画时间线：新进程先进 Q1，跑一个时间片没完就掉到 Q2；Q1 有人就先跑 Q1，Q1 空了才跑 Q2；Q2 里按"剩的时间短的先跑"。

题干专有名词从零解释：

- 先想"二级反馈队列"：两个就绪队列排队，Q1 优先于 Q2。
- 再想"时间片轮转"：每人轮流跑一个时间片。
- 再想"短进程优先"：剩的时间短的先跑。
- 再想"等待时间"：从创建到完成，减去自己真正跑 CPU 的时间——也就是排队等的时间。
- 再想"就绪队列"：等着上 CPU 的进程排的队。

考查知识点：

- 先画时间线：0 到 10ms，P1 在 Q1 跑 10ms（剩 20ms），掉到 Q2。10 到 20ms，P2 在 Q1 跑 10ms（剩 10ms），掉到 Q2。20ms 时 Q1 空了，看 Q2：P2 剩 10ms、P1 剩 20ms，短进程优先先跑 P2，20 到 30ms P2 跑完（P2 完成时刻 30ms）。30 到 50ms，P1 跑完剩下的 20ms（P1 完成时刻 50ms）。
- 再算等待时间：等待等于完成时刻减去服务时间。P1 从 0 到 50，跑了 30，等了 20；P2 从 0 到 30，跑了 20，等了 10。平均 (20+10)÷2=15ms。
- 注意：等待时间不是"完成时刻"本身，要减掉自己占 CPU 的时间；P1 等了 20 不是 50，P2 等了 10 不是 30。

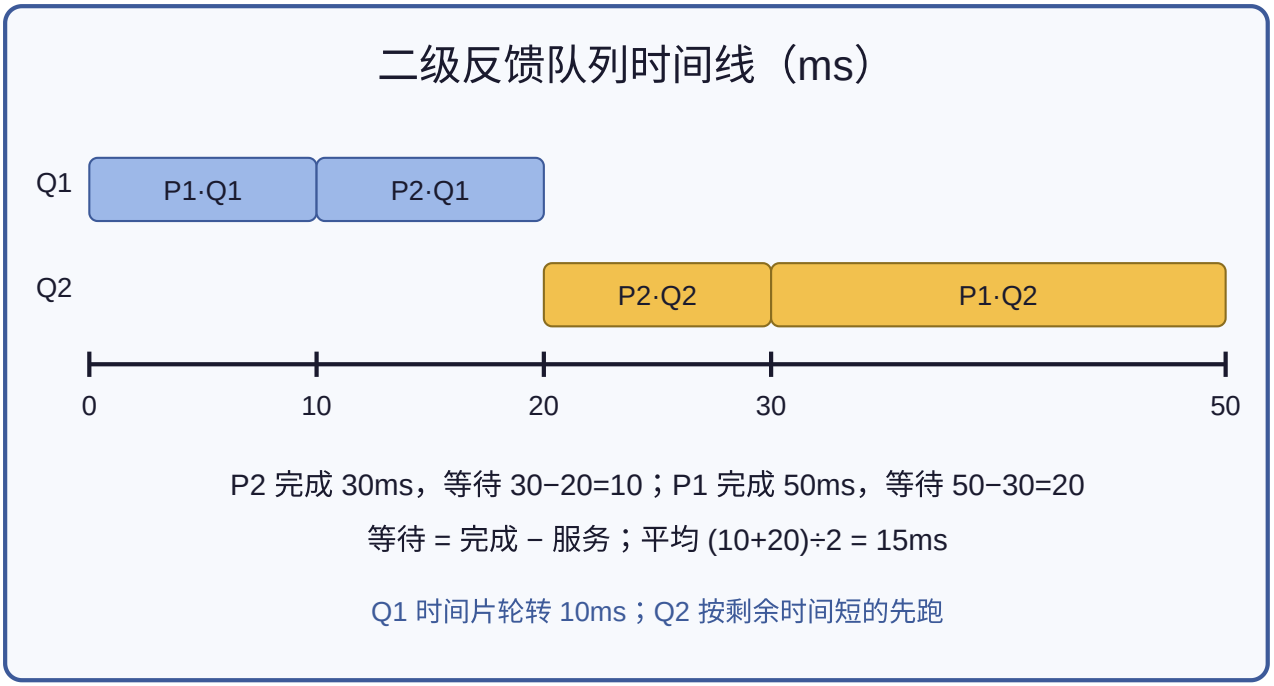
正确选项为什么对： (20+10)÷2=15ms，选 C。

错误选项为什么错＋举例：

- A（25ms）：P2 的等待算成了完成时刻 30（忘了减它自己跑的 20ms），(20+30)÷2=25。
- B（20ms）：把 P1 的等待 20 直接当成了平均，或者 P2 的等待也算成了 20——平均要把两个进程的等待加起来除以 2。
- D（10ms）：P1 的等待只算了 10 到 20 那 10ms，漏了 20 到 30 P2 在 Q2 跑的那 10ms，平均后正好是 10。

笔记本一句话： 第 27 题＋二级反馈：画时间线，P1 完成 50、P2 完成 30，等待是完成减服务，(20+10)÷2=15ms。

【图像】



二级反馈队列调度时间线：P1、P2 的完成时刻与等待时间

【定位】 操作系统 · 第2章进程管理 · 处理机调度（多级反馈队列调度算法）

【易混对比】

- "等待时间"和"完成时刻"是两回事：等待时间=完成时刻−自己真正跑 CPU 的时间；直接拿完成时刻当等待时间，会把"自己跑的那部分"也算成等，算大了。
- "新进程进哪"和"时间片用完去哪"是两回事：新来的永远先进 Q1；只有在 Q1 里一个时间片没跑完，才"降级"进 Q2，不是从 Q2 起步。
- "Q1 时间片轮转"和"Q2 短进程优先"是两回事：Q1 是轮流雨露均沾；Q2 是"剩的活儿少的先干完走人"——Q2 排次序看的是剩余服务时间，不是到达先后。

画时间线、按"等待=完成−服务"算：P1 等 20ms、P2 等 10ms，平均 15ms。

【坑位】

算出来的数有 15、20、25、10 好几种：差就差在"等待有没有减掉自己占 CPU 的时间"和"Q2 里谁先跑"。

最容易踩的坑有两个：一是把完成时刻直接当等待时间（P2 完成时刻 30ms，但它自己跑了 20ms，等待只有 10ms）；二是 Q1 空了之后不知道 Q2 按"剩余时间短"排，把 P1 先跑了。正确做法是三步：画时间线定每个进程的完成时刻，再用"完成减服务"得到等待，最后求平均。本题：0–10ms 跑 P1，10–20ms 跑 P2，20ms 时 Q1 空，Q2 里 P2 剩 10ms、P1 剩 20ms，短的先跑——P2 在 30ms 完成，P1 在 50ms 完成。

【套路】

- 第一步，画时间线：新进程先进 Q1 按时间片轮转跑；Q1 排空了才轮到 Q2。
- 第二步，排 Q2 次序：掉进 Q2 的进程按剩余服务时间从小到大跑。
- 第三步，算等待再平均：每个进程等待=完成时刻−服务时间，加起来除以进程数。

时间线画出来，完成减服务，平均一求——多级反馈就这三下。

【类比】

像银行双窗口。一楼窗口（Q1）只办 10 分钟速办业务，办不完的转二楼（Q2），二楼按"手头活儿少的先办"排队。你在银行耗的总时长减去你真正被办理的时长，才是你排队等的时间——自己占着窗口的那几分钟不算等。

【口诀】 新来先进一队列，一轮没完降二级；二级短活先干完，等待等于完成减服务。

【考频】 「多级反馈队列」在 2009–2022 年解析中出现 3 次 · 中频

第 28 题 · 分段共享段表叙述、错误项（正确答案：B）

题干与选项：

在分段存储管理系统中，用共享段表描述所有被共享的段。若进程 P1 和 P2 共享段 S，下列叙述中，错误的是（ ）。

- A. 在物理内存中仅保存一份段 S 的内容
- B. 段 S 在 P1 和 P2 中应该具有相同的段号
- C. P1 和 P2 共享段 S 在共享段表中的段表项
- D. P1 和 P2 都不再使用段 S 时才回收段 S 所占的内存空间

题意概括：分段存储里用共享段表描述共享段，P1 和 P2 共享段 S，问哪句话是错的。

入手点： 本题问"错误的是"，先想共享段是怎么共享的：内存里一份，两个进程各有一张段表指过去；段号是各进程自己的编号，不用一样。

题干专有名词从零解释：

- 先想"分段存储管理"：按逻辑段分内存，一段是一段。
- 再想"共享段"：两个进程共用的一段内存。
- 再想"共享段表"：描述所有被共享段的一张总表。
- 再想"段号"：进程自己的段表里给段编的号。
- 再想"段表项"：段表里一行，记录段的地址、长度、权限。
- 再想"回收"：没人用了才把内存还回去。

考查知识点：

- 先想 A：物理内存里只保存一份段 S 的内容——共享的意思就是一份内存、两处引用。对的。
- 再想 B：段 S 在 P1 和 P2 中应该具有相同的段号——不对。段号是每个进程自己的段表里的编号，P1 的段表和 P2 的段表是分开的，同一个共享段在两个进程里可以编不同的号。
- 再想 C：P1 和 P2 共享段 S 在共享段表中的段表项——共享段表就是干这个的，两个进程指同一个共享段表项。对的。
- 再想 D：P1 和 P2 都不再使用段 S 时才回收——还有人用就不能收，引用计数到 0 才回收。对的。

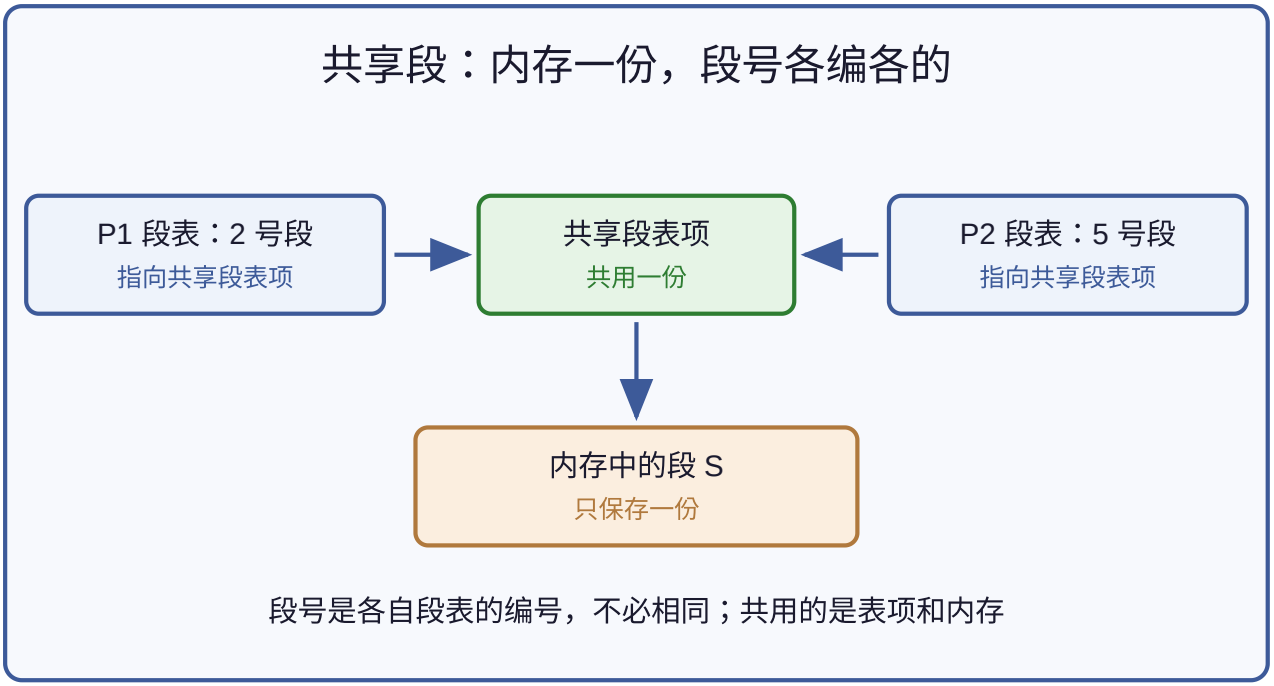
正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。B 说段号应该相同，不对。段号是进程内的局部编号，各编各的；共享的是内存内容和共享段表项，不是段号。选 B。

错误选项为什么错＋举例：

- A：共享就是内存里一份——对的说法。
- C：共用共享段表项——对的说法。
- D：没人用了才回收——对的说法；还有人用就回收，另一个进程就访问不到了。

笔记本一句话： 第 28 题＋共享段：内存一份、段表项共用，但段号各编各的，不用相同。

【图像】



共享段示意：内存一份、段号各编各的、共享段表项共用

【定位】 操作系统 · 第3章内存管理 · 分段存储管理（段的共享与保护）

【易混对比】

- "段号"和"段在内存的地址"是两回事：段号是进程自己段表里的编号（门牌是自己编的），内存地址是真实的物理位置；共享段可以"一个物理位置、两边门牌号不同"。
- "共享段表项"和"进程自己的段表项"是两回事：共享段表项是大家共用的那份描述（地址、长度、权限）；进程段表项里只放一个指向共享段表项的指针。
- "共享"和"复制"是两回事：共享是内存里一份、两处指过去；复制是内存里两份，各用各的。

共享=内存里一份、共享段表项共用；段号是各进程的局部编号，不必相同。

【坑位】

一说"共享"就联想"连编号都一样"——段号只是进程内部的索引，共享的是内容和描述项，不是编号。

最容易踩的坑是把"共享段"脑补成"两个进程的段表长得一模一样"。P1 和 P2 的段表是各管各的：P1 把共享段编成 2 号，P2 把它编成 5 号，完全没问题——只要两个段表项都指向同一个共享段表项、最终落到内存里同一份内容上，就是共享。正确做法：凡是说"段号必须相同"的，判错；凡是说"内存里只一份、表项共用、没人用才回收"的，判对。

【类比】

像合租。一间房（内存里一份）住两个人（P1、P2），两个人的通讯录里给这间房起的备注可以不同（段号不同：一个存"2号房"，一个存"5号房"）。房间的钥匙信息登记在同一本物业台账上（共享段表项共用），两个人搬走、房间空了，物业才收回房子（没人用才回收）。

【口诀】 共享内存只一份，段号各编各的门；表项共用一份存，计数归零才回收。

【考频】 「分段」在 2009–2022 年解析中出现 7 次 · 中频

第 29 题 · LRU 局部置换、页置换总次数（正确答案：C）

题干与选项：

某系统采用 LRU 页置换算法和局部置换策略，若系统为进程 P 预分配了 4 个页框，进程 P 访问页号的序列为 0, 1, 2, 7, 0, 5, 3, 5, 0, 2, 7, 6，则进程访问上述页的过程中，产生页置换的总次数是（ ）。

- A. 3
- B. 4
- C. 5
- D. 6

题意概括：LRU 页置换、局部置换，进程分到 4 个页框，给出一串访问页号，问访问过程中产生页置换的总次数。

入手点： 先分清"页置换"和"缺页"：缺页是"要的页不在"，页置换是"缺页时把旧页换出去"；一开始往空框里装不叫置换，只有框满了、换出旧页才叫置换。

题干专有名词从零解释：

- 先想"LRU（最近最久未使用）"：换出最久没用过的那页。
- 再想"局部置换"：只在本进程分到的页框里换，不动别人的。
- 再想"页框"：内存里放一页的格子，这里是 4 个。
- 再想"页置换"：缺页时把一页换出去、把要的页换进来。
- 再想"缺页"：要访问的页不在内存里。

考查知识点：

- 一步一步走，框里最多 4 页：0 缺，装进空框（不叫置换）；1 缺，装进空框；2 缺，装进空框；7 缺，装进空框——4 个框满了。
- 0 命中。5 缺，框满了，换出最久没用的 1（1 是最早装的、中间没再用过），装 5——第 1 次置换。
- 3 缺，换出最久没用的 2——第 2 次。5 命中。0 命中。
- 2 缺，换出最久没用的 7——第 3 次。7 缺，换出最久没用的 3——第 4 次。6 缺，换出最久没用的 5——第 5 次。一共 5 次。
- 注意：缺页一共 9 次（4 次装空框加 5 次置换），但题目问的是置换次数，不是缺页次数；一开始往空框里装不算置换。

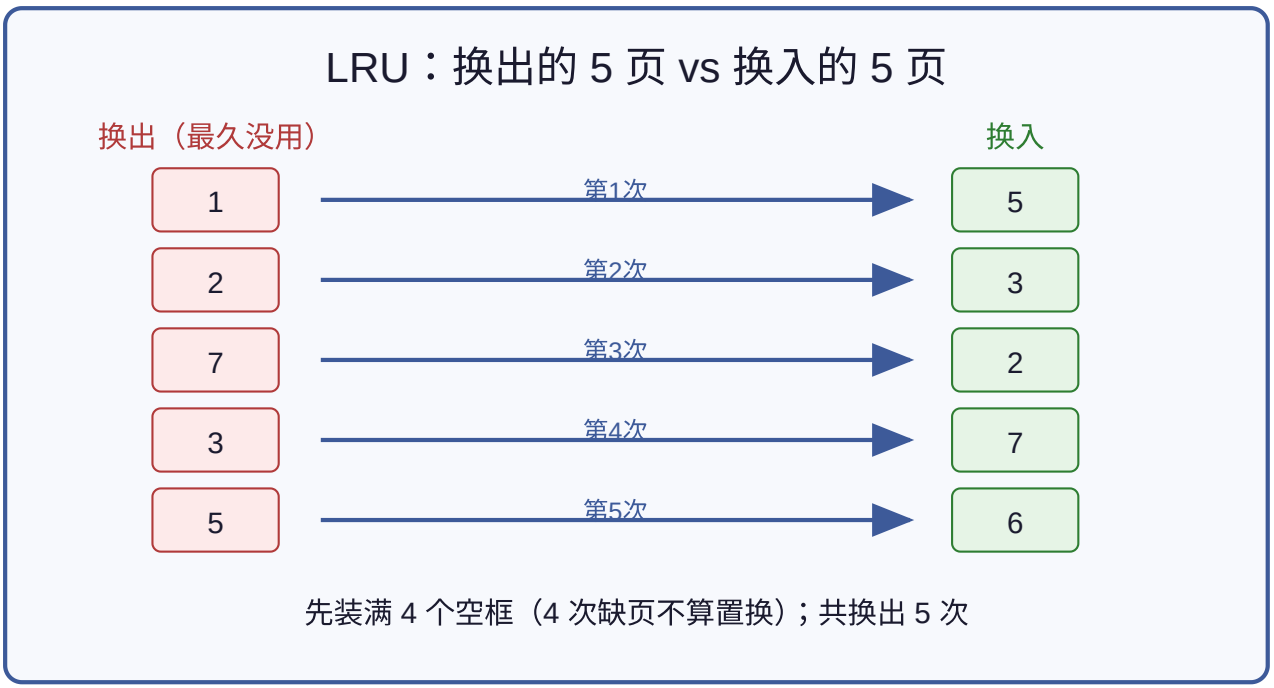
正确选项为什么对： 换出旧页一共 5 次，选 C。

错误选项为什么错＋举例：

- A（3）：换出次数数少了（比如中间某两次换出漏数了）。
- B（4）：漏了一次——最可能是最后 6 那次没数，或者中间某次换出漏了。
- D（6）：把第一次往空框里装也算成了一次置换，多了 1 次。

笔记本一句话： 第 29 题＋LRU 数置换：空框装不算置换，框满换出旧页才算，本题一共 5 次。

【图像】



LRU 置换过程：换出的五页与换入的五页

【定位】 操作系统 · 第3章内存管理 · 虚拟内存（页面置换算法 LRU）

【易混对比】

- "缺页"和"置换"是两回事：缺页是"要的页不在内存"；置换是"缺页时还得把一页旧的赶出去"。框没满时缺页只是往空框里装，不算置换。
- "局部置换"和"全局置换"是两回事：局部只动本进程分到的框（本题锁死 4 个）；全局可以在整个内存里挑。本题跟别人没关系。
- "LRU"和"OPT（最佳置换）"是两回事：LRU 看"过去谁最久没用过"，OPT 看"未来谁最晚再用"；本题只往过去看。

只数"框满后换出旧页"的次数：先装满 4 个空框不算，本题换出 5 次。

【坑位】

题目问的是置换次数，不是缺页次数——把往空框里装也算进去，就会多出 1 次。

最容易踩的坑是数缺页：全程缺页 9 次（4 次装空框＋5 次置换），但题目问的是置换。正确做法：前 4 次缺页只管装、不计数；框满之后，每次缺页必然换出一页旧的，换出一次记 1。本题换出的是 1、2、7、3、5，一共 5 次。找"最久没用"时从当前往前推：最后一次被访问的时间最早的那页出局。

【套路】

- 第一步，先装满：前 4 次缺页往空框里装，计数器不动。
- 第二步，找最久没用：框满后每次缺页，换出"从现在往前数、最后一次被用时间最早"的那页。
- 第三步，只数换出：换出一次记 1，本题一共 5 次。

空框装不算数，满了才换出；LRU 看过去，只数换出次数。

【类比】

像只有 4 个挂钩的衣帽架。新衣服来了先挂空钩（不算换衣服）；钩满了再进新衣服，就得摘掉"最久没穿"的那件。本题一共摘了 5 次旧衣服——摘衣服的次数才是置换次数，往空钩上挂不算。

【口诀】 框满才置换，装空不算数；最久没用的先出去，换出五次记清楚。

【考频】 「LRU」在 2009–2022 年解析中出现 11 次 · 高频

第 30 题 · 死锁叙述、正确项（正确答案：B）

题干与选项：

下列关于死锁的叙述中，正确的是（）。

Ⅰ. 可以通过剥夺进程资源解除死锁

Ⅱ. 死锁的预防方法能确保系统不发生死锁

Ⅲ. 银行家算法可以判断系统是否处于死锁状态

Ⅳ. 当系统出现死锁时，必然有两个或两个以上的进程处于阻塞态

- A. 仅Ⅱ、Ⅲ
- B. 仅Ⅰ、Ⅱ、Ⅳ
- C. 仅Ⅰ、Ⅱ、Ⅲ
- D. 仅Ⅰ、Ⅲ、Ⅳ

题意概括：关于死锁的四句话，问哪几句对。

入手点： 先想死锁的四个必要条件和对付办法：预防、避免、检测、解除——每句话对号入座。

题干专有名词从零解释：

- 先想"死锁"：一组进程互相等对方手里的资源，谁也走不动。
- 再想"剥夺"：把资源强行收回来。
- 再想"预防"：破坏四个必要条件之一，让死锁根本发生不了。
- 再想"银行家算法"：分配前先算"这样分会不会进不安全状态"，不安全就不分。
- 再想"阻塞态"：进程在等资源、等事件，动不了的状态。

考查知识点：

- 先想Ⅰ：可以通过剥夺进程资源解除死锁——把某个进程的资源收回来给别人，打破循环等待。对的。
- 再想Ⅱ：死锁的预防方法能确保系统不发生死锁——预防就是破坏必要条件，条件不齐，死锁就形不成。对的。
- 再想Ⅲ：银行家算法可以判断系统是否处于死锁状态——不对。银行家算法是"避免"策略：分配前判断安不安全；判断"已经死锁了没有"靠的是死锁检测算法（画资源分配图、找环），不是银行家。
- 再想Ⅳ：当系统出现死锁时，必有两个或两个以上的进程处于阻塞态——一个进程自己跟自己构不成循环等待，死锁至少是两个进程互相等。对的。

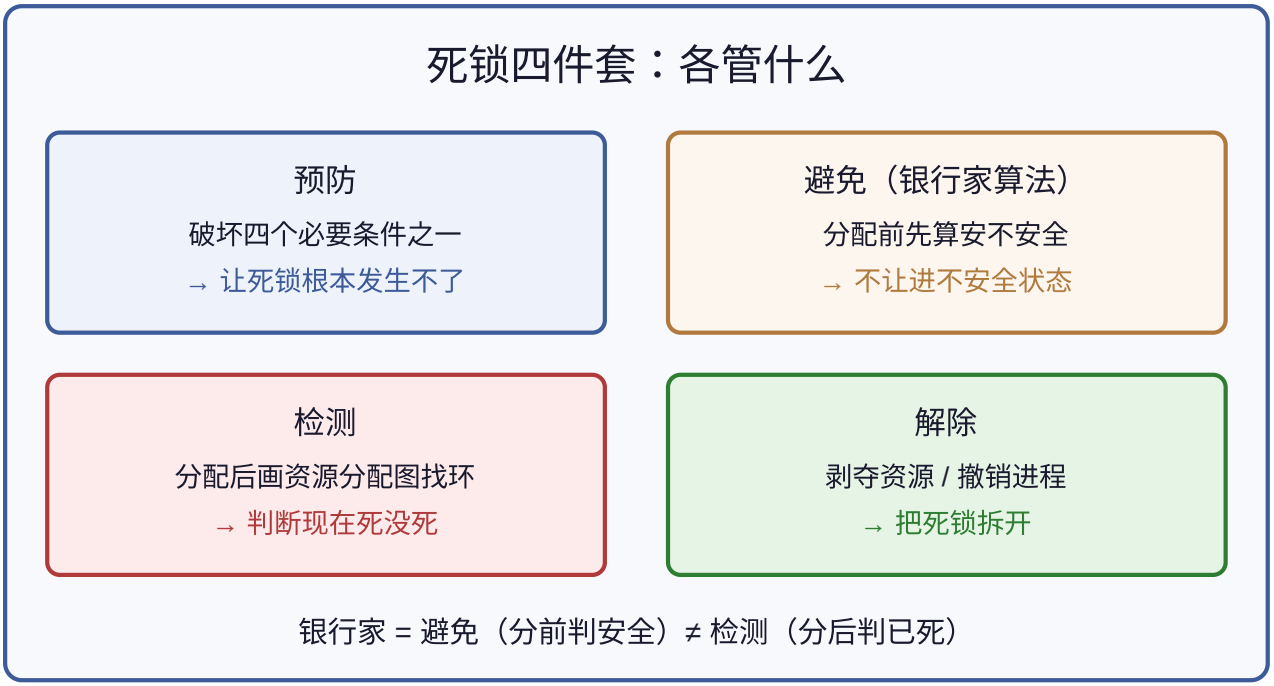
正确选项为什么对： Ⅰ、Ⅱ、Ⅳ对，Ⅲ错，选 B（仅Ⅰ、Ⅱ、Ⅳ）。

错误选项为什么错＋举例：

- A（仅Ⅱ、Ⅲ）：Ⅲ错，见上；还漏了Ⅰ和Ⅳ。
- C（仅Ⅰ、Ⅱ、Ⅲ）：Ⅲ错，见上。
- D（仅Ⅰ、Ⅲ、Ⅳ）：Ⅲ错，见上。

笔记本一句话： 第 30 题＋死锁：剥夺能解除、预防能杜绝、死锁至少两人阻塞；银行家是避免（判安全），不是检测（判已死锁）。

【图像】



死锁四件套：预防、避免、检测、解除各管什么

【定位】 操作系统 · 第2章进程管理 · 死锁（预防、避免、检测与解除）

【易混对比】

- "银行家算法"和"死锁检测"是两回事：银行家是"分配前先算安不安全"的避免策略；死锁检测是"分配完看死没死"（画资源分配图找环）。
- "预防"和"避免"是两回事：预防是破坏四个必要条件之一、让死锁连发生的土壤都没有；避免是允许条件存在、但动态算安全状态，不让系统进不安全状态。
- "解除"和"恢复"是两回事：解除是把死锁拆开（剥夺某进程资源、或撤销进程）；恢复是拆开后让系统回到正常状态接着跑。

剥夺能拆死锁、预防能杜绝死锁、死锁至少两个进程互相阻塞；银行家只判"分了会不会不安全"，不判"现在死没死"。

【坑位】

"银行家"和"死锁"老在一起出现，最容易张冠李戴——看见银行家先默念"避免不是检测"。

最容易踩的坑是把银行家算法当成"判断系统是否已死锁"的工具。银行家永远站在"分配之前"：每次分资源前算一遍"这样分下去系统还安不安全"，不安全就不分——它是防火的，不是救火的。判断"已经死没死"靠的是死锁检测算法（资源分配图里找环）。正确做法：看到"银行家"先定位到"避免→分之前→判安全"，再跟题干对。

【类比】

像十字路口四个方向的车互相堵死。预防是装红绿灯（让"互相等待"的条件不存在）；银行家是路口交警放行前算"放这辆进去会不会堵死"（分之前判安全）；检测是堵死之后航拍看有没有形成环路（分之后判已死）；解除是叫拖车拖走一辆腾出路（剥夺资源拆死锁）。

【口诀】 预防破条件，避免算安全；银行家管分之前，检测画图找死环。

【考频】 「死锁」在 2009–2022 年解析中出现 13 次 · 高频

第 31 题 · 二级分页地址结构、页目录号／页号（正确答案：A）

题干与选项：

某计算机主存按字节编址，采用二级分页存储管理，地址结构如下所示：

页目录号（10 位） 页号（10 位） 页内偏移（12 位）

虚拟地址 20501225H 对应的页目录号、页号分别是（）。

- A. 081H、101H
- B. 081H、401H
- C. 201H、101H
- D. 201H、401H

题意概括：二级分页，地址结构是页目录号 10 位、页号 10 位、页内偏移 12 位，给一个虚拟地址，问页目录号和页号各是多少。

入手点： 把虚拟地址写成 32 位二进制，从左往右切三段：前 10 位是页目录号，中间 10 位是页号，后 12 位是页内偏移。

题干专有名词从零解释：

- 先想"虚拟地址"：程序看到的地址，不是内存真实地址。
- 再想"二级分页"：地址分两级查，先查页目录、再查页表。
- 再想"页目录号"：在页目录里的位置；"页号"是在页表里的位置；"页内偏移"是页里面第几个字节。
- 再想"按字节编址"：每个字节一个地址。

考查知识点：

- 先想虚拟地址 20501225H 写成二进制：0010 0000 0101 0000 0001 0010 0010 0101。
- 再想切三段：前 10 位是 0010000001，等于 0x81，也就是 081H——这是页目录号。中间 10 位是 0100000001，等于 0x101，也就是 101H——这是页号。后 12 位是 001000100101，也就是 0x225——这是页内偏移（本题不问，但切出来可以验算：三段拼回去正好是原地址）。
- 再想小例子：地址 00000000H 切出来三段全是 0；地址 FFFFFFFFH 切出来三段全是 1。
- 教训：切的时候从左往右数位，别从右往左数错了；10 位、10 位、12 位加起来是 32 位。

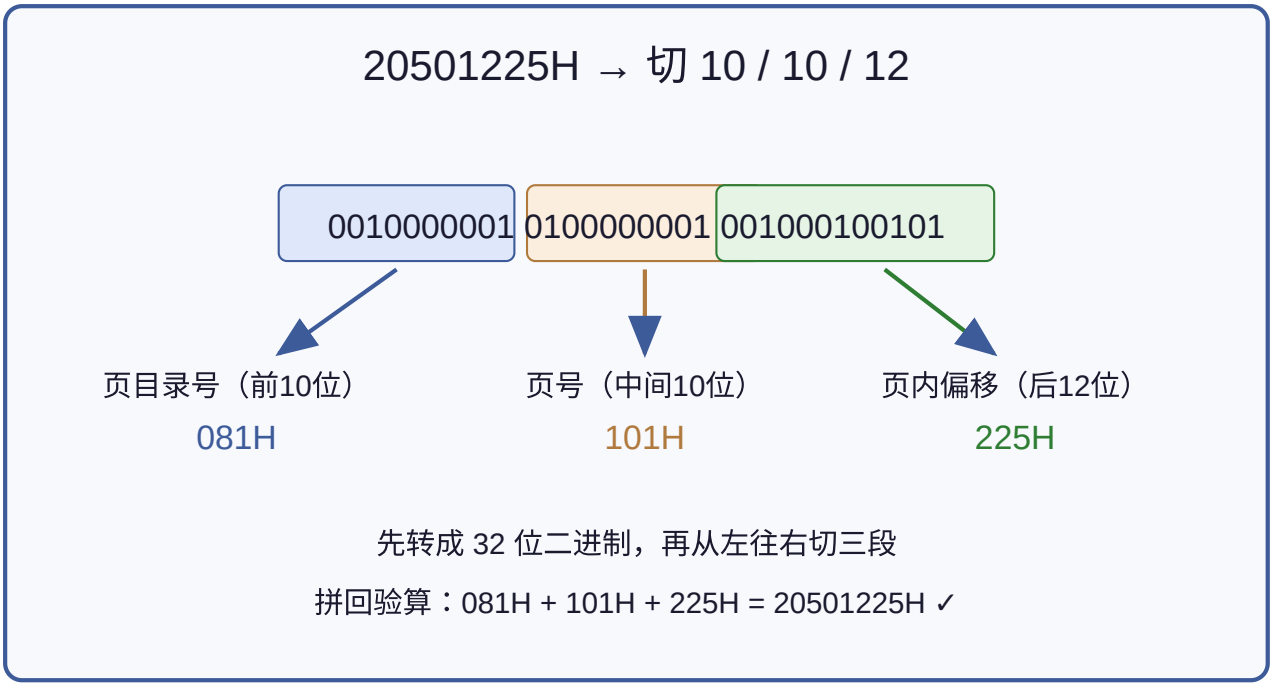
正确选项为什么对： 前 10 位 081H、中间 10 位 101H，选 A。

错误选项为什么错＋举例：

- B（081H、401H）：页目录号对了，页号错了——中间 10 位数错了（比如从第 11 位起数歪了一位）。
- C（201H、101H）：页号对了，页目录号错了——前 10 位数错了。
- D（201H、401H）：两段都数错了。

笔记本一句话： 第 31 题＋二级分页切地址：前 10 位页目录号、中间 10 位页号、后 12 位页内偏移，20501225H 切出来是 081H、101H。

【图像】



20501225H 切三段：页目录号 081H、页号 101H、页内偏移 225H

【定位】 操作系统 · 第3章内存管理 · 分页存储管理（两级页表的地址转换）

【易混对比】

- "页目录号"和"页号"是两回事：页目录号是"去目录里找哪张页表"，页号是"去页表里找哪一页"；从左往右前 10 位是目录号、中间 10 位是页号，别数反。
- "页内偏移"和"页号"是两回事：最后 12 位是页里面第几个字节，不参与查表，直接拼到物理地址后面。
- "十六进制地址"和"二进制切分"是两回事：算的时候先把地址写成完整的 32 位二进制再切，别直接在十六进制数字上按位切。

转成 32 位二进制，从左往右切 10/10/12：页目录号 081H，页号 101H。

【坑位】

数位必须从左往右数、10 位一段：数错一位，两个答案全错。

最容易踩的坑是从右往左数、或者一段数成 9 位 11 位。本题地址 20501225H 写成二进制是 0010 0000 0101 0000 0001 0010 0010 0101：前 10 位 0010000001=081H 是页目录号，中间 10 位 0100000001=101H 是页号，后 12 位 001000100101=225H 是页内偏移。正确做法：切完把三段拼回去——081H＋101H＋225H 拼回 20501225H，对上了才算数。

【套路】

- 第一步，转二进制：把地址写成完整的 32 位二进制，不够 32 位左补 0。
- 第二步，从左切三段：前 10 位页目录号、中间 10 位页号、后 12 位页内偏移。
- 第三步，转回十六进制验算：每段转回十六进制，拼回去等于原地址就对了。

转二进制、左起切 10/10/12、拼回验算。

【类比】

像查快递。快递单号分成三段："省分拨中心编号（页目录号）→市网点编号（页号）→包裹序号（页内偏移）"。先拿省编号去总目录查到市网点，再拿市编号去网点的表里找到包裹，最后按序号取件——两级页表就是"查目录再查表"的两级快递单。

【口诀】 二级分页三段切，十十十二从左切；目录找表表找页，拼回原址才算对。

【考频】 「分页」在 2009–2022 年解析中出现 20 次 · 高频

第 32 题 · 动态分区分配最易产生碎片的算法（正确答案：C）

题干与选项：

在下列动态分区分配算法中，最容易产生内存碎片的是（）。

- A. 首次适应算法
- B. 最坏适应算法
- C. 最佳适应算法
- D. 循环首次适应算法

题意概括：四种动态分区分配算法，问哪种最容易产生内存碎片。

入手点： 先想碎片是怎么来的：大块被切走一小块，剩下的小边角以后谁也用不上；再想哪种算法最爱切出小边角。

题干专有名词从零解释：

- 先想"动态分区"：内存按进程需要现切现分。
- 再想"碎片"：小到没法用的空闲块。
- 再想"首次适应"：从头找第一个够用的块。
- 再想"最佳适应"：找最小的够用块。
- 再想"最坏适应"：找最大的块。
- 再想"循环首次适应"：从上次停的地方接着找。

考查知识点：

- 先想最佳适应：每次都挑最小的够用块，切完剩下的边角最小——小边角最容易变成谁也用不上的碎片，所以最容易产生碎片。
- 再想首次适应：从头找，常常切大块的前半部分，剩下的还比较大，还能用。
- 再想最坏适应：专挑最大的块切，剩下的还是大块，最不容易碎。
- 再想循环首次适应：轮着找，空闲块用得比较均匀，也不容易碎。

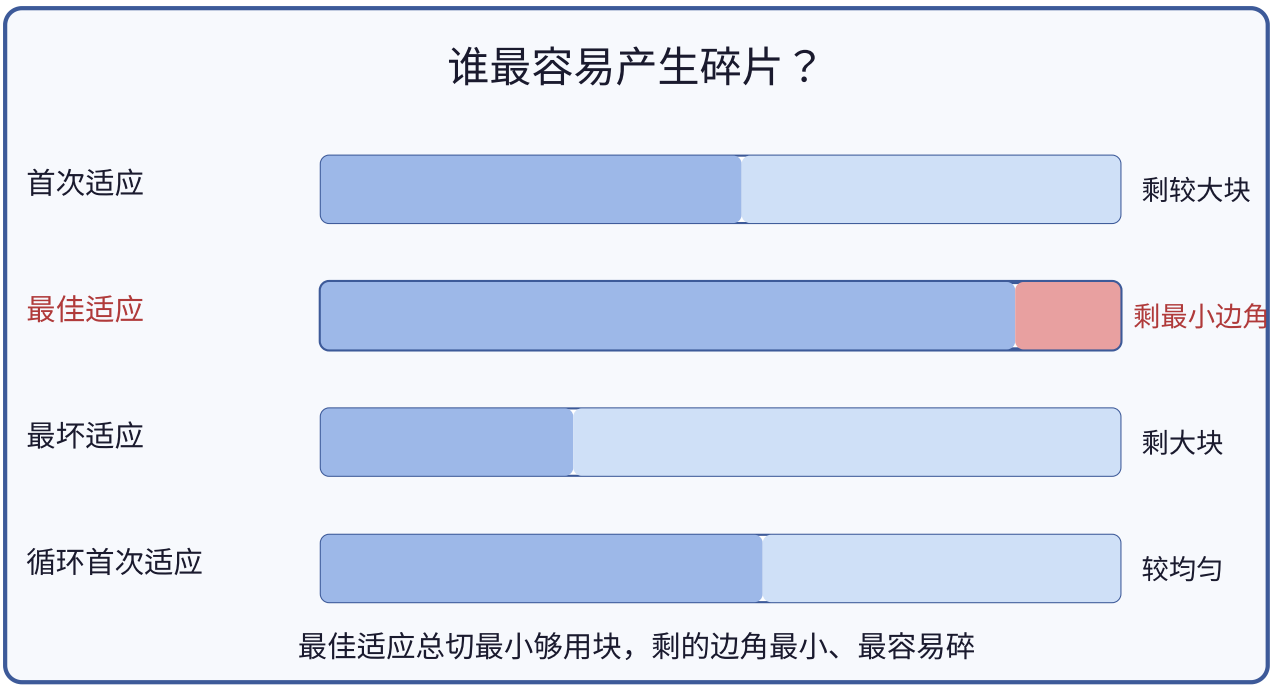
正确选项为什么对： 最佳适应总切最小够用块，剩的边角最小、最容易碎，选 C。

错误选项为什么错＋举例：

- A（首次适应）：从头找，切完剩下的块相对较大，还能用，不是最易碎的。
- B（最坏适应）：专挑最大的切，剩下的还是大块，是最不容易碎的，和题目反了。
- D（循环首次适应）：轮着找，用得均匀，也不容易碎。

笔记本一句话： 第 32 题＋分区碎片：最佳适应总切最小够用块，剩的边角最小、最容易碎。

【图像】



四种分区算法切块对比：最佳适应剩的边角最小

【定位】 操作系统 · 第3章内存管理 · 内存分配（动态分区分配算法）

【易混对比】

- "最佳适应"和"首次适应"是两回事：最佳是"找最小的够用块"，首次是"从头找第一个够用块"；切完剩下的边角，最佳的最小。
- "碎片"和"空闲块"是两回事：碎片是小到没法用的空闲块；大块空闲还能用，不是碎片——块小到"谁也用不上"才叫碎。
- "最坏适应"和"最佳适应"是两回事：最坏专挑最大块切，剩下还是大块、最不容易碎；最佳专挑最小够用块切，剩下全是小边角、最容易碎——两个极端，别搞反。

最佳适应每次都切最小的够用块，剩的边角最小、最容易变成碎片。

【坑位】

"最佳"听着最美好、最不容易出问题——恰恰相反，"最佳"是碎片之王；最坏反而最不碎。

最容易踩的坑是被名字骗了："最佳适应"听起来像最优解，直觉上觉得它最不容易产生碎片。其实它的"最佳"指的是"最省料"——每次都找刚好够用的最小块，剩下的边角料一次比一次小，最后全是没人要的小碎块。正确做法：别看名字看切法——谁切完剩的边角最小，谁就是碎片之王。

【套路】

- 第一步，想切法：拿到进程需求，每种算法会挑哪块下刀。
- 第二步，看剩下：切完剩下的边角有多大。
- 第三步，比碎的难易：边角越小越容易碎——最佳适应剩的边角最小。

挑哪块、剩多小、谁最碎——最佳适应剩最小，碎得最狠。

【类比】

像裁缝裁布。最佳适应是每次都找"刚好够用"的布头裁，剩下的全是小布条（碎片）；最坏适应是找最大块布裁，剩下的还是一大块好布；首次适应从头找、剩下的块还比较像样。裁缝最头疼的反而是那个"最节省"的——省出来的全是碎布头。

【口诀】 最佳找最小，碎片跟着走；切完剩小边，谁来都用不了。

【考频】 「动态分区」在 2009–2022 年解析中出现 5 次 · 中频

第 33 题 · OSI 第 5 层功能（正确答案：C）

题干与选项：

OSI 参考模型的第 5 层（自下而上）完成的主要功能是（ ）。

- A. 差错控制
- B. 路由选择
- C. 会话管理
- D. 数据表示转换

题意概括：OSI 参考模型从下往上数第 5 层，问它主要完成什么功能。

入手点： 先把七层从下往上数一遍：1 物理、2 数据链路、3 网络、4 传输、5 会话、6 表示、7 应用；第 5 层就是会话层。

题干专有名词从零解释：

- 先想"OSI 参考模型"：把网络通信分七层的一个标准框架。
- 再想"物理层"：传比特；"数据链路层"是相邻结点间传帧、差错控制；"网络层"是路由选择；"传输层"是端到端可靠传；"会话层"是管会话的建立、维持、断开；"表示层"是数据格式转换、加密压缩；"应用层"是给应用程序用的服务。

考查知识点：

- 先想第 5 层是会话层，管的是"会话"：两个应用进程之间一次通信的建立、维持、断开，还有会话过程中的同步（比如传一半断了，从哪个同步点接着传）。
- 再想差错控制是数据链路层的事，路由选择是网络层的事，数据表示转换是表示层的事——这三个都是别的层的活。

正确选项为什么对： 第 5 层是会话层，主要功能是会话管理，选 C。

错误选项为什么错＋举例：

- A（差错控制）：那是第 2 层数据链路层的事（相邻结点间传帧、检错纠错）。
- B（路由选择）：那是第 3 层网络层的事（分组走哪条路）。
- D（数据表示转换）：那是第 6 层表示层的事（格式、编码、加密压缩）。

笔记本一句话： 第 33 题＋OSI 七层从下往上：物、链、网、传、会、表、应，第 5 层会话管会话。

【图像】



OSI 七层从下往上：第 5 层会话层被标亮，管会话的建立、维持、断开

【定位】 计算机网络 · 第 1 章计算机网络体系结构 · OSI 参考模型七层功能

【易混对比】

- "会话层"和"传输层"是两回事：传输层管"送得到"（端到端可靠传），会话层管"这次谈话"（一次会话的建立、维持、断开）——一个管送货，一个管开会。
- "表示层"和"会话层"是两回事：表示层管"话怎么说"（数据格式转换、加密、压缩），会话层管"谈话这件事本身"（什么时候开始、什么时候结束、中途断了从哪个同步点接着来）。
- "数据链路层"和"会话层"是两回事：数据链路层管"相邻两站之间传帧不出错"（差错控制），会话层管"两个应用进程之间整场会话"。

第 5 层是会话层，只干三件事：会话的建立、维持、断开。

【坑位】

看到"差错控制"别顺手选——那是第 2 层数据链路层的活，不是第 5 层。

最容易踩的坑，是把"差错控制"当成会话层的功能。"差错"听起来很"通信"，但在 OSI 里，差错控制明确归第 2 层数据链路层（相邻结点之间传帧、检错纠错）。路由选择归第 3 层网络层，数据表示转换归第 6 层表示层——这三个都是别的层的活。正确做法：先从下往上数出第 5 层=会话层，再想会话层只管"会话"三件事，别的词再诱人也不选。

【类比】

像打电话：先拨号接通（建立会话），聊的过程中保持通话不断（维持），聊完挂电话（断开）；中途要是断线了，还知道从哪句话接着说（同步点）。会话层就是管这场"通话"本身的，不管声音传得清不清楚——那是别的事。

【口诀】 物链网传会表应，五层会话管开会。

【考频】 「OSI」在 2009–2022 年解析中出现 9 次 · 中频

第 34 题 · 100BaseT 传输介质（正确答案：A）

题干与选项：

100BaseT 快速以太网使用的导向传输介质是（ ）。

- A. 双绞线
- B. 单模光纤
- C. 多模光纤
- D. 同轴电缆

题意概括：100BaseT 快速以太网用的导向传输介质是什么。

入手点： 先拆名字：100 是速率 100Mb/s，Base 是基带，T 就是 Twisted（双绞线）的首字母——名字里已经写了答案。

题干专有名词从零解释：

- 先想"快速以太网"：100Mb/s 的以太网。
- 再想"导向传输介质"：信号沿着线走的介质，和无线相对。
- 再想"双绞线"：两根铜线拧在一起，抗干扰。
- 再想"单模光纤"：只传一种模式的光，传得远；"多模光纤"是传多种模式的光，传得近一些；"同轴电缆"是中间一根铜芯、外面包屏蔽层的电缆。

考查知识点：

- 先想 100BaseT 的 T 就是双绞线（Twisted Pair），100BaseTX 用的是 5 类双绞线。
- 再想 F 结尾的才是光纤（100BaseFX 用光纤）。
- 再想同轴电缆是 10Base5、10Base2 那个年代的东西（名字里的 5 和 2 就是同轴段长度），100BaseT 不用它。

正确选项为什么对： 100BaseT 的 T 就是双绞线，选 A。

错误选项为什么错＋举例：

- B（单模光纤）：光纤是 F 结尾的型号（100BaseFX）用的，不是 T。
- C（多模光纤）：同样是光纤，和 T 不沾边。
- D（同轴电缆）：那是 10Base5、10Base2 的老型号，100BaseT 不用。

笔记本一句话： 第 34 题＋100BaseT 的 T 就是双绞线；F 才是光纤。

【图像】



100BaseT 拆名字：100 是速率、Base 是基带、T 是双绞线 Twisted

【定位】 计算机网络 · 第 2 章物理层 · 传输介质与以太网命名规则

【易混对比】

- "100BaseT"和"100BaseFX"是两回事：尾巴字母不一样，介质就不一样——T 是双绞线（Twisted），F 是光纤（Fiber）。
- "双绞线"和"同轴电缆"是两回事：双绞线是两根铜线拧在一起（现在用的网线），同轴电缆是中间一根铜芯、外面包屏蔽层（10Base5、10Base2 那个年代的老型号）。
- "单模光纤"和"多模光纤"是两回事：都是光纤，区别在传多远——单模传得更远；但它俩都是 F 型号用的，跟 T 不沾边。

看尾巴字母定介质：T 是双绞线，F 才是光纤。

【坑位】

别被"100M 很快、肯定用光纤"的感觉带偏——名字里没 F 就别选光纤。

最容易踩的坑，是觉得速率 100Mb/s 听起来很快，"肯定用光纤"。但 100BaseT 的 T 就是 Twisted（双绞线）的首字母，名字已经把答案写出来了；光纤是 F 结尾的型号（比如 100BaseFX）才用的。正确做法：拆名字——100 是速率、Base 是基带、T 是介质代号，对号入座，不脑补。

【套路】

- 第一步，拆名字：100BaseT 拆成 100（速率 100Mb/s）、Base（基带传输）、T（介质代号）。
- 第二步，认尾巴：T = Twisted Pair 双绞线；F = Fiber 光纤。
- 第三步，排除老古董：同轴电缆只出现在 10Base5、10Base2 里，100M 的型号不用它。

拆名字、认尾巴：T 是双绞线，F 是光纤。

【类比】

像看车牌认车籍：车牌尾巴带"电"字的基本是电车，不用打开引擎盖看。100BaseT 名字尾巴的 T 就是它的"车籍章"——双绞线，瞟一眼尾巴就能定。

【口诀】 百兆 T 线不用猜，T 是双绞 F 光纤。

【考频】 「100BaseT」在 2009–2022 年解析中出现 2 次 · 低频

第 35 题 · 滑动窗口 3 比特编号、发送窗口 5、接收窗口最大（正确答案：B）

题干与选项：

对于滑动窗口协议，若分组序号采用 3 比特编号，发送窗口大小为 5，则接收窗口最大是（）。

- A. 2
- B. 3
- C. 4
- D. 5

题意概括：滑动窗口协议，分组序号用 3 比特编号，发送窗口是 5，问接收窗口最大是多少。

入手点： 序号只有 8 个（0 到 7），转一圈就重用；发送窗口和接收窗口加起来不能超过序号总数，不然新一轮的序号和旧一轮撞车，分不清。

题干专有名词从零解释：

- 先想"滑动窗口协议"：发送方一次发好几个分组，不用等一个确认再发下一个。
- 再想"分组序号"：给每个分组编的号，3 比特就是 0 到 7 一共 8 个。
- 再想"发送窗口"：发送方一次能发还没确认的分组数。
- 再想"接收窗口"：接收方一次能收、能缓存的分组数。
- 再想"序号重用"：序号用完了转回来再用。

考查知识点：

- 先想序号总数：3 比特编号，序号一共 2^3=8 个。
- 再想为什么不能超过：发送窗口 5 个，接收窗口要是也大，两边加起来超过 8，接收方就分不清"这是新一轮的 0 号还是旧一轮的 0 号"。所以发送窗口加接收窗口不能超过 8，接收窗口最大是 8−5=3。
- 小例子：序号 0 到 7，发送窗口 5（比如 0 到 4），接收窗口 3（比如 5 到 7），加起来正好 8，转一圈不撞车；接收窗口要是 4，加起来 9，就撞了。

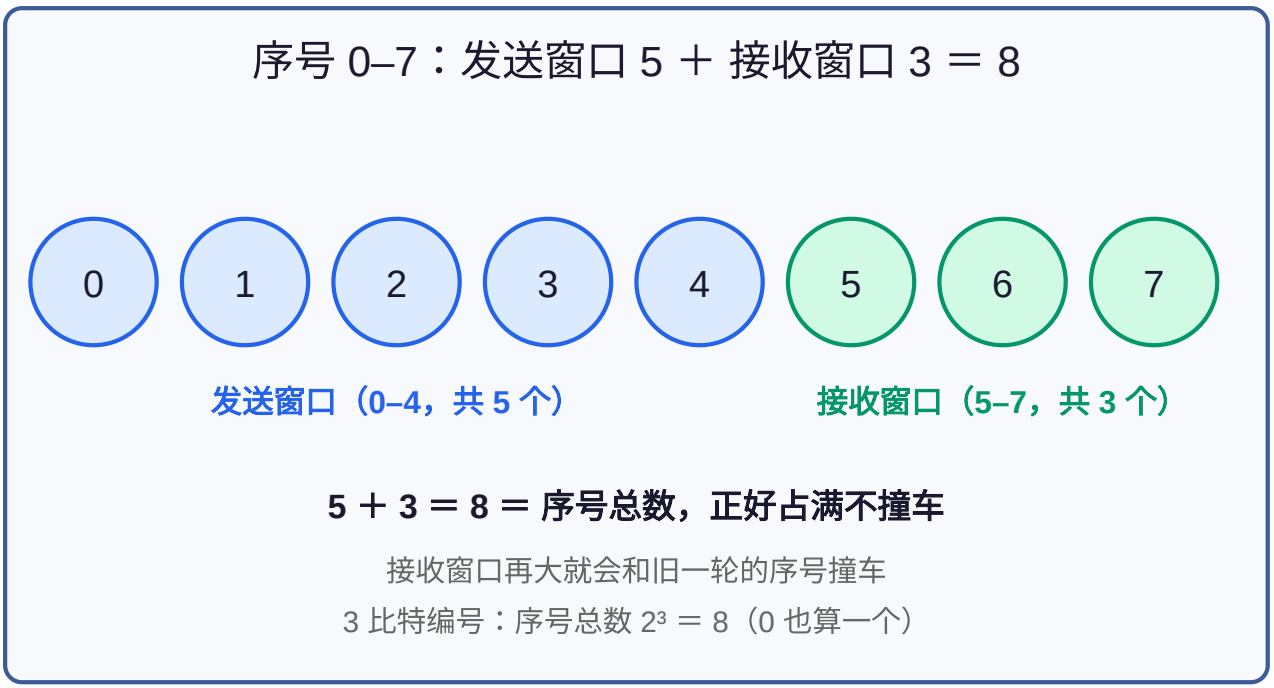
正确选项为什么对： 8−5=3，选 B。

错误选项为什么错＋举例：

- A（2）：8−5 算成了 2，或者把序号总数当成了 7（忘了 0 也算一个）。
- C（4）：那是选择重传里接收窗口单独的上限（2^(3−1)=4），但本题发送窗口已经占了 5，两个加起来 9 超了 8，不行。
- D（5）：和发送窗口一样大，加起来 10，撞车更严重。

笔记本一句话： 第 35 题＋滑动窗口防撞车：发送加接收不能超过序号总数 8，8−5=3。

【图像】



序号 0 到 7 转圈：发送窗口 5 个加接收窗口 3 个正好占满 8 个序号

【定位】 计算机网络 · 第 3 章数据链路层 · 滑动窗口协议的序号空间

【易混对比】

- "发送窗口加接收窗口 ⩽ 序号总数"和"选择重传接收窗口 ⩽ 2^(n−1)"是两回事：前者是通用防撞车规则（两边合起来不能超），后者是选择重传单独的上限；本题发送窗口已经给定，用前者直接减。
- "序号总数"和"最大序号"是两回事：3 比特编号，序号是 0 到 7，一共 2^3=8 个——0 也占一个，不是 7。
- "发送窗口"和"接收窗口"是两回事：发送窗口是"一次能发多少还没确认的"，接收窗口是"一次能收多少、能缓存多少"；本题发送窗口固定为 5，求接收窗口最大能放多大。

防撞车铁律：发送窗口＋接收窗口 ⩽ 序号总数，接收窗口最大＝8−5＝3。

【坑位】

最容易把序号总数算成 7（忘了 0 也算一个），或者把选择重传的 4 直接搬过来。

坑一是序号总数：3 比特是 0、1、2、3、4、5、6、7 一共 8 个，漏掉 0 就算成 7，后面全错。坑二是看到"接收窗口最大"就套选择重传的 4（2^(3−1)）——那是"接收窗口单独"的上限；本题发送窗口已经占了 5，两边加起来 5+4=9 超过 8，新一轮的序号会和旧一轮撞车，接收方分不清"这是新一轮的 0 号还是旧一轮的 0 号"。正确做法：先算序号总数 8，再用 8 减发送窗口 5，得 3。

【套路】

- 第一步，算序号总数：n 比特编号，序号总数 = 2^n（含 0），本题 2^3=8。
- 第二步，列防撞车不等式：发送窗口＋接收窗口 ⩽ 序号总数。
- 第三步，代入求解：接收窗口最大 = 8 − 5 = 3。

先算 2^n，再列"发送＋接收 ⩽ 序号总数"，减一下就出答案。

【类比】

像停车场只有 8 个车位（0 到 7 号）：发出去的车占了 5 个车位还没回来，剩下的最多再停 3 辆。要是硬塞第 4 辆，就得把用过的车牌号重用——车回来一看，根本分不清是新车还是旧车。

【口诀】 序号总数先算清，发送接收加起不能超。

【考频】 「滑动窗口」在 2009–2022 年解析中出现 9 次 · 中频

第 36 题 · CSMA／CD 100Mbps 最小帧 128B、单向传播延时最大（正确答案：B）

题干与选项：

假设一个采用 CSMA/CD 协议的 100Mbps 局域网，最小帧长是 128B，则在一个冲突域内两个站点之间的单向传播延时最多是（）。

- A. 2.56μs
- B. 5.12μs
- C. 10.24μs
- D. 20.48μs

题意概括：CSMA／CD 局域网，最小帧长 128B，问一个冲突域里两个站点之间的单向传播延时最多是多少。

入手点： 冲突要能被发现：帧还没发完，冲突信号就得传回来；所以帧的发送时间至少是单向传播延时的 2 倍；最小帧长就是按这个"刚好能发现"算出来的。

题干专有名词从零解释：

- 先想"CSMA／CD"：先听再说、边说边听、冲突了停下重发的一套规矩。
- 再想"冲突域"：几个站点共用一条信道、会互相冲突的范围。
- 再想"最小帧长"：为了让冲突能被发现，帧最短不能短于某个长度，这里是 128B。
- 再想"单向传播延时"：信号从一端走到另一端的时间，记作 τ。
- 再想"发送时间"：把一帧推出去要花的时间。

考查知识点：

- 先想最小帧换成比特：128B 等于 128×8=1024bit。
- 再想发送时间：速率按 100Mbps 算，发送时间等于 1024 ÷ 100000000 = 10.24μs。
- 再想冲突检测要求：发送时间 ⩾ 2τ（帧发完之前，冲突得传回来），所以 τ ⩽ 10.24÷2=5.12μs。问"最多"，就是 5.12μs。
- 小例子：帧要是只有 64B，发送时间 5.12μs，τ 最多 2.56μs——帧越短，允许的传播延时越小，冲突域就得越小。

正确选项为什么对： 1024bit ÷ (2×100Mbps) = 5.12μs，选 B。

错误选项为什么错＋举例：

- A（2.56μs）：那是按 64B 算出来的，或者把 10.24 又除了一次 2（多除了一次）。
- C（10.24μs）：那是整帧的发送时间，忘了除以 2——单向传播延时是往返的一半。
- D（20.48μs）：把发送时间乘了 2，方向反了。

笔记本一句话： 第 36 题＋CSMA／CD 冲突能发现：发送时间 ⩾ 2τ，128B 在 100Mbps 下发 10.24μs，τ 最多 5.12μs。

【图像】



冲突要能被发现：帧的发送时间至少罩住信号一个来回 2τ

【定位】 计算机网络 · 第 3 章数据链路层 · CSMA/CD 与最小帧长

【易混对比】

- "单向传播延时 τ"和"往返传播延时 2τ"是两回事：信号走过去是 τ，一个来回是 2τ；冲突要能被发现，帧的发送时间至少得罩住一个来回。
- "发送时间"和"传播延时"是两回事：发送时间是"把一帧推出去花多久"（帧长÷速率），传播延时是"信号在路上走多久"（看线多长）——一个看帧，一个看线。
- "最小帧长"和"最大帧长"是两回事：最小帧长是为"冲突能被发现"设的下限；帧太短、发得太快，冲突信号还没传回来就发完了，等于白发。

冲突能被发现的铁律：帧的发送时间 ⩾ 2τ；128B 在 100Mbps 下发 10.24μs，τ 最大 5.12μs。

【坑位】

最容易在"除以 2"上翻车：10.24μs 是整帧的发送时间，答案要的是单向延时，必须再除以 2。

这题的坑全在"2"上。坑一：把 10.24μs（整帧发送时间）直接当答案，忘了单向延时是往返的一半；坑二：多除一次 2，算出 2.56μs（那是按 64B 算的结果）；坑三：方向反了去乘 2，得 20.48μs。为什么必须除以 2？因为冲突信号要"走过去、再传回来"，帧的发送时间得罩住这一个来回，所以 τ = 发送时间 ÷ 2。正确做法三步：字节转比特（128×8=1024）、算发送时间（1024÷100Mbps=10.24μs）、除以 2 得 5.12μs。

【套路】

- 第一步，字节转比特：128B × 8 = 1024bit。
- 第二步，算发送时间：1024 ÷ 100Mbps = 10.24μs。
- 第三步，除以 2：单向传播延时最大 = 10.24 ÷ 2 = 5.12μs。

转比特、算发送时间、除以 2——三步走完就是答案。

【类比】

像对山谷喊话确认：你喊一嗓子（发帧），要听到回声（冲突传回来）才知道对面有人。你喊的时间（发送时间）至少得够声音"去一趟、回来一趟"，所以单程时间最多是你喊的一半。

【口诀】 发送时间罩来回，单向延时除以二。

【考频】 「CSMA/CD」在 2009–2022 年解析中出现 6 次 · 中频

第 37 题 · ／20 划分子网最小子网可分配 IP 数（正确答案：B）

题干与选项：

若将 101.200.16.0/20 划分为 5 个子网，则可能的最小子网的可分配 IP 地址数是（）。

- A. 126
- B. 254
- C. 510
- D. 1022

题意概括：把 101.200.16.0／20 划分成 5 个子网，问其中最小的子网能有多少个可分配 IP 地址。

入手点： 先想"划分成 5 个"不一定是等分；一个地址块二分、二分、再二分，数到 5 个子网时停下，最小的那个就是答案。

题干专有名词从零解释：

- 先想"子网"：把一个大地址块切成几个小块。
- 再想"／20"：前 20 位是网络号，后 12 位是主机号，一共 4096 个地址。
- 再想"可分配 IP 地址"：主机号全 0 是网络地址、全 1 是广播地址，这两个不能分给主机，要减掉 2 个。
- 再想"二分"：把一个块劈成两个一半大的块，掩码加 1 位。

考查知识点：

- 先想／20 的块一共 4096 个地址。劈一次：2 个 ／21；再劈一个 ／21：变成 1 个 ／21 加 2 个 ／22，一共 3 个；再劈一个 ／22：1 个 ／21、1 个 ／22、2 个 ／23，一共 4 个；再劈一个 ／23：1 个 ／21、1 个 ／22、1 个 ／23、2 个 ／24，一共 5 个，停。
- 这时最小的子网是 ／24，主机位 8 位，可分配 2^8−2=254 个。
- 注意：如果等长划分（全 ／23），每个子网都是 510 个，那是"每个子网"不是"最小子网"；题问的是最小的那个，所以是 254。

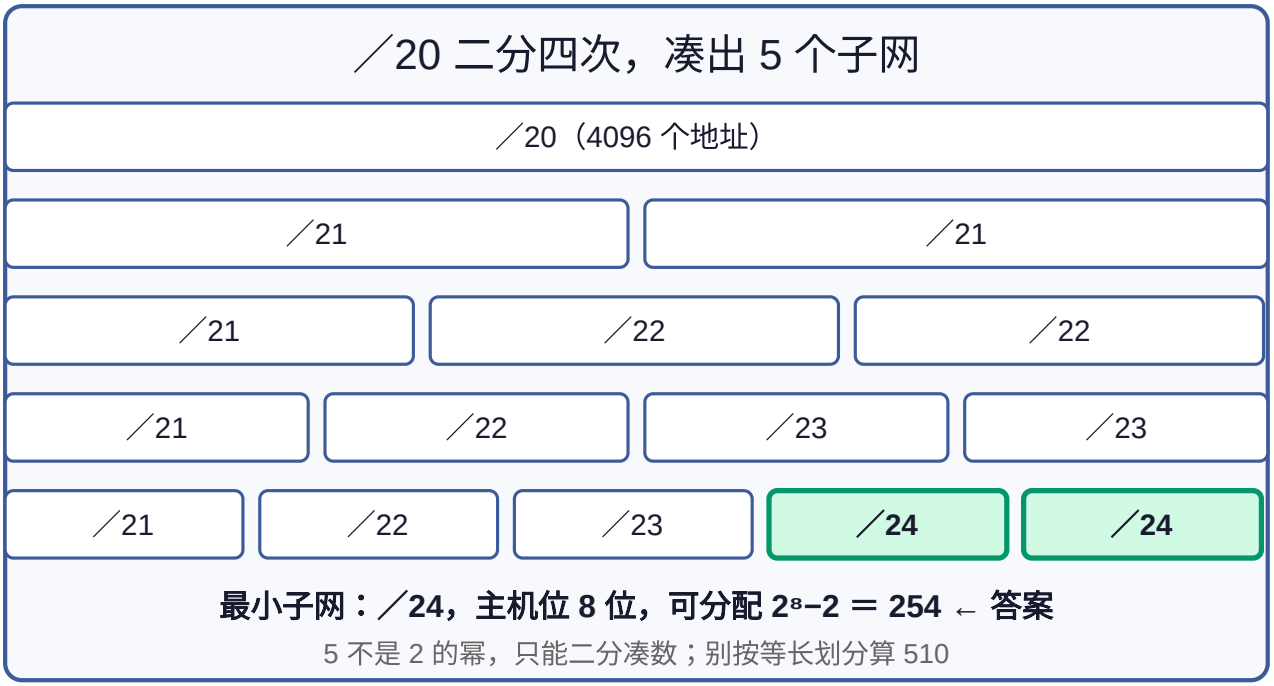
正确选项为什么对： 二分到 5 个停，最小是 ／24，2^8−2=254，选 B。

错误选项为什么错＋举例：

- A（126）：那是 ／25 的数（2^7−2），多劈了一次。
- C（510）：那是等长划分全 ／23 的数，题问的是最小子网，不是每个子网。
- D（1022）：那是 ／22 的数（2^10−2），劈少了。

笔记本一句话： 第 37 题＋／20 劈成 5 个子网：二分到 5 个停，最小是 ／24，2^8−2=254。

【图像】



／20 二分四次凑出 5 个子网：最小的是 ／24

【定位】 计算机网络 · 第 4 章网络层 · 子网划分与可分配地址数

【易混对比】

- "划分成 5 个子网"和"等分成 5 个"是两回事：5 不是 2 的幂，没法等分；只能一个块二分、再二分，数到 5 个停——子网有大有小。
- "可分配 IP 数"和"地址总数"是两回事：子网里主机号全 0（网络地址）和全 1（广播地址）不能分给主机，可分配数 = 2^主机位数 − 2。
- "最小子网"和"每个子网"是两回事：题问的是 5 个里最小的那个（／24，254 个），不是等长划分下每个子网（／23，510 个）。

二分到 5 个停，最小是 ／24，可分配 2^8−2=254 个。

【坑位】

最容易按"等长划分"算成 510——题问的是最小子网，不是每个子网。

头号坑是想当然"5 个子网就每个一样大"，然后按 ／23 算出 510。5 不是 2 的次方，根本切不出 5 个等大的块；只能二分：／20 劈成 2 个 ／21，再劈一个得 3 个，再劈得 4 个，再劈得 5 个——这时最小的块是 ／24。坑二是忘了减 2（网络地址和广播地址不能分配）。正确做法：老老实实二分、数到 5 个停，找到最小的 ／24，主机位 8 位，2^8−2=254。

【套路】

- 第一步，确认不能等分：5 不是 2 的幂，只能二分凑数。
- 第二步，二分数到 5：／20 劈成 2 个 ／21，再劈得 3 个、4 个、5 个，最小是 ／24。
- 第三步，算可分配数：主机位 8 位，2^8−2=254。

二分凑到 5 个停，找最小块，2^主机位减 2。

【类比】

像切蛋糕招待 5 位客人：蛋糕只能对半切，切 2 次得 4 块不够，再把其中一块对半切，凑出 5 块——这 5 块有大有小。问"最小那块多大"，就看最后那次对半切出来的小块，别拿"平均每块"来答。

【口诀】 二分凑够五个块，最小那块减二算。

【考频】 「子网划分」在 2009–2022 年解析中出现 9 次 · 中频

第 38 题 · TCP 快速重传重发时刻（正确答案：C）

题干与选项：

某客户通过一个 TCP 连接向服务器发送数据的部分过程如题 38 图所示。客户在 t0 时刻第一次收到确认序列号 ack_seq = 100 的段，并发送序列号 seq = 100 的段，但发生丢失。若 TCP 支持快速重传，则客户重新发送 seq = 100 段的时刻是（）。

- A. t1
- B. t2
- C. t3
- D. t4

> 注：本题题干依赖图片（TCP 过程图），图片未能直接查看全部细节，本题标注"部分核对"。图中过程（服务器三次重复发 ack_seq=100 在 t1、t2、t3，t4 时刻 seq=100 段超时）按可见信息写；答案按答案表写，图中细节以原卷为准。

题意概括：客户通过 TCP 连接向服务器发数据，中间一段丢了，服务器连着发了几次重复确认，问客户在哪个时刻重新发送丢失的那段。

入手点： 快速重传的规矩是"收到 3 个重复确认就立即重发，不等超时"；数重复确认是第几个，数到 3 的那个时刻就是重发时刻。

题干专有名词从零解释：

- 先想"TCP 连接"：可靠传输的连接，发出去的每段都要对方确认。
- 再想"确认序号（ack_seq）"：告诉对方"我下次想收第几号字节"。
- 再想"重复确认"：同一个确认序号连着发好几次，意思是"我还是卡在这一号，后面的都到了但缺这一段"。
- 再想"快速重传"：收到 3 个重复确认就认定丢了，马上重发。
- 再想"超时重传"：等计时器到时间了还没确认才重发，比快速重传慢。

考查知识点：

- 先想图里的过程：服务器先发 ack_seq=100，客户随后发 seq=100（丢了）、seq=200、seq=300、seq=400；服务器每收到一段不是 100 的，就重复发 ack_seq=100——t1、t2、t3 一共三次。
- 再想快速重传的触发点：客户收到第 3 个重复确认（t3）时，按快速重传规矩立即重发 seq=100，不用等到 t4 的超时。
- 再想 t4 是什么：t4 是超时时刻，是"如果没有快速重传、傻等计时器"的时刻，不是快速重传的时刻。

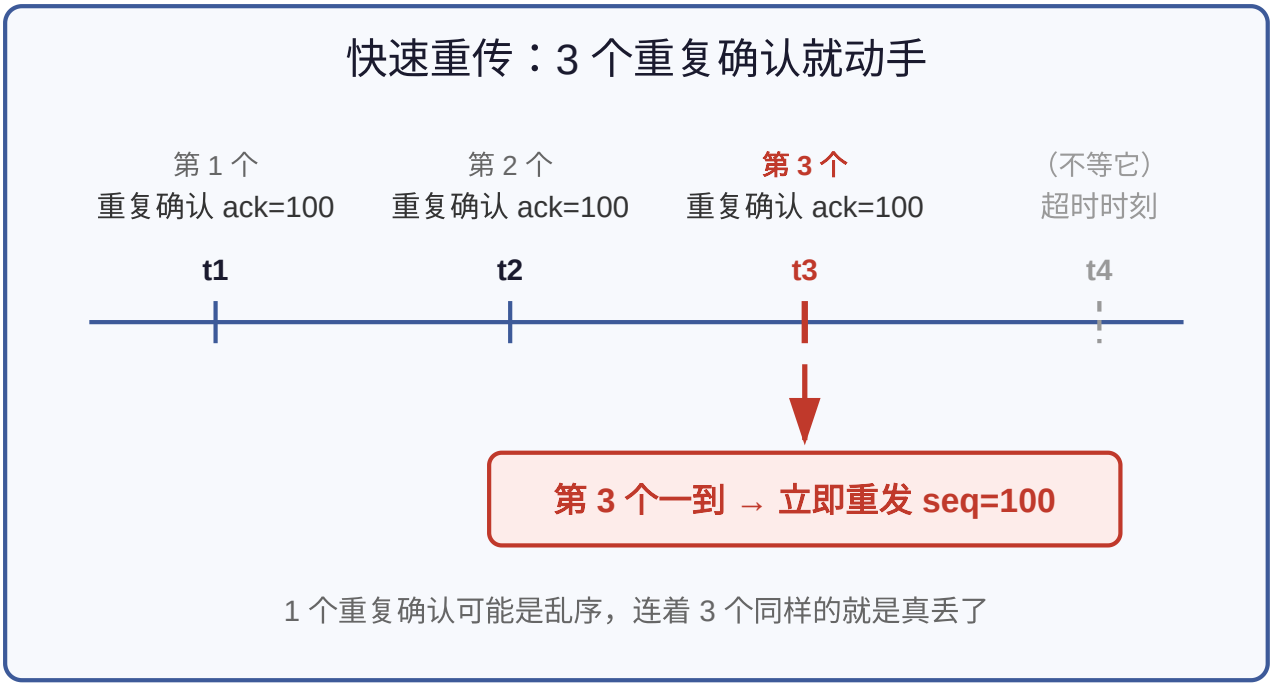
正确选项为什么对： 第 3 个重复确认在 t3，收到就重发，选 C。

错误选项为什么错＋举例：

- A（t1）：第 1 个重复确认时就重发——太早了，快速重传要 3 个重复确认才触发，1 个可能是乱序不是丢失。
- B（t2）：第 2 个重复确认时重发——还是不够 3 个。
- D（t4）：那是超时重传的时刻；题干说了支持快速重传，就不用傻等到超时。

笔记本一句话： 第 38 题＋TCP 快速重传：3 个重复确认就重发，t3 就是第 3 个，不等 t4 超时。

【图像】



t1、t2、t3 三个重复确认：第 3 个一到就重发，不等 t4 超时

【定位】 计算机网络 · 第 5 章传输层 · TCP 可靠传输：快速重传

【易混对比】

- "快速重传"和"超时重传"是两回事：快速重传是"收到 3 个重复确认就动手"，超时重传是"傻等计时器到点"；本题支持快速重传，就不用等到 t4。
- "重复确认"和"普通确认"是两回事：重复确认是同一个确认号连发好几次，意思是"我还卡在这儿，后面的都到了、就缺这一段"；普通确认是确认号往前走的。
- "第 3 个重复确认"和"第 1 个"是两回事：1 个重复确认可能是乱序（后面的段先到了），连着 3 个同样的确认号，基本就是丢了——所以规矩定在 3 个。

3 个重复确认一到就重发：t3 是第 3 个，不等 t4 超时。

【坑位】

最容易选 t4——那是"没有快速重传、傻等超时"的时刻，不是快速重传的时刻。

这题的坑是 t4 看起来"最保险"：等到超时再重发，总不会错吧？错了——题干明确支持快速重传，规矩就是"不等超时"。为什么定 3 个？因为 1、2 个重复确认可能是网络乱序（后面的段先到了），连着 3 个同样的确认号，说明接收方真的卡在这一段、后面的段其实都到了，这时再等超时就是浪费时间。正确做法：数重复确认是第几个，t1 第 1 个、t2 第 2 个、t3 第 3 个——数到 3 就动手，重发时刻是 t3。

【套路】

- 第一步，认重复确认：同一个 ack_seq 连发几次，就是几次重复确认（本题 t1、t2、t3）。
- 第二步，数到 3：第 3 个重复确认到达的时刻，就是重发时刻。
- 第三步，排除超时：t4 是超时时刻，有快速重传就不等它。

数重复确认到 3 就动手，不等超时。

【类比】

像等快递：快递员连着三天给你打电话都说"还是缺那一件"（重复确认），你第三天就该补发了，不用等到第七天系统自动判定丢件（超时）才行动。

【口诀】 重复确认数到三，立即重发不等超时。

【考频】 「快速重传」在 2009–2022 年解析中出现 36 次 · 高频

第 39 题 · TCP 第三次握手确认序号（正确答案：D）

题干与选项：

若主机甲主动发起一个与主机乙的 TCP 连接，甲、乙选择的初始序列号分别为 2018 和 2046，则第三次握手 TCP 段的确认序列号是（）。

- A. 2018
- B. 2019
- C. 2046
- D. 2047

题意概括：主机甲主动向乙发起 TCP 连接，两边选的初始序号都给了，问第三次握手里确认序号是多少。

入手点： 三次握手一步一步数：第一次甲发 SYN 带自己的初始序号，第二次乙回 SYN 带自己的初始序号、确认号是甲的初始序号加 1，第三次甲回确认号是乙的初始序号加 1。

题干专有名词从零解释：

- 先想"三次握手"：TCP 建连接的三步。
- 再想"初始序号（ISN）"：连接开始时双方各自随机选的起始序号。
- 再想"SYN"：请求建连接的段。
- 再想"确认序号（ack）"：告诉对方"我下次想收第几号"。
- 再想"序号（seq）"：每段数据的第一个字节的编号。

考查知识点：

- 先想第一次：甲→乙，SYN，seq=2018。
- 再想第二次：乙→甲，SYN，seq=2046，ack=2018+1=2019（确认甲的 SYN）。
- 再想第三次：甲→乙，ACK，seq=2019，ack=2046+1=2047（确认乙的 SYN）。
- 问的是第三次握手的确认序号，就是 2047。规律：确认号永远是"对方初始序号加 1"（SYN 占一个序号）。

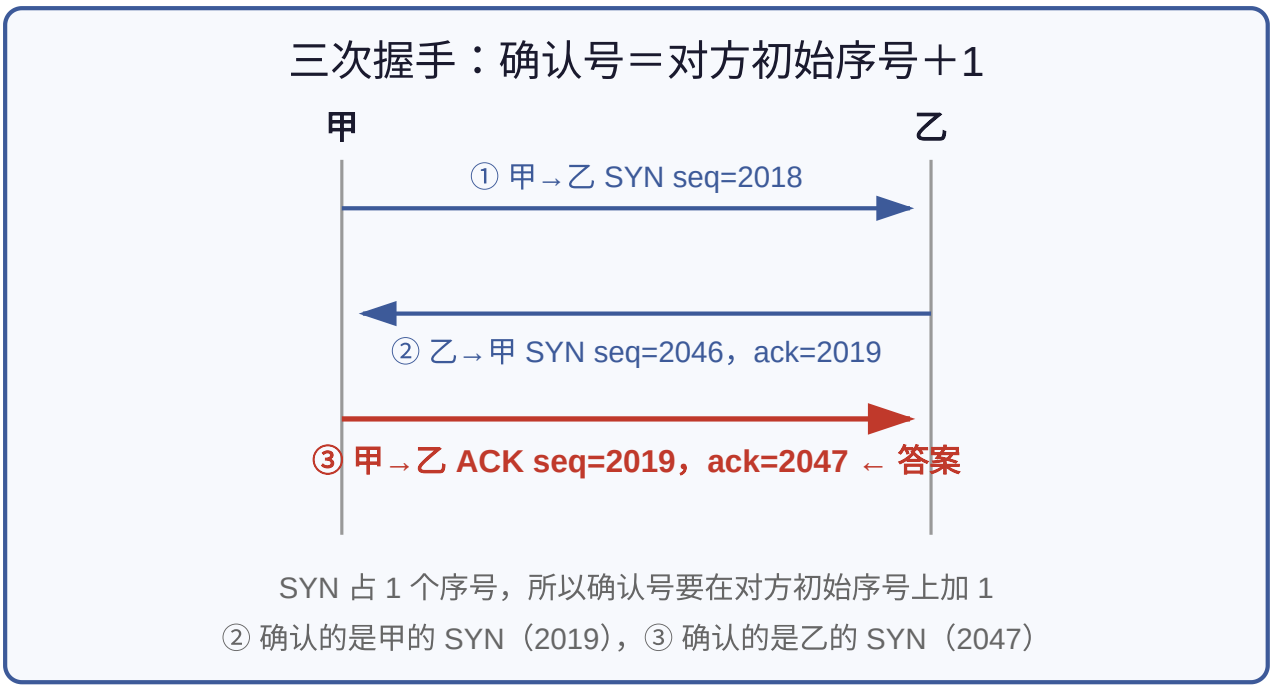
正确选项为什么对： 第三次握手的确认号是乙的初始序号加 1，2046+1=2047，选 D。

错误选项为什么错＋举例：

- A（2018）：那是甲自己的初始序号，不是确认号。
- B（2019）：那是第二次握手的确认号（确认甲的 SYN），不是第三次的。
- C（2046）：那是乙自己的初始序号，确认号要加 1。

笔记本一句话： 第 39 题＋三次握手确认号：第三次是乙的初始序号加 1，2046+1=2047。

【图像】



三次握手：第三次的确认号是乙的初始序号加 1，等于 2047

【定位】 计算机网络 · 第 5 章传输层 · TCP 连接建立：三次握手

【易混对比】

- "确认序号"和"序号"是两回事：序号（seq）是"我这段从第几号开始"，确认序号（ack）是"我下次想收第几号"——一个说自己，一个盼对方。
- "第三次握手的确认号"和"第二次的"是两回事：第二次确认的是甲的 SYN（2018+1=2019），第三次确认的是乙的 SYN（2046+1=2047）——看清"确认的是谁的"。
- "初始序号"和"确认号"是两回事：初始序号是自己起的头（2046），确认号是对方的头加 1（2047）——差 1，别直接抄。

确认号永远是"对方初始序号＋1"：第三次确认的是乙的 SYN，2046+1=2047。

【坑位】

最容易直接抄 2046（乙的初始序号）当答案——确认号要加 1，因为 SYN 占一个序号。

这题的坑是"2046 看起来就是答案"：题干给了乙的初始序号 2046，手一快就选了。确认序号的意思是"我下次想收第几号"——乙发了 SYN（占 1 个序号），甲收到了，下次想收的就是 2046 的下一号，即 2047。坑二是拿第二次握手的 2019 来答——那是确认甲的 SYN 的，不是第三次。正确做法：先定"第几次握手"，再定"确认的是谁的 SYN"，最后"对方初始序号＋1"。

【套路】

- 第一步，定第几次：第三次握手，是甲发给乙的确认。
- 第二步，定确认谁：确认的是乙的 SYN（第二次握手里乙发来的）。
- 第三步，加 1：乙的初始序号 2046＋1=2047。

第几次、确认谁的、对方序号加 1——三步锁定。

【类比】

像回执签收：对方寄来编号 2046 的挂号信（SYN 占一个号），你回执上写"2047 号及以后的请继续寄"——回执号永远是"对方最后一个号＋1"，不是把对方的号原样抄回去。

【口诀】 三次握手看仔细，确认号取对方加一。

【考频】 「三次握手」在 2009–2022 年解析中出现 7 次 · 中频

第 40 题 · 网络应用模型叙述、错误项（正确答案：B）

题干与选项：

下列关于网络应用模型的叙述中，错误的是（）。

- A. 在 P2P 模型中，结点之间具有对等关系
- B. 在客户/服务器（C/S）模型中，客户与客户之间可以直接通信
- C. 在 C/S 模型中，主动发起通信的是客户，被动通信的是服务器
- D. 在向多用户分发一个文件时，P2P 模型通常比 C/S 模型所需的时间短

题意概括：关于网络应用模型的四句话，问哪句是错的。

入手点： 本题问"错误的是"，先分清两种模型：C／S 是客户围着服务器转，P2P 是结点之间对等；"客户之间直接通信"那句放在 C／S 里就可疑。

题干专有名词从零解释：

- 先想"客户／服务器模型（C／S）"：客户主动找服务器，服务器被动等。
- 再想"P2P 模型"：结点之间对等，谁都能当客户、谁都能当服务器。
- 再想"结点"：网络里的参与方。
- 再想"分发文件"：把一个文件发给很多用户。

考查知识点：

- 先想 A：P2P 里结点之间是对等的，没有谁高谁一等。对的。
- 再想 B：C／S 模型中客户与客户之间可以直接通信——不对。C／S 里客户只和服务器说话，客户之间不直接通信；想"客户之间传东西"得经过服务器中转，或者那是 P2P 的做法。
- 再想 C：C／S 里主动发起通信的是客户，被动通信的是服务器——定义就是这样。对的。
- 再想 D：向多用户分发一个文件时，P2P 通常比 C／S 快——因为每个拿到文件的用户又能当源头传给别人，源头越传越多；C／S 只有一个服务器往外发，带宽是瓶颈。对的。

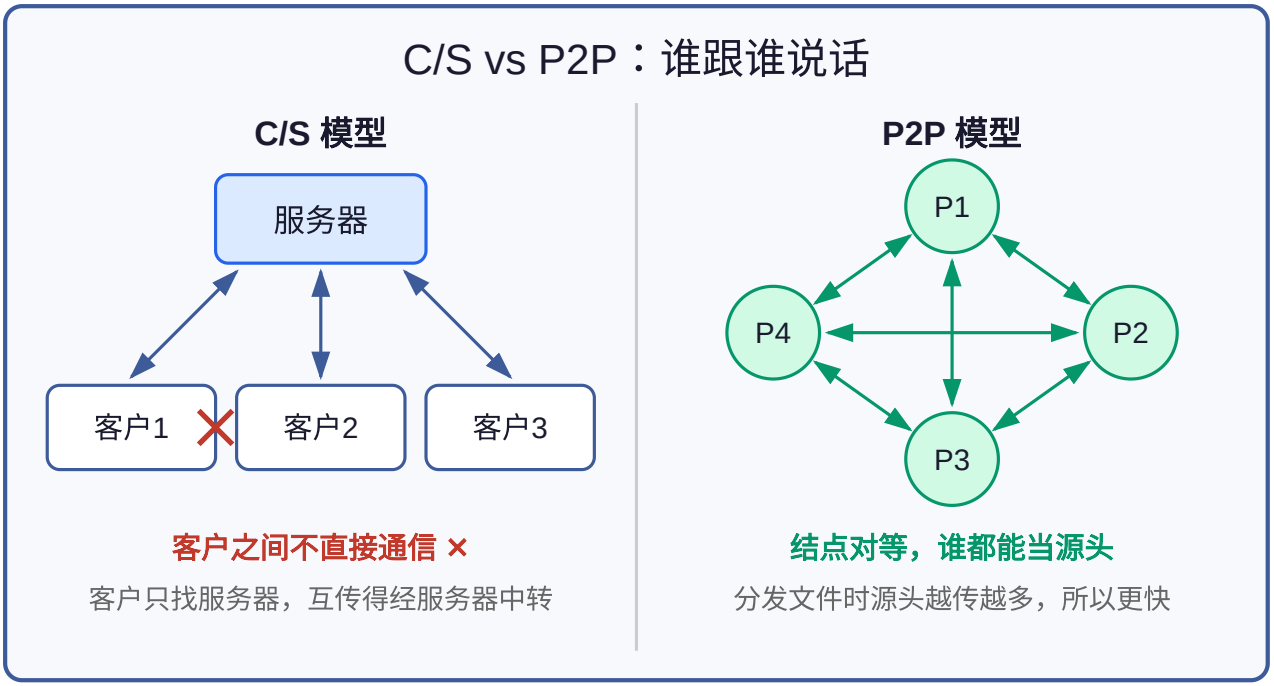
正确选项为什么对： 本题问"错误的是"，即这个说法本身不成立。B 把 P2P 的特点安到了 C／S 头上。C／S 里客户之间不直接通信，选 B。

错误选项为什么错＋举例：

- A：P2P 结点对等——对的说法。
- C：C／S 客户主动、服务器被动——对的说法。
- D：P2P 分发越传源头越多、所以快——对的说法。

笔记本一句话： 第 40 题＋C／S 客户之间不直接通信，客户只找服务器；P2P 才对等互传。

【图像】



C/S：客户只找服务器；P2P：结点之间对等互传

【定位】 计算机网络 · 第 6 章应用层 · 网络应用模型：C/S 与 P2P

【易混对比】

- "C/S"和"P2P"是两回事：C/S 是客户围着服务器转（客户只找服务器），P2P 是结点之间对等、谁都能当源头。
- "客户之间直接通信"放在 C/S 里和放在 P2P 里是两回事：在 P2P 里这是正常做法，在 C/S 里就是错的——C/S 里客户之间要传东西，得经服务器中转。
- "主动"和"被动"是两回事：C/S 里主动发起的是客户、被动等的是服务器；P2P 里没有固定的主动被动，结点角色随时换。

C/S 里客户只找服务器，客户之间不直接通信；"客户直接互传"是 P2P 的特点。

【坑位】

题问的是"错误的是"——别顺手把对的说法选了，先圈出"错"字再动手。

这题的坑不在知识，在题型："错误的是"三个字一晃就过，很多人按"找对的"习惯做题，直接选了个正确说法。B 为什么错？它把 P2P 的特点（客户之间直接通信）安到了 C/S 头上——C/S 模型里，客户只和服务器说话，客户之间不直接通信。正确做法：先圈"错误的是"，再逐句验：A（P2P 结点对等）对，C（C/S 客户主动、服务器被动）对，D（P2P 分发时源头越传越多、所以快）对，剩下 B 就是那个错的。

【类比】

C/S 像银行柜台：所有客户都排队找柜员办业务，客户之间不直接互相转账，都得经过柜台。P2P 像跳蚤市场：摊主之间直接交易，谁买到好货转手又能卖给别人，货源越传越多。

【口诀】 客户围着服务器，客户之间不传话。

【考频】 「C/S」在 2009–2022 年解析中出现 1 次 · 低频

附注

- 本册只含 2019 年 408 选择题第 1–40 题，不含综合应用题；升级版每题含完整题干与选项，可脱离原卷独立使用；讲解重心在方法、推理、易混点上。
- 每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。
- 题干问"错误的是"的互换题（第 4 节讲正确选项即那个错述为什么成立，第 5 节讲其余选项为什么是对的）：第 12、14、16、20、23、28、40 题。
- 语言风格：大白话为主，每个术语第一次出现配一句大白话解释，顺序按"从具体到抽象"（先讲生活例子、小数字、具体场景，再贴术语名）；讲解比 2022 版更深，考查知识点与错误选项两节把定义、适用条件、边界情况、易混对子讲到位；每题自成体系，不依赖其他题的解析，不预设前置概念。
- 答案表（逐题值）：
- 数据结构：第 1 题 B，第 2 题 B，第 3 题 C，第 4 题 A，第 5 题 C，第 6 题 A，第 7 题 D，第 8 题 C，第 9 题 B，第 10 题 D，第 11 题 B。
- 计组：第 12 题 C，第 13 题 A，第 14 题 D，第 15 题 D，第 16 题 D，第 17 题 B，第 18 题 C，第 19 题 B，第 20 题 C，第 21 题 A，第 22 题 D。
- 操作系统：第 23 题 B，第 24 题 C，第 25 题 C，第 26 题 B，第 27 题 C，第 28 题 B，第 29 题 C，第 30 题 B，第 31 题 A，第 32 题 C。
- 计网：第 33 题 C，第 34 题 A，第 35 题 B，第 36 题 B，第 37 题 B，第 38 题 C，第 39 题 D，第 40 题 B。

核对结果

- 双源比对（答案表级）：N诺《2019年（408）计算机学科专业基础综合试题》（https://noobdream.com/Practice/exam_solution/40114/）与夸克版《2019年计算机408统考真题解析》第 1 页答案表逐题比对，40/40 一致，差异 0 题；答案按一致值写，无自行改动。
- 第 36 题：题干速率以 N诺版 100Mbps 为准；夸克抄录的"10Mb/s"疑为预览小字误读（100 误读为 10）；按 100Mbps 算，单向传播延时 = 1024bit ÷（2×100Mbps）= 5.12μs，与两源答案 B 吻合；按 10Mb/s 算不出任何所列选项。
- 第 37 题：限定词以 N诺版"最小子网"为准；答案 B（254）两源一致；不确定标注取消。
- 第 23 题：答案 B 经 N诺佐证（与夸克放大核对一致）。
- 图题部分核对（2 题，图片未能直接查看全部细节，图中细节以原卷为准）：
- 第 5 题：AOE 网图题，活动 d 为 ②→④ 的边；b、c 两条边的具体起止顶点未看清。
- 第 38 题：TCP 过程图题，过程按可见信息写（服务器三次重复发 ack_seq=100 在 t1、t2、t3，t4 时刻 seq=100 段超时）。
- 第 31 题：地址结构表题，表格结构（页目录号 10 位、页号 10 位、页内偏移 12 位）已给出，虚拟地址 20501225H。
- 语言与结构自查：全文检索跳步词与跨题外指，检索结果：跳步词 0 处，跨题外指 0 处；每题自成体系，不依赖其他题的解析。